StarLink — Employee & Customer Communication Platform

Solution Architecture & Product Technical Design

Project	StarLink
Document	Solution Architecture & Product Technical Design
Version	2.2
Status	PROPOSED / ARCHITECTURE-ALIGNED FOR CTO & PROJECT HEAD REVIEW — CCS PRO + CY Brain compatibility integrated; production acceptance still requires load, security, privacy and failure tests.
Prepared for	Project Head / CTO, CoverYou
Prepared by	Engineering
Date	24 August 2026
Classification	Internal — CoverYou only
Supersedes	Solution Architecture v2.1 · v2.0 · v1.0 · StarLink Architecture V1, V2, V3 — retained as history
Change from v2.1	Reconciles StarLink with CCS PRO Conversation OS, Work Orchestrator, canonical IAM/Consent/Event/Audit and CY Brain command/assurance model; upgrades realtime for burst and multi-instance scale; adds omnichannel continuity, AI-assisted service, DPDP/privacy controls, queue/capacity/backpressure, event/outbox contracts, load/chaos acceptance and 500+ simultaneous-user scale envelope.
Related	CCS PRO Final Master Architecture v1.8.5 (supreme business/platform architecture) · CY Brain architecture + privacy control plane (command/intelligence/assurance) · NorthStar only as historical implementation learning, never runtime authority.

Table of contents

> ***How to read this.*** *A reviewer who has approved the product can start at* ***§14****; one seeing StarLink for the first time should read* ***§1–13*** *first. The customer journey this version was written to capture is* ***§21****; the chat-versus-ticketing recommendation is* ***§22****; after-hours and SLA are* ***§23****. Decisions needing your approval are* ***§39*** *(technology) and* ***§44*** *(business).*

---

1. Purpose of this document

This document exists to be reviewed, argued with, and signed off — or sent back. It is not an implementation log and not a feature catalogue.

It is written to make the following twenty-six questions answerable in a single sitting. The section that answers each is named, so the document can be read in the order the reader cares about rather than the order it was written.

#	Question	§
1	What is StarLink?	6
2	What problem are we solving?	3, 4
3	Who are the users?	7
4	Why do we need both employee and customer communication?	3.4
5	What can employees do?	9, 10
6	What can customers do?	9.4, 10
7	How does a customer reach the company?	**D-01 — undecided**
8	Does a customer contact a person, team, department or queue?	21.2, **D-04**
9	Who owns a customer conversation?	11.3
10	Can conversations be transferred?	9 (UC-E05), 11.4
11	Can multiple employees participate?	9 (UC-E08), 11.2
12	What happens when an employee is unavailable?	9 (UC-E07), **D-05**
13	What customer information is visible to employees?	**D-10 — undecided**
14	What is visible to the customer?	9.4, 11.7
15	What permissions are required?	24.3, 10.2
16	What happens on authentication / session expiry?	9 (UC-C05), 24.4
17	What happens to conversations when an employee leaves?	9 (UC-E07), 11.3
18	What happens when a conversation is closed or reopened?	21.4, 11.5
19	What notifications are required?	10.13, 27
20	Which actions require realtime?	20.2
21	What data is stored?	22

#	Question	§
22	What are the major system components?	16, 17
23	What are the major integrations?	34
24	What belongs to V1?	39
25	What is future scope?	40
26	What decisions require approval?	**42**
27	**What technology do we propose to build it with?**	**14**
28	**Why that technology, and what did we reject?**	**15, 38**
29	**How is the system structured internally?**	**16–19**
30	**How will components communicate?**	**20, 23**
31	**How will it be deployed and operated?**	**30–35**
32	**What did NorthStar teach us technically?**	**36**
33	**Which technology decisions need your approval?**	**39**
34	**What happens when a customer opens the website chat?**	**21.5**
35	**How does the system know whom to contact?**	**21.8**
36	**What if the preferred employee is unavailable?**	**21.9**
37	**What happens after working hours?**	**23.3**
38	**How are SLA and escalation handled?**	**23.5, 23.6**
39	**Is this a chat system or a ticketing system?**	**22.2**
40	**Who can see what — and what can a customer never see?**	**22.5, 27.16, 11.7**
41	**How does employee chat differ from customer communication?**	**21.4**

Rows 1–26 are the product contract carried from v1.0. Rows 27–33 are what version 2.0 added. Rows 34–41 are what version 2.1 adds — the customer journey, after-hours, SLA and the chat-versus-ticketing question.

Three of the product rows — 7, 13 and parts of 8 — are **business decisions this document deliberately does not answer.** They are recorded in §44 with options and impact. Inventing them would be the single most expensive mistake available at this stage.

1.1 Conflicts with version 2.0, and how each was resolved

Version 2.1 adds requirements that touch statements version 2.0 already made. Rather than overwrite them quietly, each conflict is named and resolved where it arises. **Nothing was removed except two rows that were mislabelled.**

ID	The conflict	Resolution	Where
K-01	v2.0 listed *"analytics beyond queue and load"* as FUTURE, which read as excluding SLA	SLA **state, breach and escalation** are CORE V1 because they drive routing and notification. SLA **reporting and dashboards stay FUTURE**; per-agent scorecards stay **forbidden** (**P-08**). Both FUTURE lines amended	§12.4 · §23 · §42
K-02	v2.0 defined six states, then listed any → reassigned and	**queued added** (routed, unclaimed) which v2.0 lacked and after-hours needs. Transfer	§21.4

ID	The conflict	Resolution	Where
	any → escalated in its transitions table — neither is a state	and escalation become **events**; escalation additionally becomes a **level**, because a case can be escalated *and* active *and* waiting at once	
K-03	v2.0 used *intent* informally; the journey specifies *category* and *sub-category*	Category is the **concrete realisation** of intent. Terminology unified; no earlier decision overturned	§21.6
K-04	Routing to a *designated employee* appears to contradict **BR-02** ("never a named individual")	**No conflict.** The customer picks a **category**; the **system** picks the person. A customer cannot request an individual, and cannot see who their designated employee is. BR-02 stands, clarified	§11.1 · §21.7
K-05	v2.0 deferred Redis; SLA clocks, an after-hours queue and a business calendar are new reasons to reconsider	**Re-evaluated, verdict unchanged.** Elapsed time is computed on read, the queue is a database state, the calendar is configuration. Redis stays FUTURE with the same multi-instance trigger	§23.9 · §33.2
K-06	The customer must not see internal SLA or escalation metadata, yet is expected to see "status"	**Two vocabularies**: the internal state machine, and a **customer-visible status** mapped from it. The customer vocabulary is itself a decision	§22.5 · §27.16 · **D-26**

One defect in version 2.0 was also found and fixed while producing this version: its table of contents listed Part II and Part III with numbers two lower than the actual sections, and omitted *Integrations and dependencies* entirely. The contents page is now generated from the headings rather than maintained by hand.

---

2. Executive summary

StarLink is a communication platform for CoverYou supporting two populations on one foundation: employees talking to each other, and employees talking to customers.

The business case. Approximately **147 of 195 people** in the company directory hold customer-contact roles across 26 departments. Customer conversation today happens on channels the company does not own, does not record, and cannot audit. The consequence is not merely untidy: conversation history that should be a company asset is instead held in individual phones and inboxes, and leaves when the individual does.

What is proposed. A single application with two deliberately different interfaces over one identity, authorization and audit foundation. Employees get a workload; customers get a conversation. The security boundary between them is the primary design constraint, not an afterthought.

What is not proposed. Not a CRM, not a policy administration system, not a customer master, not a social network, not a NorthStar module. §6.2 states the exclusions and why each one is excluded.

Delivery shape. Two phases inside V1. **V1-A internal communication** carries low risk, has real users from day one, and proves identity, authorization, realtime, notification and audit end to end. **V1-B a customer pilot in one department** then exercises the customer boundary at small scale. Company-wide rollout follows evidence, not optimism.

What is being asked of the reader. Sixteen open decisions in §44. **Five of them block the customer half of V1**; none of them blocks V1-A, which is precisely why the phasing exists. The most consequential is **D-01 — which channel**

customers reach us on — and this document offers **no recommendation** on it, because it is a business and regulatory choice about how CoverYou meets its customers.

What we propose to build it with — v2.2 production baseline. TypeScript end to end remains appropriate: Next.js surfaces and NestJS modular services. StarLink is no longer allowed to own duplicate customer identity, consent, work-allocation or enterprise audit truth; those resolve through CCS canonical services. Conversation/message persistence may remain MongoDB initially, but routing/ownership/work and cross-module business state are governed by CCS. For the customer pilot and any 500+ simultaneous-user target, multi-instance-ready realtime, Redis-compatible shared pub/sub/counters, durable async work queue/outbox, object storage, and adapter-based channel transports are baseline capabilities rather than undefined future options. Kafka/Kubernetes/sharding remain trigger-based, not reflexive.

Relationship to NorthStar and CCS. StarLink has no runtime dependency on NorthStar and must pass the NorthStar-off acceptance test. That independence must not be confused with duplicating CCS PRO. CCS Platform Kernel/IAM/Consent/Work/Event/Audit are intentional enterprise dependencies and authorities. StarLink is a specialised Conversation OS bounded context that can degrade gracefully if CY Brain is unavailable, while customer/employee authorization and hard compliance decisions fail closed when their canonical CCS authority cannot be verified.

---

3. Business context

3.1 The company, as the directory describes it

Attribute	Value	Source
People in directory	195	Internal directory, aggregate count
Departments	26	Internal directory
Sub-departments	50	Internal directory
Estimated customer-contact roles	~147 (≈75%)	Derived from department and role names
Largest customer-facing units	Fresh Sales 63 · Retention 21 · Corporate Sales 21 · Renewals 19 · Support 18	Internal directory
Smallest critical units	Claims 4 · Legal 3 · Grievance 1	Internal directory

> ***ASSUMPTION A-01 — the business domain.*** *This document reads CoverYou as an* ***insurance distribution business*** *(sales, renewals, retention, claims support, grievance handling). That reading is* ***inferred from department and role naming****, not from a business brief. It is confirmable in two minutes and is recorded as* ***decision D-14****. If it is wrong, §7, §11 and §21 need rework. Everything downstream of it is marked where it depends on it.*

3.2 How communication happens today

Channel	Used for	Recorded?	Auditable?	Survives staff departure?
Personal mobile / WhatsApp	Customer conversation, widely	No	No	**No**
Personal or shared email	Customer conversation, formal matters	Partially	Weakly	Sometimes
Telephone	Sales and support	No transcript	No	No
NorthStar internal chat	Employee ↔ employee only	Yes	Partially	Yes
Physical / verbal	Internal escalation	No	No	No

The pattern: internal communication is partly systematised; **customer communication is not systematised at all.** The company's most commercially significant conversations are its least visible.

3.3 Gaps this creates

#	Gap	Business consequence
G-01	Customer conversation is not recorded	No evidence of what was promised. In a regulated distribution business this is a compliance exposure, not just an inconvenience
G-02	History belongs to the individual, not the company	An advisor's departure takes the relationship context with them
G-03	No ownership model	Nobody can say who is answerable for a given customer conversation
G-04	No visibility of load	Leadership cannot see who is overloaded or which customers are waiting
G-05	No handover path	Leave, shift change and departure are handled by informal favour
G-06	Escalation is untracked	A grievance reaches the right person by luck and memory
G-07	No audit of access	No record of who read a customer's history, which is the question that matters after an incident
G-08	Customer and Policy exist in no system	Recorded by leadership as absent entities; every team keeps a private version

3.4 Why both populations, in one platform

A reasonable challenge is: *why not buy a customer chat tool and leave internal chat alone?*

Shared concern	Why one platform answers it once
Identity	An advisor is one person with one identity, whether talking to a colleague or a customer
Authorization	Escalation crosses the boundary — a grievance officer reads a customer thread an advisor owned
Audit	"Who saw this customer's history" must include internal participants, or it answers nothing
Internal notes	Staff discussion **about** a customer conversation must sit beside it, not in a separate tool where it can be pasted into the wrong window
Handover	Cover and reassignment are internal acts with customer consequences

Two tools would mean two identity models, two audit trails, and a copy-paste bridge between them. **The copy-paste bridge is where customer data leaks.**

3.5 Expected business value

Value	Mechanism	Measurable as
Conversation becomes a company asset	Server-side record, not a personal device	% of customer conversation inside StarLink
Accountability	Named owner per conversation	% conversations with an active owner
Continuity	Ownership survives absence and departure	Conversations orphaned by departure (target: zero)
Compliance evidence	Append-only record, audited privileged	Audit queries answerable without

Value	Mechanism	Measurable as
	access	engineering help
Operational visibility	Queue and workload state	Customers waiting beyond a service standard
Retained institutional knowledge	History outlives the employee	Context available at handover

---

4. Problem statement

> *CoverYou conducts its commercially and legally most significant conversations — with customers — on channels it does not own, cannot audit, and cannot transfer between staff. Internal communication is partially systematised in NorthStar, but the internal and customer sides share nothing, so escalation, handover and oversight fall back on informal effort.*
>
> *The company therefore cannot answer, for any given customer:* ***who is handling this, what was said, what was promised, who else has seen it, and what happens when the person handling it is unavailable.***

Stated as a requirement: a single platform that records customer conversation under a named owner, carries it across handover and escalation, keeps internal discussion adjacent but strictly invisible to the customer, and produces an audit trail that answers who accessed what.

---

5. Objectives

ID	Objective	Success indicator	Phase
O-01	All internal employee communication on a platform the company owns	V1-A adopted across the directory	V1-A
O-02	Customer conversation recorded server-side under a named owner	100% of pilot conversations owned and recorded	V1-B
O-03	A conversation survives absence, handover and departure	Zero conversations orphaned by an inactive owner	V1-B
O-04	The customer cannot see internal material	Zero internal-note exposure incidents	V1-B
O-05	Privileged access to customer history is attributable	Every privileged read carries actor, target and time	V1-A
O-06	Escalation reaches the right function with full context	Grievance and claims escalations tracked end to end	V1-B
O-07	Leadership can see load and waiting customers	Queue and workload visible without a report request	V1-B
O-08	Identity is replaceable without re-architecture	HRMS adoption changes one component (§17.2)	V1-A
O-09	No runtime dependency on any other internal platform	StarLink operates with NorthStar stopped	V1-A

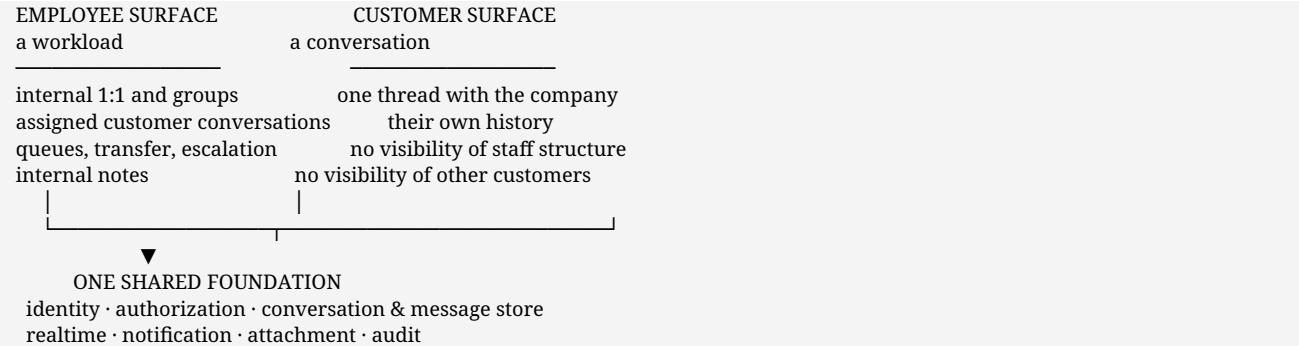
Explicit non-objectives. Replacing telephony · becoming the customer master (**D-03**) · measuring individual staff productivity · replacing NorthStar's work management.

---

6. Product overview

6.1 What StarLink is

A conversation platform with two surfaces and one foundation.



The two surfaces are **separate interfaces**, not one interface with elements hidden by role. That is a product decision with a real cost, recorded as **D-16**, and the reasoning is in §11.7.

6.2 What StarLink is not

Not	Why excluded
A CRM	Leadership has asked for a CRM framework separately. StarLink is the **conversation layer** such a framework would consume; it does not model pipeline, opportunity or quotation
A customer master	Whether StarLink holds customers is **D-03** and defaulting to yes would be a large, probably wrong commitment
A policy administration system	Policy data belongs to the system of record, whatever that becomes
A telephony or contact-centre platform	No call handling, IVR or dialler
A social platform	No feeds, follows, reactions-as-engagement or profiles beyond directory identity
A document management system	Attachments are conversation evidence, not a filing system
A NorthStar module	§38 explains what is borrowed and §36 confirms nothing is depended on
A productivity measurement tool	§6.4

6.3 Core capabilities

Capability	Employee	Customer
Internal 1:1 and group conversation	✅	—
Company announcements	✅ receive	—
Employee directory	✅	❌ never
Customer conversation intake	✅ receive	✅ initiate
Ownership and assignment	✅	❌ not visible
Transfer and escalation	✅	❌ not visible
Internal notes	✅	❌ never
Attachments	✅	**D-07**
Search	✅ scoped	**D-08**
Realtime delivery, typing, unread	✅	✅ partial (§20.2)

Capability	Employee	Customer
Conversation history	✅ scoped	✅ own only
Audit trail	✅ compliance role only	❌

6.4 Design principles

Stated once, then held to throughout. Where a later section appears to violate one of these, it is a defect in this document.

P-01 · The cannot list comes before the can list. For the customer surface, what is forbidden is specified before what is permitted (§11.7). A capability list written first tends to produce a permission model that is a series of afterthoughts.

P-02 · Authentication is not authorization. Knowing who someone is says nothing about what they may read. These are separate components with separate failure modes (§26).

P-03 · Participation is not ownership. Being added to a conversation grants reading it. It does not grant replying to the customer, reassigning, or closing (§11.2).

P-04 · Realtime is justified per event, not assumed. Every live channel is a cost in connections, correctness and debugging. §20.2 justifies each one individually; six events qualify and six do not.

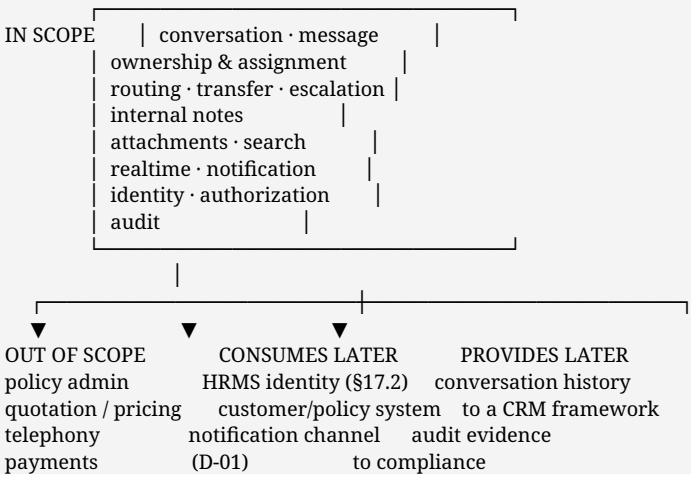
P-05 · The record is written first, then delivery is attempted. A notification that fails must never mean a message that was not stored (§20.1).

P-06 · Privileged reads are audited; ordinary internal messages are not. Auditing every internal message is surveillance, not accountability. Auditing who read a *customer's* history is exactly the question an incident asks (§27.6).

P-07 · Hiding is not a security model. The customer surface is built from the customer's use cases rather than assembled by concealing employee features (**D-16**).

P-08 · The platform is not a productivity instrument. No message counts, response-time leaderboards or activity scoring. Volume metrics exist for capacity planning at team level, never as individual performance measures.

6.5 Product boundary



7. Stakeholders and user personas

Personas are derived from department and role names in the directory. They are **not** copied from NorthStar's five work-management bands (Employee / Manager / Founder / Founder's Office / Admin), which describe a different product's authority model and would import assumptions StarLink has not tested.

7.1 Primary personas

ID	Persona	Approx. size	Primary need	Success looks like
PA-01	**Customer**	External	Reach the company and be answered, without managing our internal structure	One thread, answered, history intact
PA-02	**Advisor** (Fresh Sales, Corporate Sales)	~84	Handle many new conversations quickly	Next conversation is obvious; nothing waits unseen
PA-03	**Relationship advisor** (Retention, Renewals)	~40	Continue with customers they already know	The customer reaches *them*, with history
PA-04	**Support agent**	~18	Resolve issues with full context	The thread carries what happened before
PA-05	**Claims handler**	~4	Case-shaped work with documents	Case history and attachments in one place
PA-06	**Grievance officer**	1	Complete context on complaints	Prior history readable on escalation
PA-07	**Team lead**	Per department	See load, reassign, cover absence	Queue depth and waiting customers visible
PA-08	**Compliance / Legal**	~3	Answer "what was said" and "who saw it"	Audit query without engineering help
PA-09	**Administrator**	Few	Manage accounts, roles, participants	Role change takes effect immediately
PA-10	**Leadership**	Few	Know whether customers are being served	Waiting customers and load, not activity feeds

> ***D-11 — which roles actually exist*** *is open. These personas are inferred and must be confirmed before the permission model is fixed. Notably:* ***may a team lead read any conversation in their team by default?*** *Proposal: no — scoped grant, audited as privileged.*

7.2 Why sizes matter to the architecture

Observation	Architectural consequence
Fresh Sales (63) has no prior customer relationship	Queue-shaped routing; speed to first response dominates
Retention and Renewals (~40) have named relationships	Sticky ownership; "the advisor is on leave" is a **weekly** event, not an edge case (§9 UC-E07)
Claims (4) and Grievance (1) are tiny but critical	Single points of failure in the org. Software must make waiting **visible**, not silently hold work
~147 of 195 are customer-facing	The customer surface is the main event, not a bolt-on

---

8. User journeys

End-to-end walks in Actor → Action → System → Result form, readable without any architecture knowledge. Each names the decisions it depends on.

Journey A — Employee to employee

Actor	Action	System	Result
Employee	Searches the directory for a colleague	Resolves identity, checks directory visibility	Colleague found
Employee	Opens or creates the conversation	Creates conversation; both become participants	Thread open
Employee	Sends a message	**Stores first**, then publishes to connected recipients	Message durable
System	—	Delivers live; raises unread; notifies externally if away	Recipient sees it
Colleague	Opens the thread	Marks read for that person only	Sender sees read state

Notes. No lifecycle — an internal thread is open indefinitely (**D-15**). Adding someone to a group exposes them to prior history, and the interface must say so **before** the add. Participant changes are audited; ordinary internal messages are not (P-06).

Journey B — Customer to company

Actor	Action	System	Result
Customer	Visits the CoverYou website and opens StarLink chat	Widget loads; no identity yet	Chat open
Customer	Is asked *"How can we help you?"*	Presents the category list (**D-17**)	Choice offered
Customer	**Selects a category**, and a sub-category if offered	Records it; selects the candidate team (**D-18**)	Requirement captured
Customer	Identifies themselves	Verifies identity (**D-02**, **D-03**)	Verified session
System	—	**Checks the team's business calendar** (§23.1)	Open, or after hours
Customer	Sends the first message	Stores; creates conversation in new	Message durable
System	—	**If after hours:** queues, acknowledges, starts no clock (§23.3)	queued, nothing lost
System	—	**If open:** routes by the decision tree of §21.8 — designated employee if available, else the approved fallback, else the team queue	assigned or queued
System	—	Notifies the owner or the team; **starts the SLA clock** (§23.5)	Staff aware, clock running
Advisor	Claims or receives it	Assigns ownership; blocks double-claim	Named owner
Advisor	Replies	Stores; delivers to customer	active

The customer never chooses an individual. Ownership is the company's to assign (§11.1).

Journey C — Company to customer

Actor	Action	System	Result
Advisor	Opens an owned conversation	Authorizes: owner or participant	Thread readable
Advisor	Types a reply	Optionally signals typing to staff	—
Advisor	Sends	Stores; delivers on the customer's channel	Customer notified
Advisor	Or adds an **internal note**	Stores, marked internal, **never delivered to the customer**	Staff-only context
System	—	Customer reply returns to the same thread and owner	Continuity

The leak risk lives here. Internal notes must be visually unmistakable in the interface (§11.6).

Journey D — Transfer and escalation

Actor	Action	System	Result
Advisor or lead	Requests transfer, **with a reason**	Authorizes `conversation.transfer`	Reason recorded
System	—	Reassigns ownership; prior history readable by the new owner	New owner
System	—	Notifies both parties and the team	Handover visible
System	—	**Audits** the transfer and the reason	Attributable
Grievance case	Escalated	Routes to Grievance; original owner **remains a participant**	Right function, full context
Grievance officer	Reads prior history	**Audits the read as privileged**	Access attributable

The customer is not told the internal mechanics — only, where appropriate, that someone else is now helping.

Journey E — The owning employee is unavailable

Actor	Action	System	Result
Customer	Replies to an owned thread	Detects owner unavailable	—
System	—	Surfaces to the team as needing cover	Somebody can answer
Covering agent	Answers	Authorizes via **team scope**, not the owner's identity	Customer answered
System	—	Audits the cover — it changed who read the thread	Attributable
Owner	Returns	Notified of what changed	Context restored

Three cases, one open decision (D-05). Short absence → hold ownership, surface for cover. Extended leave → return to queue, named backup, or lead reassigns — **undecided**. Departure → ownership **must** be reassigned; a conversation owned by a deactivated account is unreachable work, and this is the case that silently loses customers.

Journey F — Closure and reopening

Actor	Action	System	Result
Owner	Marks resolved, **with an**	Authorizes; state → `resolved`	Customer told why

Actor	Action	System	Result
	outcome		
System	—	Notifies the customer; audits	Closure attributable
System	After the reopen window	State → closed	Terminal
Customer	Replies inside the window	Reopens the same thread to the same owner	Continuity
Customer	Replies after the window	Starts a **new** conversation, prior history linked for staff	Clean state

Window length is D-08. Proposal: 7 days — arbitrary until the business gives a service standard.

---

9. Use case catalogue

9.1 Catalogue

Priority: **P0** required for the phase to function · **P1** important, phase can pilot without it · **P2** valuable, deferrable.

ID	Actor	Use case	Description	Priority	Phase	Notes
UC-E01	Advisor	Receive a customer conversation	Routed or claimed from a queue	P0	V1-B	Double-claim must be impossible
UC-E02	Advisor	Reply to a customer	In an owned thread	P0	V1-B	Non-owner reply is **D-04a**
UC-E03	Advisor	Add an internal note	Staff context inside the thread	P0	V1-B	**Leak risk** — must be unmistakable
UC-E04	Advisor, lead	Transfer a conversation	Reassign with a reason	P0	V1-B	Reason mandatory (proposal)
UC-E05	Advisor, lead	Escalate to grievance/claims	Route to the owning function	P0	V1-B	Prior-history read is audited
UC-E06	Employee	Internal 1:1 and group	Colleague conversation	P0	V1-A	No lifecycle (**D-15**)
UC-E07	Team, lead	Cover an unavailable owner	Absence, leave, departure	P0	V1-B	Policy is **D-05**
UC-E08	Advisor, lead	Add a second employee	Specialist input, escalation overlap	P0	V1-B	Participation ≠ ownership
UC-E09	Employee	Search	Scoped to what they may read	P0	V1-A	Term is audited
UC-E10	Advisor	Claim from a queue	Pull unassigned work	P0	V1-B	Atomic claim
UC-E11	Lead	Reassign someone else's conversation	Load balancing, escalation	P0	V1-B	Reason required
UC-E12	Employee	Send an attachment	File with a message	P0	V1-A	**Download** is the audited event
UC-E13	Employee	See unread state	Counts per conversation	P0	V1-A	Counting, not a live feed
UC-E14	Employee	Read receipts (internal)	Who has seen it	P1	V1-A	Internal only
UC-E15	Employee	Typing indicator	Someone is	P1	V1-A	Cheap, expected

ID	Actor	Use case	Description	Priority	Phase	Notes
			composing			
UC-E16	Employee	Reply to a specific message	Quote in long threads	P1	V1-A	High value in claims
UC-E17	Employee	Archive / mute	Personal inbox hygiene	P1	V1-A	**Personal view state**
UC-E18	Owner	Resolve / close	With an outcome	P0	V1-B	Customer threads only
UC-E19	Owner, customer	Reopen	Inside a bounded window	P0	V1-B	Window is **D-08**
UC-E20	Lead, leadership	See queue and load	Waiting customers, workload	P0	V1-B	Not a leaderboard (P-08)
UC-E21	Compliance	Query the audit trail	Who accessed what, when	P0	V1-A	Restricted role
UC-E22	Lead	Read a team conversation	Oversight	P2	V1-B	**D-11** — audited as privileged
UC-E23	Admin	Administer users and roles	Accounts, roles, participants	P0	V1-A	Must-succeed audit
UC-E24	All employees	Receive announcements	Company notices	P1	V1-A	Who may post — **D-11**
UC-C01	Customer	Start a conversation	**Open website chat, choose a category, identify, send**	P0	V1-B	Categories **D-17** · channel **D-01**
UC-C02	Customer	Receive a reply	Same thread	P0	V1-B	—
UC-C03	Customer	Continue a conversation	Same context, whoever answers	P0	V1-B	—
UC-C04	Customer	See own history	Their conversations only	P0	V1-B	Read is audited
UC-C05	Customer	Re-verify after session expiry	Resume mid-conversation	P0	V1-B	See 9.5
UC-C06	Customer	See resolution	And be told why	P0	V1-B	—
UC-C07	Customer	Reopen	Inside the window	P0	V1-B	**D-08**
UC-C08	Customer	See own unread	Their own only	P1	V1-B	—
UC-C09	Customer	Send an attachment	Claims plainly wants this	P1	**D-07**	AV scanning is a real cost
UC-C10	Customer	Reply to a specific message	Long claim threads	P2	Future	—
UC-C11	Customer	Search own history	—	P2	**D-08**	—
UC-C12	Customer	Export own history	Possibly a compliance obligation	P2	**D-08**	Ask Legal, not engineering
UC-C13	Customer	**Contact us outside working hours**	Message accepted, stored, acknowledged, queued for the next business period	**P0**	**V1-B**	Behaviour **D-20** · hours **D-21** · §23.3
UC-E25	Owner, lead	**Handle an SLA warning or breach**	Warning to owner; breach to owner and lead; escalation	**P0**	**V1-B**	Thresholds **D-25** · **never customer-visible** · §23.6

ID	Actor	Use case	Description	Priority	Phase	Notes
			per policy			

9.2 Detailed use cases — the ones that carry risk

Six cases are specified in full because each is either the security surface, or a place where a wrong assumption is expensive. The remainder are adequately described by the catalogue.

UC-E01 · Receive a customer conversation

- **Actor** Advisor · **Secondary** team lead, system router
- **Trigger** A customer sends a first message on the agreed channel
- **Preconditions** Customer identity verified (**D-02**); routing model configured (**D-04**)
- **Main flow** Intake creates the conversation in `new` → router evaluates relationship → routes to the owning advisor, or to the team queue → notifies → advisor claims or receives → state `assigned` → advisor opens and reads
- **Alternative flows** Two advisors claim simultaneously → **the claim must be atomic**; the loser is told it is taken, not shown an error · No advisor available → conversation stays queued and **visibly unanswered**; it is never silently held · Relationship exists but the owner is unavailable → UC-E07
- **Postconditions** Exactly one owner; the customer is waiting on a named person
- **Permissions** `conversation.claim` within team scope
- **Data** Conversation, participants, assignment, message
- **Realtime** Yes — the queue must not require a refresh
- **Notification** Owner, or the team on queue arrival
- **Audit** Yes — assignment is a change of who may read

UC-E03 · Add an internal note

- **Actor** Any staff participant
- **Trigger** Context that belongs beside the conversation but must not reach the customer
- **Main flow** Compose in a **visibly distinct** internal mode → store, marked internal → deliver to staff participants only
- **Alternative flows** Customer-facing composer confused with internal → **this is the primary leak scenario**. Mitigation is interface-level and non-negotiable: distinct colour, persistent label, and a mode that does not silently reset after send
- **Permissions** Staff participant; never any customer principal
- **Data** Message with an internal marker
- **Realtime** Yes, to staff only
- **Notification** No — internal notes must not generate customer-visible activity
- **Audit** Yes
- **Failure requirement** If the internal marker cannot be established with certainty, **the send must fail** rather than default to customer-visible

UC-E05 · Escalate to grievance or claims

- **Actor** Advisor or team lead
- **Trigger** A complaint, or a service failure
- **Main flow** Escalate with a reason → conversation flagged and routed to the owning function → **full prior history becomes readable to the receiving officer** → the original owner remains a participant
- **Alternative flows** Grievance has one person and they are unavailable → the case is queued and visibly waiting. **This is an organisational single point of failure, not a software one**, and the software's job is to make it undeniable
- **Permissions** conversation.escalate; the receiving function's scope
- **Realtime** Yes · **Notification** Receiving officer and team lead
- **Audit** Escalation recorded. **The officer's read of prior history is itself audited** — this is the privileged-access case the audit model exists for

UC-E07 · Cover an unavailable owner

Not an edge case: with sticky ownership across Retention and Renewals (~40 people), "the owner is on leave" occurs weekly.

- **Actor** Covering agent, team lead · **Trigger** A customer replies to a thread whose owner is away
- **Main flow** System detects unavailability → surfaces to the team as needing cover → a colleague answers under **team scope** → owner is notified on return
- **Alternative flows Short absence** → hold ownership, surface for cover *(proposal)* · **Extended leave** → return to queue / named backup / lead reassigns — **D-05, undecided · Departure** → ownership **must** be reassigned; otherwise the conversation is unreachable work · **Nobody available** → queued and visibly unanswered
- **Permissions** Team scope, not the owner's identity
- **Audit** Yes — cover and reassignment both change who may read
- **Dependency worth naming** This is the **only** justification for a presence system (§20.2). If D-05 is answered "named backup" or "lead reassigns", a human decides and no presence tracking is needed

UC-E08 · A second employee joins a customer conversation

UC-E05 depends on this capability, so it is specified rather than implied.

- **Actor** Owner, team lead, or an escalation
- **Trigger** Specialist input, escalation, handover overlap, cover
- **Main flow** Second employee added → reads the conversation **including prior history** → contributes, by default via internal notes → the owner remains the customer's single point of contact
- **Alternative flows May a non-owner reply to the customer directly?** — **D-04a**, proposal **no**: two staff voices in one thread reads as disorganised and makes accountability ambiguous; a lead may override · The second employee leaves → prior access is history, not retroactively revoked
- **Permissions Participation is not ownership** (P-03). Being added grants read and internal-note write; it does **not** grant customer-reply, reassignment or closure. **Exactly one owner at a time**
- **Audit** Yes — participant added and removed, because it changes who may read the customer's history

UC-C05 · Customer session expires mid-conversation

- **Actor** Customer · **Trigger** A bounded session ends while reading or composing
- **Main flow** Session expires → thread becomes unreadable, composer disabled → customer asked to re-verify (**D-03**) → on success the **same** conversation and full history return

- **Alternative flows** Re-verification abandoned → nothing readable, and the screen says *the session ended*, never implying the conversation is gone · Expiry mid-compose → *(proposal)* the draft is held in the customer's own browser only and restored after re-verification; **never posted on their behalf** · Expiry with an unread staff reply → the reply waits; the notification still goes out
- **Permissions** Re-verification restores the prior scope and **nothing more** — expiry is not a path to widened access
- **Realtime** The live channel closes **server-side**, not merely visually. A socket outliving its session is an authorization hole
- **Audit** Expiry is not audited (it is a clock, not an act). **Re-verification is** — a burst of attempts against one identity is what a takeover looks like

9.3 Traceability

Every CORE V1 feature in §12 traces to at least one use case ID above. Any feature that cannot is not in V1 — it is in FUTURE or OUT OF SCOPE, by construction.

9.4 What the customer surface permits

Derived from UC-C01–UC-C12 only. The customer surface implements nothing that is not on this list.

9.5 What the customer surface forbids

See §11.7 — written before capabilities, per P-01.

---

10. Functional requirements

Grouped by module. FR-<module>-<n>. Each requirement is testable; each traces to a use case or a principle.

10.1 Authentication

ID	Requirement	Trace
FR-AUTH-1	Employees authenticate against StarLink's own credential store in V1; the mechanism must be replaceable without touching authorization	O-08
FR-AUTH-2	Sessions are server-issued, signed, and revocable **by version**, so a role change or deactivation takes effect on the next request	UC-E23
FR-AUTH-3	Session cookies are HttpOnly and SameSite-restricted; Secure whenever TLS is present	§27.1
FR-AUTH-4	Customer authentication is a **separate flow** from employee authentication and shares no credential path	**D-02**
FR-AUTH-5	Customer sessions are bounded; expiry requires re-verification and restores exactly the prior scope	UC-C05
FR-AUTH-6	Every failed authentication returns an identical response regardless of cause	§27.1
FR-AUTH-7	Re-verification and authentication failures are audited; ordinary expiry is not	UC-C05

10.2 Authorization

ID	Requirement	Trace
FR-AUTHZ-1	Authorization is evaluated **before** any message content is read, on every path without exception	P-02
FR-AUTHZ-2	Participation grants access to **that conversation only** — never a global grant	P-03
FR-AUTHZ-3	An unknown permission is **refused**, never allowed	§27.2
FR-AUTHZ-4	Elevated grants may be time-boxed, and expiry is evaluated from the clock — never from a sweep job	§27.2
FR-AUTHZ-5	"Exists but you may not see it" and "you may not act on it" return the same response, disclosing nothing	§27.3
FR-AUTHZ-6	Broadcast/announcement visibility is **kind-scoped**: employee principals only, never a customer	§38 (rejected pattern)
FR-AUTHZ-7	Administration does not imply reading: an admin manages roles and participants without silently reading everyone's messages	UC-E23

10.3 Employee and user management

ID	Requirement	Trace
FR-EMP-1	Every employee is one principal with a stable identifier that is the **only** join key used anywhere	§24.2
FR-EMP-2	Deactivation immediately ends sessions and blocks sending; **history remains readable** to those entitled	UC-E07
FR-EMP-3	Deactivation must surface every conversation the person owned, for reassignment	UC-E07
FR-EMP-4	Organisational attributes (department, sub-department, manager) are readable for scope resolution; absent attributes are **absent**, never blank	§27.2
FR-EMP-5	Directory visibility is configurable; the default is company-wide	**D-11**

10.4 Customer management

ID	Requirement	Trace
FR-CUST-1	A customer is a principal of a **distinct kind**, never an employee record with a flag	P-01
FR-CUST-2	Customer records hold the minimum needed to identify and contact; StarLink does not become the customer master unless **D-03** says so	§6.2
FR-CUST-3	Customer isolation is absolute: no customer can observe another, or that	FR-SEC-2

ID	Requirement	Trace
	another exists	
FR-CUST-4	History from before identity verification is **not** readable, preventing a hijacked session inheriting a thread	§27.3
FR-CUST-5	Access ends on account closure	§11.7

10.5 Conversation management

ID	Requirement	Trace
FR-CONV-1	Conversations are typed: internal 1:1, internal group, announcement, customer	§24.1
FR-CONV-2	Participants carry a **role and an optional expiry** from the first record, not a flat list of identifiers	§38 (IMPROVE)
FR-CONV-3	Customer conversations have exactly **one owner** at any time	§11.3
FR-CONV-4	Internal conversations have **no lifecycle**	**D-15**
FR-CONV-5	Customer conversations follow new → assigned → active → waiting → resolved → closed	§21.4
FR-CONV-6	Every state transition records actor, time and reason where a reason applies	§21.4
FR-CONV-7	Adding a participant exposes prior history, and the interface must state this **before** the action	UC-E06
FR-CONV-8	Archive and mute are **personal view state**, never conversation state	UC-E17

10.6 Messaging

ID	Requirement	Trace
FR-MSG-1	A message is durable **before** any delivery is attempted	P-05
FR-MSG-2	Messages are immutable; correction is a new message, not an edit	O-05
FR-MSG-3	Message history is paged at the database with a **stable, opaque cursor**; no offset paging	§38 (KEEP)
FR-MSG-4	Paging is deterministic under concurrent insertion — ordering carries a tiebreaker beyond timestamp	§38 (KEEP)
FR-MSG-5	Internal notes are messages with an internal marker, delivered to staff participants only	UC-E03
FR-MSG-6	If the internal marker cannot be established, the send **fails**	UC-E03
FR-MSG-7	Reply-to references another message in the same conversation only	UC-E16

10.7 Assignment and routing

ID	Requirement	Trace
FR-ROUTE-1	Every incoming customer conversation is routed by an explicit, inspectable rule	**D-04**
FR-ROUTE-2	Where a relationship exists, route to the owning advisor; otherwise to the team queue *(proposed hybrid)*	**D-04**
FR-ROUTE-3	Claiming from a queue is **atomic**; a second claimant is told it is taken	UC-E10
FR-ROUTE-4	An unclaimed conversation is **visibly waiting** and never silently held	UC-E01
FR-ROUTE-5	Unavailability surfaces the conversation for cover without losing the relationship	UC-E07
FR-ROUTE-6	Assignment history is **append-only** — the cover and the return are both retained	§24.3

10.8 Transfer and escalation

ID	Requirement	Trace
FR-TRAN-1	Transfer requires a reason *(proposal: mandatory)*	UC-E04
FR-TRAN-2	Transfer moves ownership and grants the new owner prior history	UC-E04
FR-TRAN-3	Escalation routes to the owning function and **retains the original owner as a participant**	UC-E05
FR-TRAN-4	Both notify all affected parties, and both are audited	UC-E04/05
FR-TRAN-5	The customer is not exposed to internal transfer mechanics	§11.7

10.9 Attachments

ID	Requirement	Trace
FR-ATT-1	Attachments are authorized through their conversation — never by knowing a URL	§27.4
FR-ATT-2	Attachment references are derived on read, never stored as routes	§38 (IMPROVE)
FR-ATT-3	Type and size limits are enforced server-side	§27.4
FR-ATT-4	Files are served with headers that prevent inline execution in the browser	§27.4
FR-ATT-5	**Download is audited**, not upload — the upload is attributable via its message; the download is the exfiltration event	P-06
FR-ATT-6	Customer uploads require AV scanning if permitted at all	**D-07**

10.10 Search

ID	Requirement	Trace
FR-SRCH-1	Scope is resolved **before** the query, never	§27.3

ID	Requirement	Trace
	filtered after results	
FR-SRCH-2	Results are limited to conversations the searcher may already read	UC-E09
FR-SRCH-3	Searches are audited **with the term** — "someone searched" answers nothing	P-06
FR-SRCH-4	Empty results say "no matches", never "no data"	UC-E09
FR-SRCH-5	Very short terms are refused rather than returning everything	UC-E09

10.11 Read and unread state

ID	Requirement	Trace
FR-READ-1	Unread is per person per conversation	UC-E13
FR-READ-2	Read state is recorded on thread open, not on delivery	UC-E14
FR-READ-3	Read receipts are internal-only in V1; customer-visible receipts are a **decision**	§11.7
FR-READ-4	Read tracking must not write on every scroll event	§13.3

10.12 Realtime

ID	Requirement	Trace
FR-RT-1	Realtime is **additive**: no state exists only in a live event	P-05
FR-RT-2	A dropped connection recovers by re-reading, not by replaying events	§20.3
FR-RT-3	Live channels are authorized at subscribe **and** invalidated on session end	UC-C05
FR-RT-4	Events carry identifiers, never message bodies to unauthorized subscribers	§27.3
FR-RT-5	Only the six events justified in §20.2 are live	P-04

10.13 Notifications

ID	Requirement	Trace
FR-NOT-1	Notification failure never blocks or reverses a stored message	P-05
FR-NOT-2	In-app notification is required in V1; external channels are adapter-based	§17.7
FR-NOT-3	Notifications are actionable, not activity echoes	§20.4
FR-NOT-4	Internal notes generate no customer-visible notification	UC-E03
FR-NOT-5	Customer notification channel and opt-out are a **decision**	**D-12**

10.14 Audit

ID	Requirement	Trace
FR-AUD-1	The audit ledger is **append-only**; no update or delete path exists in the application	O-05
FR-AUD-2	Audited: authentication events, authorization grants and revocations, assignment, transfer, escalation, participant change, privileged reads, attachment downloads, searches with terms, administrative action	P-06
FR-AUD-3	**Not** audited: ordinary internal message sends	P-06, P-08
FR-AUD-4	Identifying request context (address, agent) is stored **separately** with its own retention, so retention policy and ledger integrity do not conflict	§27.6
FR-AUD-5	Audit write failure for a security-critical action **fails the action**	§27.6
FR-AUD-6	Audit is queryable by actor, target, action and period, by a restricted role	UC-E21
FR-AUD-7	Audit is never exposed through ordinary conversation interfaces	§27.6

10.15 Administration

ID	Requirement	Trace
FR-ADM-1	Accounts, roles and participants are managed through an interface, not the database	UC-E23
FR-ADM-2	Role assignment is audited as **must-succeed** — whoever assigns roles could otherwise grant themselves anything	FR-AUD-5
FR-ADM-3	Deactivation is a first-class action with the consequences in FR-EMP-2/3	UC-E07
FR-ADM-4	Administration does not confer message read access	FR-AUTHZ-7

10.16 Reporting and oversight

ID	Requirement	Trace
FR-REP-1	Queue depth and waiting time are visible to leads and leadership	UC-E20
FR-REP-2	Workload is visible per team and per owner **for capacity purposes**	UC-E20
FR-REP-3	No individual productivity score, ranking or response-time leaderboard	**P-08**
FR-REP-4	Oversight reads of conversation content are privileged and audited	UC-E22, **D-11**

---

11. Business rules

Stated as rules because they must survive interpretation by whoever implements them.

11.1 Who may initiate

Rule	Statement
BR-01	Any employee may start an internal conversation with any visible colleague
BR-02	A customer may start a conversation with **the company**, choosing a **category** — never a named individual. The customer picks *what they need*; the **system** picks *who handles it* (§21.7). A customer cannot request an individual, and cannot see who their designated employee is
BR-03	An employee may not start a conversation with a customer who has no existing relationship, unless the business grants outbound initiation *(not in V1)*
BR-04	Announcements may be posted only by authorised roles (**D-11**)

11.2 Who may participate

Rule	Statement
BR-05	Participation is granted by the conversation creator, the owner, a lead, or an escalation
BR-06	**Participation grants reading that conversation. Nothing else**
BR-07	Adding a participant exposes all prior history, and the interface must say so first
BR-08	A customer is never a participant in an internal conversation, by any path
BR-09	Removing a participant ends future access; it does not un-read what was read

11.3 Ownership

Rule	Statement
BR-10	Every customer conversation has **exactly one owner** at any moment
BR-11	Ownership is assigned by the system or a lead — never chosen by the customer
BR-12	The owner is accountable for response and resolution
BR-13	Ownership by a deactivated account is invalid and **must** be reassigned
BR-14	Ownership history is append-only

11.4 Transfer and escalation

Rule	Statement
BR-15	The owner or a lead may transfer; a reason is required *(proposal)*
BR-16	Escalation preserves the original owner as a participant
BR-17	The receiving party gains full prior history, and that read is audited

Rule	Statement
BR-18	Transfer never silently drops the customer's waiting state

11.5 Closure and reopening

Rule	Statement
BR-19	Only the owner or a lead may resolve, and an outcome is recorded
BR-20	The customer is told the conversation was resolved, and why
BR-21	A reply inside the reopen window reopens the same thread to the same owner
BR-22	After the window, a new conversation is created with prior history linked **for staff**
BR-23	Internal conversations are never closed (**D-15**)

11.6 Internal notes

Rule	Statement
BR-24	Internal notes are visible to staff participants only
BR-25	They must be **visually unmistakable** in every interface that renders them
BR-26	They generate no customer-visible notification or activity signal
BR-27	If internal status cannot be established with certainty, the send fails

11.7 What a customer cannot do

Written before the capability list, per **P-01**. This is the security surface.

A customer cannot	Why
See internal notes	The single worst leak available in this product
See who else is on the conversation, or their roles	Staff structure is not the customer's business
See any other customer, or that one exists	Hard isolation boundary
Search or browse the employee directory	Same
Start a conversation with a named individual	Ownership is the company's to assign (BR-02)
Reach a team they are not entitled to	A prospect does not open a claim
Read history from before their identity was verified	Prevents a hijacked session inheriting a thread
Retain access after account closure	FR-CUST-5
Reopen indefinitely	Bounded window (**D-08**)
Delete our messages or edit history	The record must be trustworthy enough to argue from
See staff delivery/read receipts	**Needs discussion** — pressure on advisors, no clear customer benefit
See internal SLA state, targets, or whether one was breached	A breach is **our** failure to manage. Showing it hands the customer a grievance we generated ourselves (§22.5)
See escalation level, reason, or history	Internal handling is not the customer's business
See internal routing information — the queue, the fallback path, who was tried first	Discloses staffing and structure, and invites *"why not the good one?"*
See assignment history	Same. **D-27** decides whether even a case reference is shown
See conversation priority	An operational judgement, not a service promise

A customer cannot	Why
See the employee hierarchy, or who else is on the case	Restated because the case model (§22) adds fields that must not leak

11.8 Employee access boundaries

Rule	Statement
BR-28	An employee reads a customer conversation only as owner, participant, or under an explicit scoped grant
BR-29	Department membership alone grants nothing
BR-30	A lead does not read team conversations by default; oversight is a scoped, audited grant (**D-11**)
BR-31	Compliance may query the audit trail without gaining message read access
BR-32	Every scoped grant is time-boxed where the business allows

---

12. Feature prioritisation

Five buckets. The fifth is the one that matters most in review.

12.1 CORE V1 — required

Feature	Use case	Phase
Authentication and session management	FR-AUTH	V1-A
Authorization: permissions, roles, scopes	FR-AUTHZ	V1-A
Internal 1:1 and group conversation	UC-E06	V1-A
Employee directory	UC-E06	V1-A
Messaging, history, cursor paging	UC-E01–03	V1-A
Employee attachments	UC-E12	V1-A
Unread state	UC-E13	V1-A
Search, scoped and audited	UC-E09	V1-A
In-app notifications	FR-NOT-2	V1-A
Realtime: message, typing, conversation state	§20.2	V1-A
Audit ledger	UC-E21	V1-A
Administration console	UC-E23	V1-A
Customer intake	UC-C01	V1-B · blocked on **D-01**
Routing and queue	UC-E01, E10	V1-B · **D-04**
Ownership and assignment	BR-10	V1-B
Cover for an unavailable owner	UC-E07	V1-B · policy **D-05**
Multiple staff on a conversation	UC-E08	V1-B
Transfer and escalation	UC-E04, E05	V1-B
Internal notes	UC-E03	V1-B
Customer conversation surface	UC-C01–C08	V1-B · **D-16**
Category intake on the website	UC-C01	V1-B · categories **D-17**
Service case metadata	§22	V1-B · naming **D-28**

Feature	Use case	Phase
Business calendar and after-hours queue	**UC-C13**	V1-B · hours **D-21**, behaviour **D-20**
SLA state, warning, breach and escalation	**UC-E25**	V1-B · targets **D-22**, thresholds **D-25**
Lifecycle: resolve, close, reopen	UC-E18, E19	V1-B
Queue and load view	UC-E20	V1-B

Open questions on these rows concern a **detail** of a feature that is certainly needed — which roles exist, what the limits are. The feature itself is not in doubt.

12.2 IMPORTANT — V1.1

Read receipts (internal) · typing indicator · reply-to-message · archive and mute · external notification delivery · announcement channel · customer unread state.

12.3 NEEDS BUSINESS DECISION — existence undecided

Distinguished from *deferred*: these are not scheduled because nobody has yet said whether they should exist.

Feature	Priority if approved	Real cost	Decision
Customer attachments	P1	AV scanning — recurring cost, and a real risk if skipped	**D-07**
Customer search of own history	P2	Customer scope resolution	**D-08**
Customer export of own history	P2	May be a compliance obligation rather than a feature	**D-08**
Customer-visible read receipts	P2	Advisor pressure, no clear customer benefit	§11.7
Multi-brand / multi-tenant separation	P0 **if yes**	Cheap now, expensive to retrofit	**D-09**
Customer information panel for employees	P1	Breach surface widens with context	**D-10**
Lead default read of team conversations	P2	The widest privacy surface in the product	**D-11**

12.4 FUTURE

Reactions · message forwarding (**the leak primitive** — if built, source and every target are recorded) · server-side drafts · pinned conversations · presence and availability (**only** if D-05 requires automatic routing around absence) · voice notes · CRM framework integration · analytics beyond queue, load and SLA state.

> ***Conflict resolved — K-01.*** *Version 2.0's wording ("analytics beyond queue and load") read as excluding SLA.* ***SLA state tracking, breach flags and escalation are CORE V1*** *(§23) because they drive routing and notification, which are already V1.* ***SLA reporting, trend dashboards and cross-team benchmarking remain FUTURE****, and per-agent SLA scorecards are* ***forbidden*** *by* ***P-08****, not merely deferred.*

12.5 OUT OF SCOPE

Excluded	Reason
Policy administration, quotation, pricing	Belongs to the system of record
Telephony, IVR, dialler	Different product category
Payments	Not a conversation concern
Document management	Attachments are conversation evidence, not a filing system
Individual productivity scoring	**P-08**

Excluded	Reason
Message deletion and history editing	The record must be trustworthy (§11.7)
Customer-to-customer contact	No business case; large abuse surface
Public or unauthenticated chat	Every conversation has an identified customer (**D-02**)
Federation with external chat networks	No business case
Microservice decomposition	§37.1 — no problem yet exists that it solves

---

13. Non-functional requirements

Targets are proposals where no business standard exists, and are marked. **D-13** (expected volume) is an input this section needs; without it, sizing is estimation.

13.1 Security

ID	Requirement
NFR-SEC-1	Authorization on every content path, evaluated before content is read
NFR-SEC-2	Customer isolation verified by test, not by inspection
NFR-SEC-3	Secrets from configuration only; production refuses to start on a shipped default
NFR-SEC-4	Transport encryption in production; secure cookies whenever TLS is present
NFR-SEC-5	Rate limiting on authentication, intake and search
NFR-SEC-6	Attachment authorization through the conversation; no URL-guessing path
NFR-SEC-7	Session revocation effective on the next request, not on a cache expiry
NFR-SEC-8	**Encryption at rest** for the database, object storage and backups. Transport encryption (NFR-SEC-4) protects data in flight; it does nothing for a stolen disk, a copied backup or a decommissioned volume
NFR-SEC-9	Internal operational metadata — SLA state, escalation level, priority, assignment history — is **excluded at serialisation** on every customer-facing path, fail-closed (§27.16)

13.2 Availability

ID	Requirement
NFR-AVL-1	Business-hours availability target **99.5%** *(proposal — needs a business standard)*
NFR-AVL-2	Realtime degradation must not prevent sending or reading — the application remains usable without a live channel
NFR-AVL-3	Notification transport failure never degrades conversation function
NFR-AVL-4	Planned maintenance outside business hours, using the working calendar

13.3 Performance

ID	Target *(proposals pending D-13)*
NFR-PRF-1	Conversation list and first message page **< 500 ms** at the 95th percentile
NFR-PRF-2	Message send acknowledged **< 300 ms** excluding delivery
NFR-PRF-3	Realtime delivery to a connected recipient **< 1 s**
NFR-PRF-4	Paged reads must examine a bounded number of records — **measured on the reference platform at 301 examined per page with the correct compound index versus 20,000 without it** (§38)
NFR-PRF-5	Read-state tracking must not write per scroll event
NFR-PRF-6	Search returns within **2 s** on the V1 corpus
NFR-PRF-7	SLA elapsed time is **computed from the calendar on read**, never accumulated by a job — so a corrected calendar or a retrospectively-added holiday re-derives history correctly (§23.5)
NFR-PRF-8	The after-hours queue opening must route the overnight backlog without a visible stall — the surge is bounded by arrival volume, not by queue length

13.4 Scalability

ID	Requirement
NFR-SCL-1	Design point: ~200 employees, growth to ~500 without re-architecture
NFR-SCL-2	Customer concurrency **unknown — D-13**. The architecture must not assume employee-scale concurrency for customers
NFR-SCL-3	Message volume grows without bound; paging and indexing must not degrade with history depth
NFR-SCL-4	Realtime fan-out is the first component expected to need horizontal scaling (§37.3)

13.5 Reliability

ID	Requirement
NFR-REL-1	A message is durable before delivery is attempted (**P-05**)
NFR-REL-2	No conversation state exists only in a realtime event
NFR-REL-3	A dropped connection recovers by re-reading, not by replay
NFR-REL-4	Duplicate intake from a channel retry must not create duplicate conversations
NFR-REL-5	Background failures are retried with a dead-letter path, never silently dropped

13.6 Auditability

ID	Requirement
NFR-AUD-1	Append-only ledger with no application update or delete path
NFR-AUD-2	Every audited event carries actor, action, target, time and outcome
NFR-AUD-3	Audit survives the deletion of identifying request context (§27.6)
NFR-AUD-4	Audit queryable by a restricted role without engineering assistance

13.7 Maintainability

ID	Requirement
NFR-MNT-1	Module boundaries enforced by test, not convention (§17.1)
NFR-MNT-2	Identity is replaceable by changing one component (§17.2)
NFR-MNT-3	Notification and storage transports are adapter-based
NFR-MNT-4	No runtime dependency on any other internal platform, verified by test (§36)
NFR-MNT-5	Data shapes that are expensive to migrate are decided before first write (§24.5)

13.8 Data protection

ID	Requirement
NFR-DAT-1	Customer data is held only as needed to identify, contact and converse
NFR-DAT-2	Retention periods per data class — **D-06**, requires Legal
NFR-DAT-3	Identifying request context has its own, shorter retention (§27.6)
NFR-DAT-4	A deletion obligation, if one exists, must be reconciled with the append-only ledger — **D-06**
NFR-DAT-5	No credentials, tokens or secrets in messages, logs or audit records
NFR-DAT-6	Production data is never copied to development environments

13.9 Accessibility

ID	Requirement
NFR-ACC-1	WCAG 2.1 AA as the target for both surfaces *(proposal)*
NFR-ACC-2	Full keyboard operability of conversation and composer
NFR-ACC-3	Internal-note distinction must not rely on colour alone — **it is a security signal**
NFR-ACC-4	Screen-reader announcement of new messages and state changes

13.10 Mobile and responsiveness

ID	Requirement
NFR-MOB-1	Employee surface usable on tablet and phone; field staff are not desk-bound
NFR-MOB-2	Customer surface is **mobile-first** — customers arrive on phones
NFR-MOB-3	Realtime must tolerate mobile network interruption and background suspension
NFR-MOB-4	No horizontal scrolling of primary content at common widths

13.11 Recoverability

ID	Requirement
NFR-RCV-1	Backup covers **conversations, messages, attachments and audit** — a backup that omits attachments does not restore a claim
NFR-RCV-2	Restore is rehearsed, and the rehearsal is what proves the backup

ID	Requirement
NFR-RCV-3	Recovery point objective **≤ 24 h**, recovery time objective **≤ 4 h** *(proposals — need a business standard)*
NFR-RCV-4	Audit retention independent of conversation retention

---

14. Proposed technology stack

> ***Status of this entire section: PROPOSED.*** *Nothing below has been confirmed by the Project Head or CTO. Every row is a recommendation with reasoning in §15 and an ADR in §40, and every row is open to being overruled. Where a choice depends on an unresolved *business* decision, the dependency is named.*

14.1 The stack at a glance

Layer	Proposed	Status	Required for V1?	Depends on a business decision?
Language	TypeScript, both ends	**PROPOSED**	Yes	No
Frontend framework	Next.js (React)	**PROPOSED**	Yes	No
Styling	Tailwind CSS	**PROPOSED**	Yes	No
UI primitives	Headless component library (Radix-style), styled in-house	**PROPOSED**	Yes	No
Backend framework	NestJS	**PROPOSED**	Yes	No
Backend shape	Modular monolith, enforced boundaries	**PROPOSED**	Yes	No
Database	MongoDB	**PROPOSED**	Yes	No
Realtime transport	Socket.IO over WebSocket	**PROPOSED**	Yes	No
Session	StarLink-owned signed cookie, version-revocable	**PROPOSED**	Yes	Customer half: **D-02**
Authorization	In-house permission → role → scope	**PROPOSED**	Yes	Roles: **D-11**
Attachment storage	Object-storage abstraction; local driver in dev, S3-compatible in production	**PROPOSED**	Yes	Customer uploads: **D-07**
Notification transport	Adapter interface; in-app first, email adapter next	**PROPOSED**	In-app yes	Channels: **D-01**, **D-12**
Reverse proxy	nginx (or the platform's managed equivalent)	**PROPOSED**	Production only	No
Packaging	Docker; Compose for development	**PROPOSED**	Dev yes; prod is a separate decision	No
Search	MongoDB text indexes	**PROPOSED**	Yes	Customer search: **D-08**
Background work	In-process scheduled workers	**PROPOSED**	Yes	No

14.2 Deliberately not proposed for V1

The brief asks that nothing be added because it is common. Each of these was evaluated and set aside, with the condition that would earn it a place stated as a **trigger**. None is a permanent refusal.

Component	Status	Why not now	Trigger that would change this
Redis	**FUTURE**	There is nothing for it to hold. Sessions are signed cookies, so no session store is shared. Realtime is single-process, so no cross-instance fan-out exists. Rate-limit counters fit in process	**The first moment a second API instance holds realtime connections** (§33.2), or rate limiting must be shared across instances
Message broker (Kafka, RabbitMQ)	**REJECTED FOR V1**	The only at-least-once requirement is notification delivery, and an outbox collection with a worker satisfies it at this volume (§29.4)	Fan-out to many external consumers, or notification volume where a database-backed outbox becomes the bottleneck
Dedicated search engine (Elasticsearch, OpenSearch)	**FUTURE**	MongoDB text indexes are sufficient for V1's corpus, and search is already scope-resolved before query rather than filtered after (§30)	Search exceeding **NFR-PRF-6**, or a requirement for relevance ranking, fuzzy matching, or cross-entity search
Container orchestration (Kubernetes)	**REJECTED FOR V1**	One deployable and a small team. Orchestration is operational cost with no V1 benefit	More than a handful of instances, or a platform decision made for the whole company
Microservices	**REJECTED FOR V1**	Full evaluation in §18.5. The boundaries are not yet proven, and module boundaries are cheap to move where service boundaries are not	A module developing genuinely different scaling or availability characteristics — the realtime layer is the likely first candidate
GraphQL	**REJECTED FOR V1**	The client surface is narrow and well known. REST with a small number of purpose-built read endpoints is simpler to authorize, and **authorization is the hard problem here** — a flexible query layer multiplies the places a scope check must hold	Many heterogeneous consumers with genuinely divergent read needs
Separate mobile applications	**FUTURE**	Both surfaces are responsive; the customer surface is mobile-first (**NFR-MOB-2**)	A requirement that needs device capabilities the browser cannot reach

14.3 A note on the earlier prototype

An evaluation prototype exists from earlier work and is deliberately set aside. It was built in **Python/FastAPI with a React/Vite frontend** — a different stack from the one proposed here.

Its code therefore does not carry forward. What carries forward is what it established:

- The independence architecture holds. The prototype ran with NorthStar completely stopped, and the guard tests that proved it are a *pattern* to reproduce, not files to copy (§36.2).
- The authorization model catches the defect it was designed to catch. An early version granted every member a global read, and an acceptance test caught a non-participant reading a conversation. That finding is why §26.3 step 2 is worded the way it is.

If the CTO prefers the prototype's stack over the one proposed here, §39 and §40 are where that argument belongs — the *architecture* in this document is largely stack-independent, which is deliberate.

15. Technology rationale

Each entry answers the same six questions. The test applied throughout: **does this choice serve a requirement in Part I?** A justification that would read identically for any other chat product is not a justification.

15.1 TypeScript, both ends — PROPOSED, required for V1

Responsibility	The single implementation language for API, both frontends, and shared contracts
Why it fits StarLink	The riskiest objects in this product are *shapes*: a message carrying an internal marker (**BR-24**), a participant carrying a role and an expiry (**FR-CONV-2**), a principal whose `kind` decides whether a customer may see something (**FR-CUST-1**). One language means those types are **declared once and shared** rather than restated on each side and drifting. `internal: boolean` getting lost in translation between backend and frontend is the leak in **UC-E03**, and a shared type makes that a compile error
Alternatives	Python backend with a TypeScript frontend (the prototype's shape) · Java or C# backend · JavaScript without types
Why not	Two languages means the contract is expressed twice and generated or hand-maintained. Given that the most security-critical field in the product is a boolean on a message, duplication is the wrong trade. Untyped JavaScript discards the property that makes this choice worth anything
Trade-offs	A build step everywhere. Types describe shape, not truth — a correctly-typed `internal` flag can still be set wrongly, so §27.8's controls remain necessary regardless
V1	Required. It is the premise the other choices rest on

15.2 Next.js — PROPOSED, required for V1

Responsibility	Both user surfaces: the employee workload application and the customer conversation application
Why it fits StarLink	Three specific needs. **(a)** Two surfaces with different audiences and hard isolation between them (**D-16**) — Next.js route groups give two independently built and independently deployed entry points from one codebase (§19.2). **(b)** The customer surface is mobile-first (**NFR-MOB-2**) and is often a first-time visitor, so server rendering of the initial view matters for a customer arriving on a phone on a poor connection. **(c)** The employee surface is a long-lived application shell where a client-side router is the right model. Next.js supports both postures in one framework, which a small team benefits from more than a large one
Alternatives	Plain React with Vite (the prototype's choice) · Remix · two separate SPAs · a server-rendered template stack
Why not	Vite plus React would require assembling routing, server rendering and build-splitting by hand — reasonable, but it is work with no product benefit. Two separate SPAs duplicate the design system and the API client (see §19.2 for why this was rejected). A template

	stack fights the realtime, stateful nature of a chat interface
Trade-offs	A larger framework surface than the product strictly needs, and a rendering model that must be understood to be used safely — a server component that accidentally renders employee data into a customer page would be a real defect, which is why §19.3 makes the session boundary explicit per surface
V1	Required

15.3 Tailwind CSS + headless component primitives — PROPOSED, required for V1

Responsibility	Styling, and accessible interaction primitives (dialogs, menus, tooltips)
Why it fits StarLink	**NFR-ACC-3** is unusual and decides this: the internal-note distinction *must not rely on colour alone*, because it is a security signal. That means the distinction is layered — border, label, background, icon — and must be applied identically wherever a message renders, across two separate surfaces. Utility classes composed into one shared message component make that consistency inspectable in one place. Headless primitives are proposed because **NFR-ACC-1/2** require keyboard operability and screen-reader announcement, and hand-built menus and dialogs are where accessibility silently fails
Alternatives	A full component library (Material, Ant, Chakra) · CSS modules · styled-components
Why not	A full component library brings an opinionated visual identity that must then be fought, and its components are hard to modify when a security-motivated visual requirement like **NFR-ACC-3** does not fit its model. CSS modules are perfectly workable but make cross-surface consistency a matter of discipline rather than of shared composition
Trade-offs	Verbose markup. Requires a design-token discipline established early or it degrades into arbitrary values — the token set is part of §19.4
V1	Required. The specific library for primitives is **open**; the *headless* posture is the proposal

---

15.4 NestJS — PROPOSED, required for V1

Responsibility	The API, the realtime gateway, background workers, and the module boundaries that §18 depends on
Why it fits StarLink	The central architectural requirement is **enforced module boundaries** (§18.3) — and NestJS's module system with explicit dependency injection makes a boundary violation a wiring error rather than a convention someone forgot. Two further fits: **guards** are the natural home for the "authorize before content is read" rule (**FR-AUTHZ-1**) applied uniformly rather than per-endpoint, which is exactly the failure mode §17.1 warns about; and its **WebSocket gateway** sits in the same module graph as the HTTP layer, so realtime authorization reuses the same session and permission code rather than reimplementing it — the defect §38 records as a REJECT
Alternatives	Express or Fastify directly · a Python framework (FastAPI, Django) ·

	Go
Why not	Express or Fastify alone provide no module system, so the boundary enforcement that is the point of §18 would be built and policed by hand. FastAPI is a strong framework and was the prototype's choice, but it costs the shared-types property of §15.1. Go would suit the realtime layer well and suits the CRUD-and-authorization majority of this product less
Trade-offs	Opinionated and heavier than a bare router; decorator-driven code can obscure control flow; more concepts for a new team member to absorb before being productive
V1	Required

15.5 MongoDB — PROPOSED, required for V1

Responsibility	Conversations, messages, participants, identity, ownership records, notifications, and the audit ledger
Why it fits StarLink	Four reasons specific to this data, not generic ones. **(1) Messages vary by type in the same stream** — a customer message, an internal note, an attachment-bearing message and a reply-to all sit in one conversation and are read as one page; a document model holds that variation without a nullable column per variant. **(2) Participants are read with their conversation, always** — **FR-CONV-2** requires role, added-by, added-at and optional expiry per participant, and every authorization decision (§26.3) needs the participant set at the same moment it needs the conversation. As a nested array that is one read; as a join table it is two. **(3) The audit ledger is append-only with no referential integrity requirement** (**FR-AUD-1**) — it references actors and targets but nothing references it (§24.2), so the relational guarantees a normalised store provides are guarantees this data does not need. **(4) The paging pattern is already proven on this exact shape** — §38 records a measured result on the reference platform: 301 documents examined per page with the right compound index against 20,000 without
Alternatives	PostgreSQL · PostgreSQL with JSONB for message bodies · a document store plus a relational store
Why not	**PostgreSQL is a legitimate choice and would work.** The honest summary: it would give stronger multi-row atomicity for the claim-and-assign operation (§21.2) and better ad-hoc reporting, at the cost of a join per authorization decision and a migration for every message-type variation. The deciding factor is that the *dominant* access pattern is "one conversation, its participants, one page of its messages" — which is a document read — and the reporting need in V1 is limited to queue depth and load (**FR-REP-1/2**). Two stores were rejected as operational cost a small team should not take on for V1
Trade-offs — stated plainly, because they are real	**(a) Multi-document atomicity is not free.** The atomic claim in **FR-ROUTE-3** must be a single-document conditional update, and any operation genuinely spanning documents needs an explicit transaction on a replica set. **(b) No joins for the queue-plus-ownership view**, so §24.5 accepts controlled denormalisation and states which fields are duplicated and why. **(c) Text search is weaker** than a dedicated engine — §30 assesses whether that is sufficient for V1 and names the trigger where it stops being. **(d) Schema discipline moves into the application**, so §24 fixes the shapes that cannot be changed cheaply before first write
V1	Required. **Replica set required even for a single node**, because transactions and change awareness depend on it

15.6 Socket.IO over WebSocket — PROPOSED, required for V1

Responsibility	Delivering the six live events justified in §20.2 to authorized, connected clients
Why it fits StarLink	**Rooms map exactly onto conversations**, which is the authorization unit in this product (§26.3), so "who receives this event" becomes "who is joined to this room" and the join is the place the scope check goes. **Automatic reconnection with backoff** matters because of **NFR-MOB-3** — customers arrive on phones, and phones suspend, change networks and lose signal; the alternative is writing that reconnection logic by hand on the surface where it matters most. And because realtime is **additive** (§20.3), reconnection can be a plain re-read, which is a far simpler contract than replay
Alternatives	Raw WebSocket · Server-Sent Events · long polling
Why not	Raw WebSocket is leaner and has no client library, but reconnection, heartbeat and room fan-out would be hand-built — and hand-built reconnection on a mobile customer surface is where subtle bugs live. SSE is one-directional, which fits message delivery but not typing indicators. Long polling is a fallback, not a design
Trade-offs	A protocol layer above WebSocket and a required client library, so it is not interoperable with a plain WebSocket client. Its multi-instance story depends on an adapter — which is precisely the **Redis trigger** named in §14.2 and §33.2, and stating that now is better than discovering it later
V1	Required for the six events in §20.2. **The application must remain fully usable without it** (**NFR-AVL-2**) — that is a hard constraint, not an aspiration

15.7 Session and authorization, in-house — PROPOSED, required for V1

Responsibility	Issuing, verifying and revoking sessions; deciding what a principal may do
Why it fits StarLink	**Revocation must be real** (**FR-AUTH-2**): a role change or deactivation takes effect on the *next request*, because **FR-EMP-2** requires deactivation to end access immediately and **UC-E07** depends on it. A signed cookie carrying a version, checked against the principal on every request, delivers that in one lookup. Authorization is in-house because the model is **permission → role → scope with participation as a conversation-scoped grant** (§26.3) — that is not a standard RBAC library's shape, and bending one to fit it would obscure the exact property §38 records as the reference platform's most serious defect
Alternatives	An identity provider (Auth0, Cognito, Keycloak) · a server-side session store · JWT with short expiry and refresh tokens · an off-the-shelf RBAC library
Why not	An external identity provider is a **runtime dependency on a third party for the ability to log in**, which is a poor trade for an internal tool — and it would compete with the HRMS, which **A-13** expects to become the identity source. It remains the natural answer for single sign-on later (§36.1). A server-side session store adds a lookup and, in V1, a component (Redis) that §14.2 shows nothing else needs. Bare JWTs are the wrong shape here: their selling point

	is avoiding a revocation check, and revocation is a requirement
Trade-offs	We own the security-critical code, so it must be reviewed properly — §45 Phase 3 makes a security review a gate, not a courtesy. Customer authentication is genuinely undecided (**D-02**, **D-03**) and this proposal covers the employee half plus the *shape* of the customer half
V1	Required

15.8 Object-storage abstraction — PROPOSED, required for V1

Responsibility	Holding attachment bytes; serving them only to entitled readers
Why it fits StarLink	**FR-ATT-1** requires authorization through the conversation rather than through URL knowledge, which means bytes must **never** be reachable by a public URL — so the abstraction exists to keep the application in the request path. An interface with a local-filesystem driver for development and an S3-compatible driver for production also means **NFR-RCV-1** (backup must cover attachments) is a deployment concern rather than a code change, and **D-07** can be answered either way without touching the conversation domain
Alternatives	Commit to a named provider now · store bytes in MongoDB · a filesystem in production
Why not	Naming a provider now is a decision with no information behind it — hosting is not settled (§37.1). Storing bytes in the database inflates it, complicates backup, and pushes claim documents through a store sized for messages. A production filesystem makes multiple instances and backup harder for no gain
Trade-offs	An interface that only ever has one real implementation is overhead — mitigated because the local driver is genuinely used every day in development. Streaming through the application costs bandwidth versus a signed direct URL; §28.4 explains why that cost is accepted, and when a scoped time-limited URL becomes acceptable
V1	Required. **The provider is deliberately unnamed**

15.9 Notification adapters — PROPOSED, in-app required for V1

Responsibility	Delivering actionable notifications through whichever channels the business chooses
Why it fits StarLink	The channel set is **not knowable yet**. **D-01** decides how customers are reached, **D-12** decides what they are told, and the employee channel may differ from both. An adapter interface lets in-app notification ship in V1-A while the rest waits on decisions — and **P-05** (record first, then attempt delivery) means a missing or failing adapter cannot cost a stored message
Alternatives	Commit to a provider now · a transactional email service directly · a broker-driven pipeline
Why not	Committing now would be inventing a business answer, which the brief forbids. A direct integration to one service hard-codes an assumption **D-01** may overturn. A broker is evaluated and rejected in §29.4 — the outbox pattern meets the same requirement at this volume

Trade-offs	Adapters constrain themselves to a common denominator, so a channel-specific capability (WhatsApp template messages, for instance) needs either a leaky interface or a channel-specific path. If **D-01** answers WhatsApp, expect the latter, and §37.2 already notes the separate webhook receiver that answer implies
V1	In-app required. Email adapter expected in V1-A. All others follow **D-01** and **D-12**

15.10 nginx or equivalent reverse proxy — PROPOSED, production only

Responsibility	TLS termination, static asset serving, request and connection limits, routing the two surfaces to their hostnames
Why it fits StarLink	**NFR-SEC-4** requires transport encryption in production, and **D-16** with §19.2 means two hostnames must resolve to two different builds. A proxy is the ordinary place for both. It is also the right layer for the coarse connection limits that complement the application's own rate limiting (§27.5)
Alternatives	A cloud load balancer or managed ingress · Node serving TLS directly · a CDN in front
Why not	A managed equivalent is **equally acceptable and may be preferable** — this proposal is about the *role*, not the product. Terminating TLS in the application is workable and gives up a natural place for static serving and connection limits
Trade-offs	One more component to configure and keep patched. Its configuration becomes security-relevant, so it belongs under change control
V1	Production only. **Not required in development**, and it should stay out of the development setup so a developer's first run does not need it

---

15.11 Docker — PROPOSED for development; production packaging is a separate decision

Responsibility	Reproducible development environments; a deployment artefact
Why it fits StarLink	A new developer needs the API, both frontends and a MongoDB **replica set** (required per §15.5) running together. Compose makes that one command instead of a page of instructions, which matters more than it sounds: an independence claim (§36.2) is only credible if a clean checkout genuinely starts
Alternatives	Locally installed runtimes and database · a dev container · a hosted development database
Why not	Local installation is what the prototype did, and it produced exactly the friction expected — including a portable database unpacked by hand to avoid a system-level install. A shared hosted database breaks the independence property and lets one developer's data affect another's
Trade-offs	Resource overhead on developer machines; file-watching across the container boundary can be slow, so the frontends may be better run on the host against a containerised API and database
V1	Development: proposed. **Production: how the artefact is deployed**

	is a PRODUCTION DECISION (§37.1) and is explicitly not settled here. Compose is a development tool and this document does not propose it as a production topology

---

15.12 Summary — what each choice is actually buying

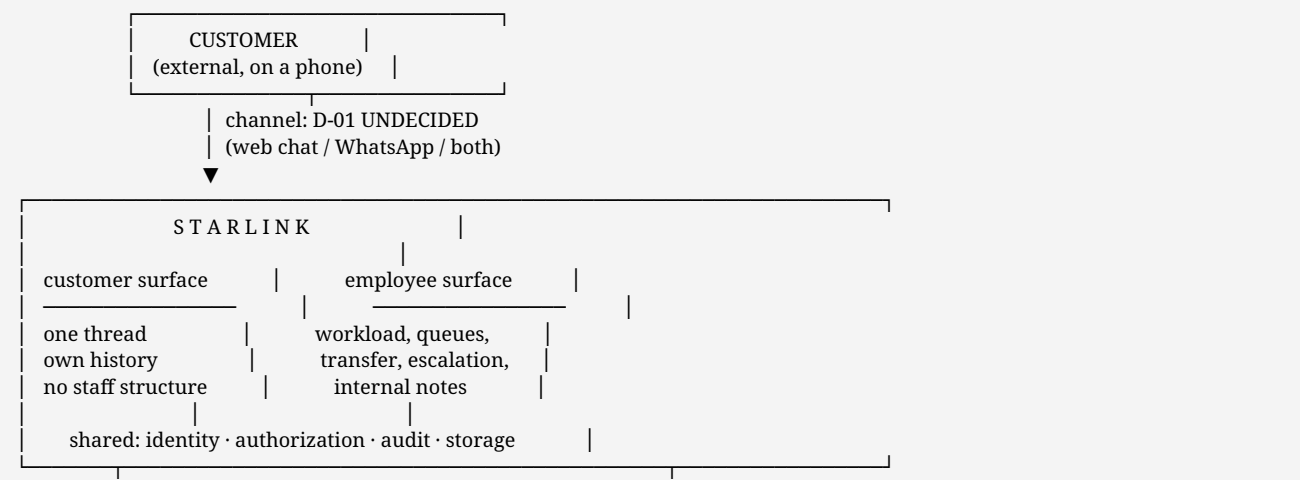
Choice	The requirement it exists to serve
TypeScript both ends	One declaration of the shapes that carry security meaning (**UC-E03**, **FR-CONV-2**)
Next.js	Two isolated surfaces from one codebase (**D-16**), mobile-first customer entry (**NFR-MOB-2**)
Tailwind + headless primitives	Internal-note distinction that is not colour-only, applied identically everywhere (**NFR-ACC-3**)
NestJS	Module boundaries enforced by wiring, and one guard layer for **FR-AUTHZ-1**
MongoDB	Conversation-shaped reads, per-type message variation, append-only audit, proven paging
Socket.IO	Rooms as the authorization unit; reconnection on the mobile surface (**NFR-MOB-3**)
In-house session and authorization	Revocation effective next request (**FR-AUTH-2**); a permission model no library matches
Storage abstraction	Bytes never publicly reachable (**FR-ATT-1**); backup as a deployment concern (**NFR-RCV-1**)
Notification adapters	A channel set that **D-01** and **D-12** have not chosen yet
Reverse proxy	TLS (**NFR-SEC-4**) and two hostnames for two surfaces
Docker	A clean checkout that starts, which is what makes independence credible

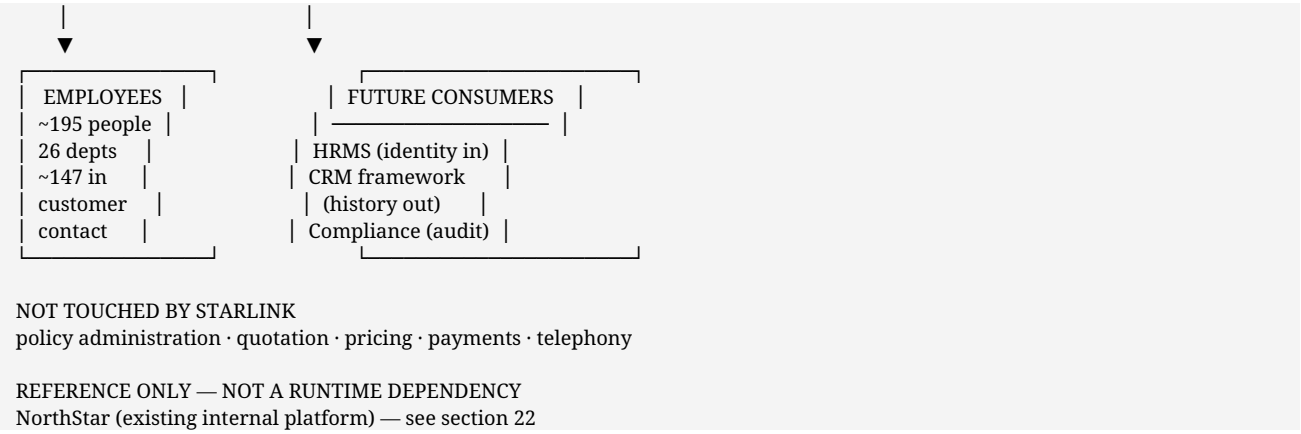
---

16. High-level system architecture

16.1 Business context — diagram 1

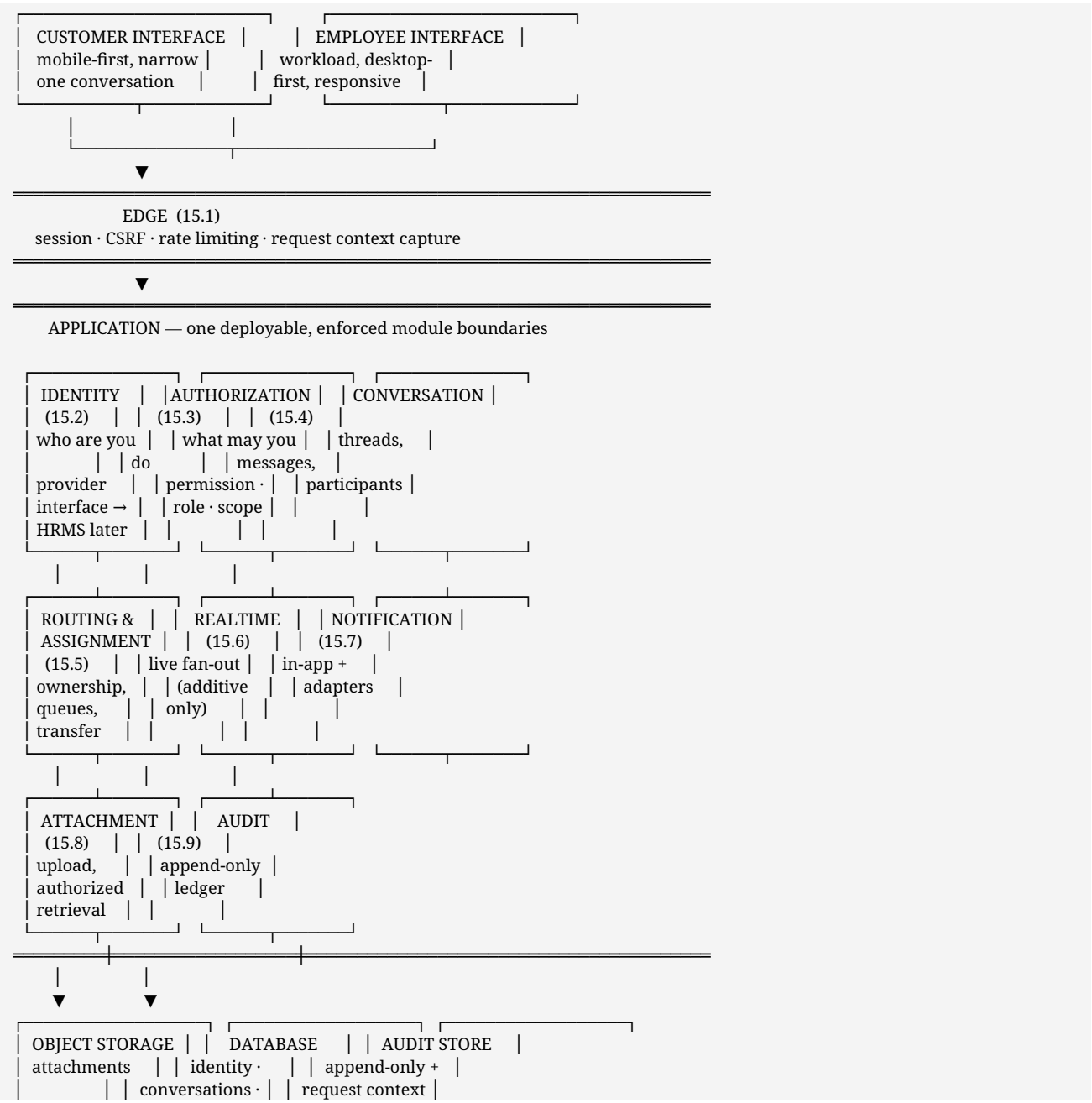
Where StarLink sits, and what it does and does not touch.





16.2 High-level architecture — diagram 2

Components named before technologies. Each is explained in §17.



16.3 Architectural style

A modular monolith with enforced internal boundaries — PROPOSED. One deployable unit containing the components above as modules that may interact only through declared interfaces. Version 1.0 of this document stamped this *decided*; it is **downgraded to PROPOSED** here, because it is a technical judgement the CTO should be free to overrule. The full evaluation is §18.5 and the record is **ADR-04**.

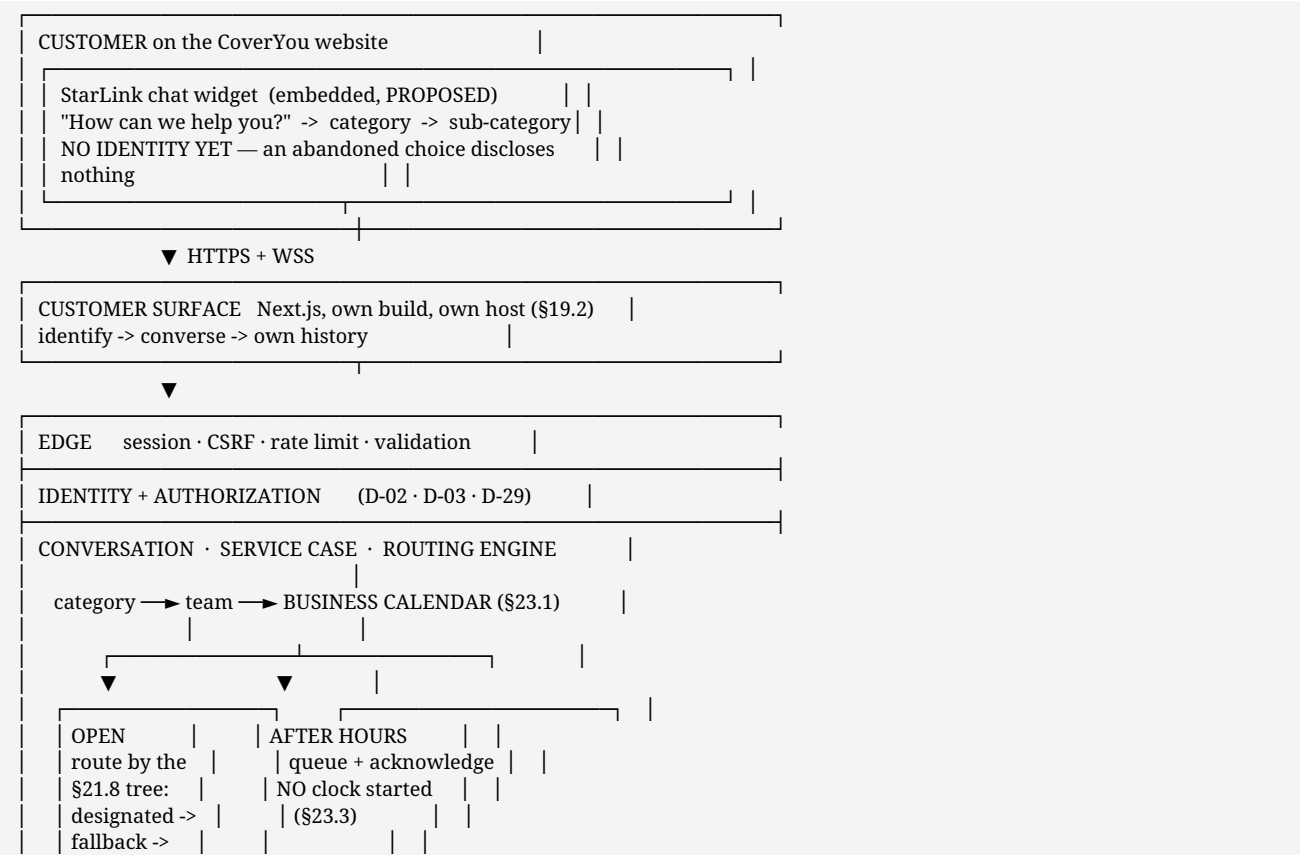
Why not microservices in V1	
No scaling problem exists yet	~200 employees; customer volume is **D-13**, unknown
The team is small	Distributed operational burden would exceed delivery capacity
The boundaries are not yet proven	Module boundaries can be moved cheaply; service boundaries cannot
Conversation, identity and audit are transactionally related	A message and its audit record crossing a network boundary introduces failure modes with no compensating benefit

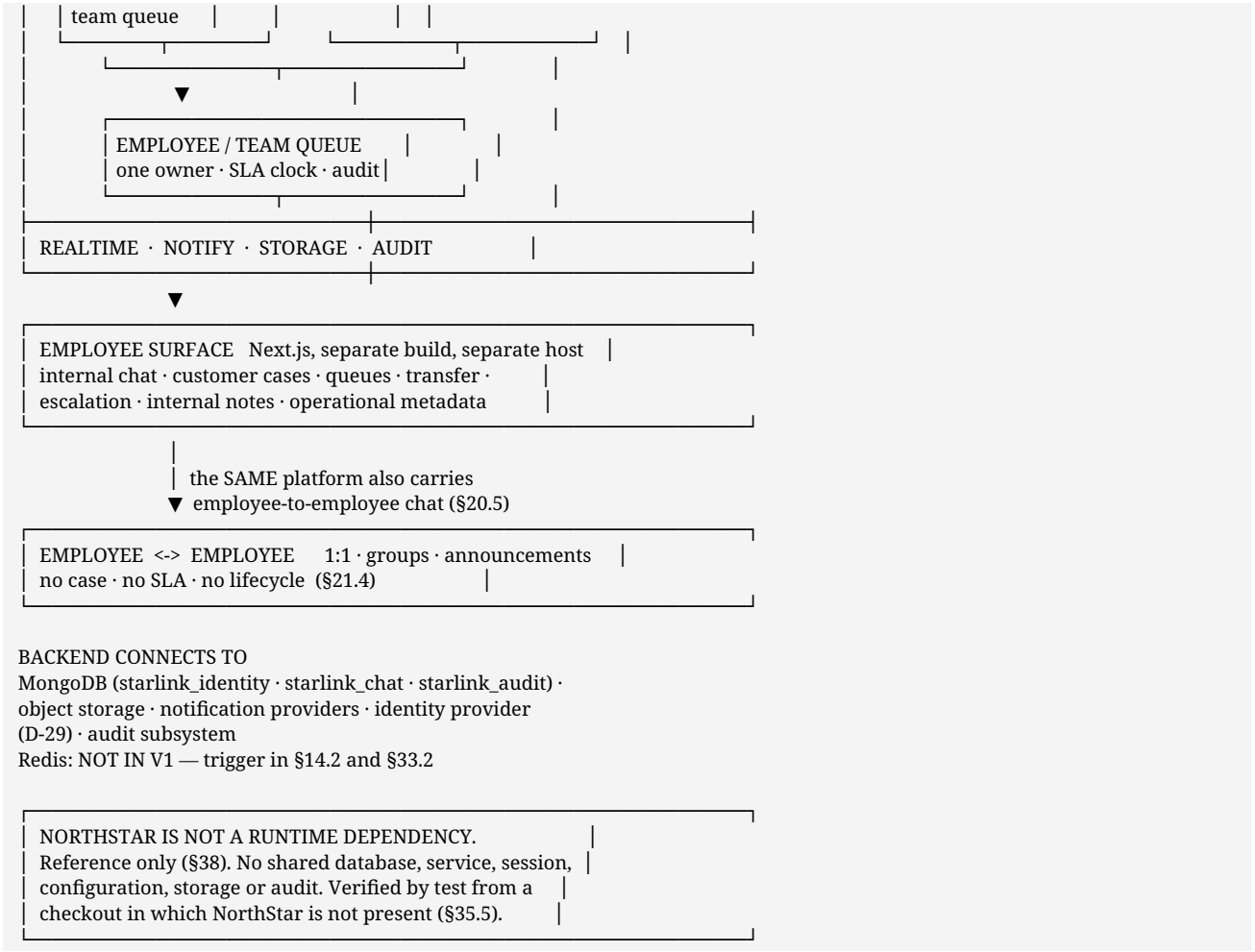
Enforced boundaries are the point. Modules are separated by tests that fail the build on a cross-boundary import. If a module later needs to become a service, the boundary already exists and the extraction is mechanical rather than archaeological.

The honest caveat. If **D-01** is answered "WhatsApp", the inbound webhook receiver may warrant a separate process — a public endpoint with different availability and security characteristics from the main application. That is one additional process, not a decomposition, and it is contingent on a decision not yet made.

16.6 The customer entry path — diagram 21

Version 2.0's diagrams began at "customer client". The actual V1 entry point is the CoverYou website, and the category step happens **before** identity — both of which change what the edge sees.

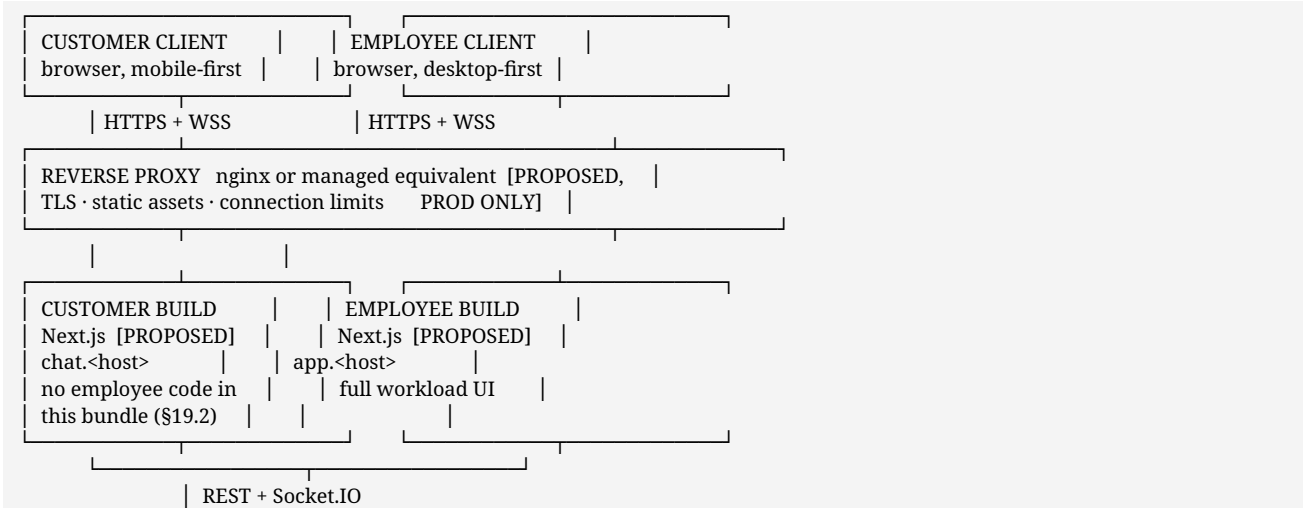


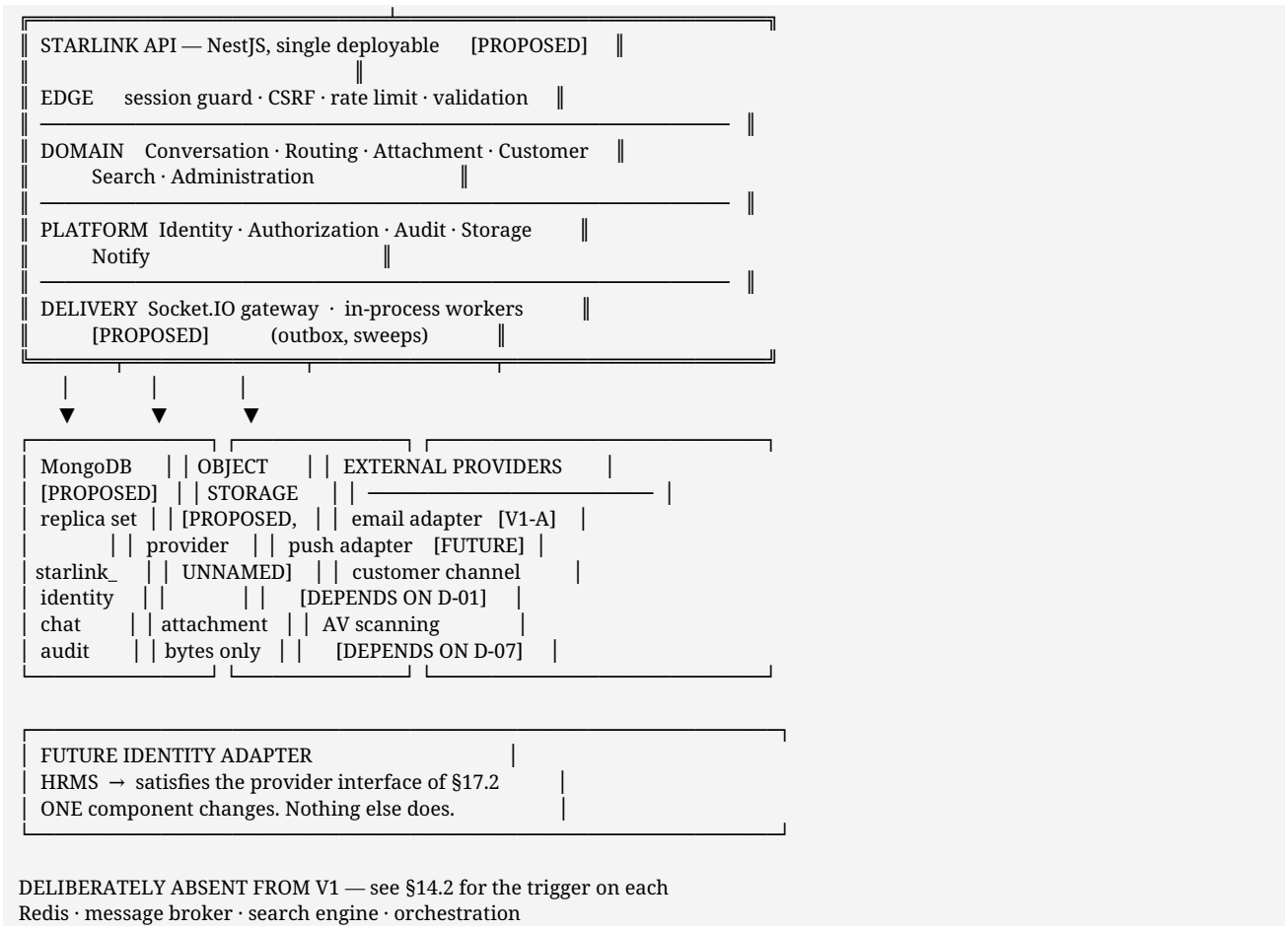


Two properties this diagram is drawn to show. The category step sits **before** identity, so the edge handles an unidentified visitor making a choice — which is why intake is rate-limited (§27.5). And the business-calendar check sits **inside** routing rather than beside it, because *are we open* is a question about the team the category selected, not about the company.

16.4 The same architecture with technology named — diagram 13

The diagram in §16.2 is deliberately technology-free: it is what must exist. This one adds the **PROPOSED** technology, and is the version to argue with.





16.5 What the diagram commits to, and what it does not

Committed as an architectural requirement	Merely proposed
Two separate client builds (**D-16** pending)	That they are Next.js
Authorization before content on every path	That it is a NestJS guard
Own databases, starlink_* namespace, refusing others	That the database is MongoDB
Realtime additive and degradable	That the transport is Socket.IO
Attachment bytes never publicly reachable	The storage provider
Notification off the write path	The email provider
Audit append-only, context separately retained	The database it lives in
No runtime dependency on another platform	Everything about deployment

The left column survives a change of stack. The right column is what §39 asks you to approve or overrule.

---

17. Component architecture

Each component is described as **responsibility · inputs · outputs · dependencies · why it exists**.

17.1 Edge

Responsibility	Establish who is calling and whether the call is well-formed, before any domain logic runs
Inputs	HTTP requests, session cookies, realtime upgrade requests
Outputs	An authenticated request context, or a refusal
Dependencies	Identity (to validate the session)
Why it exists	Session validation, CSRF defence and rate limiting are cross-cutting. Implemented per-endpoint they are implemented inconsistently — and the inconsistent endpoint is the one that gets exploited

Concerns: session read and verification · CSRF defence on state-changing requests · rate limiting on authentication, intake and search · request-context capture for audit · uniform refusal responses.

17.2 Identity

Responsibility	Answer "who is this principal, and what are their organisational attributes"
Inputs	Principal identifier; directory queries; scope-resolution requests
Outputs	Principal record, directory listing, organisational attributes, reporting scope, contact channels
Dependencies	Identity data store
Why it exists	**Identity will change.** A central HRMS is being built. Every other component must be insulated from that change

The critical design property. Identity is consumed through a **provider interface** with five operations: resolve a principal · list the directory · read organisational attributes · resolve reporting scope · resolve contact channels. V1 implements this against StarLink's own store. When the HRMS arrives a second implementation satisfies the same interface and **no other component changes**. This directly serves **O-08**.

The join key. A single stable principal identifier is the only key any other component uses. Names, usernames and email addresses are display and contact data, never join keys — because they change, and because a rename must not orphan history.

17.3 Authorization

Responsibility	Decide whether this principal may perform this action on this conversation
Inputs	Principal, permission, conversation, role assignments, current time
Outputs	Allow or deny
Dependencies	Identity (for scope attributes)
Why it exists	**P-02.** Conflating authentication and authorization is the defect class that produces "any logged-in user can read anything"

Model: **permission → role → scope**. A permission is a specific action. A role is a named set of permissions. A scope bounds where a role applies — global, department, team, or a single conversation.

Three rules that are easy to get wrong and expensive to get wrong:

1. **Participation is a conversation-scoped grant, not a global one.** An ordinary member holds no company-wide read permission; their access comes from being in the conversation.

2. **An unknown permission is denied**, never treated as unrestricted.

3. **Expiry is read from the clock.** A time-boxed grant lapses because time passed, not because a cleanup job ran — so no job's failure can silently extend someone's access.

17.4 Conversation

Responsibility	Own conversations, messages and participants — the durable record
Inputs	Create, send, page, add or remove participant, state transition
Outputs	Conversations, message pages, participant sets, domain events
Dependencies	Authorization (before any content read), database, audit
Why it exists	The product's core asset. Everything else supports it

No path to message content bypasses authorization. A single operation loads a conversation *and* authorizes it; message reads take the conversation from that operation, never from an identifier supplied by the caller.

17.5 Routing and assignment

Responsibility	Decide who owns an incoming customer conversation, and maintain ownership over time
Inputs	New customer conversation, relationship history, team configuration, availability, transfer and escalation requests
Outputs	Assignment records, queue state, ownership changes, domain events
Dependencies	Identity (teams, attributes), conversation, notification, audit
Why it exists	Ownership is what makes a customer conversation answerable. Without it everything is everyone's problem, and therefore nobody's

Configuration-driven rather than hard-coded — **D-04** is unanswered and the department shapes differ (§21.2). Assignment history is append-only, so cover and return are both retained.

17.6 Realtime

Responsibility	Deliver events to connected clients
Inputs	Domain events; subscribe requests
Outputs	Events to authorized subscribers
Dependencies	Authorization (at subscribe and at session end), conversation
Why it exists	Chat without live delivery is email. But it is **additive only** (§20.3)

Two invariants: **authorize at subscribe and invalidate at session end** — a live channel outliving its session is an authorization hole; and **events carry identifiers, not bodies**, to any subscriber whose entitlement for that content has not been established.

17.7 Notification

Responsibility	Tell someone who is not looking that something needs them

Inputs	Domain events, recipient preferences, contact channels
Outputs	In-app notifications; external dispatch through adapters
Dependencies	Identity (contact channels), conversation
Why it exists	Realtime reaches people who are present. Most people are absent most of the time

The record is written first, then delivery is attempted (**P-05**). Notification failure never blocks or reverses a stored message. External channels are adapters because the customer channel is **D-01** and the employee channel may differ from it.

17.8 Attachment

Responsibility	Accept files, store them, and serve them only to entitled readers
Inputs	Uploads; retrieval requests
Outputs	Attachment references; file responses; audit events on download
Dependencies	Authorization, conversation, object storage, audit
Why it exists	Claims and grievance work is document work. A claim without its documents is not a claim

Authorization is through the **conversation**, never through knowledge of a URL. References are derived on read rather than stored, so a route change is not a data migration. **Download is the audited event** — the upload is already attributable through its message; the download is the exfiltration event.

17.9 Audit

Responsibility	Record what happened, permanently, in a form that answers questions after an incident
Inputs	Security-relevant and ownership-relevant events
Outputs	Append-only records; restricted query results
Dependencies	A separate, protected store
Why it exists	**O-05.** After an incident the question is not "what does the data say now" but "who did what, when"

Two stores, deliberately. The ledger holds actor, action, target, time and outcome, retained as long as the business requires. Identifying request context — network address, user agent — is held **separately with its own shorter retention**. This is what lets a retention obligation be honoured without puncturing an append-only ledger (§27.6).

For a security-critical action, **audit write failure fails the action**. An unaudited role assignment is worse than a refused one.

17.10 Each component, mapped to proposed technology

Component	Proposed realisation	Status
Edge (§17.1)	NestJS guards, interceptors and validation pipes	**PROPOSED**
Identity (§17.2)	NestJS module over starlink_identity;	**PROPOSED**

Component	Proposed realisation	Status
	provider interface with a local implementation	
Authorization (§17.3)	In-house decision function called from one guard and one object check	**PROPOSED**
Conversation (§17.4)	NestJS module over starlink_chat	**PROPOSED**
Routing (§17.5)	NestJS module; atomic claim by conditional update	**PROPOSED**
Realtime (§17.6)	Socket.IO gateway in the same module graph	**PROPOSED**
Notification (§17.7)	Adapter interface + outbox collection + in-process worker	**PROPOSED**
Attachment (§17.8)	Storage driver interface; local in dev, S3-compatible in production	**PROPOSED**
Audit (§17.9)	Module owning starlink_audit exclusively; append and query only	**PROPOSED**

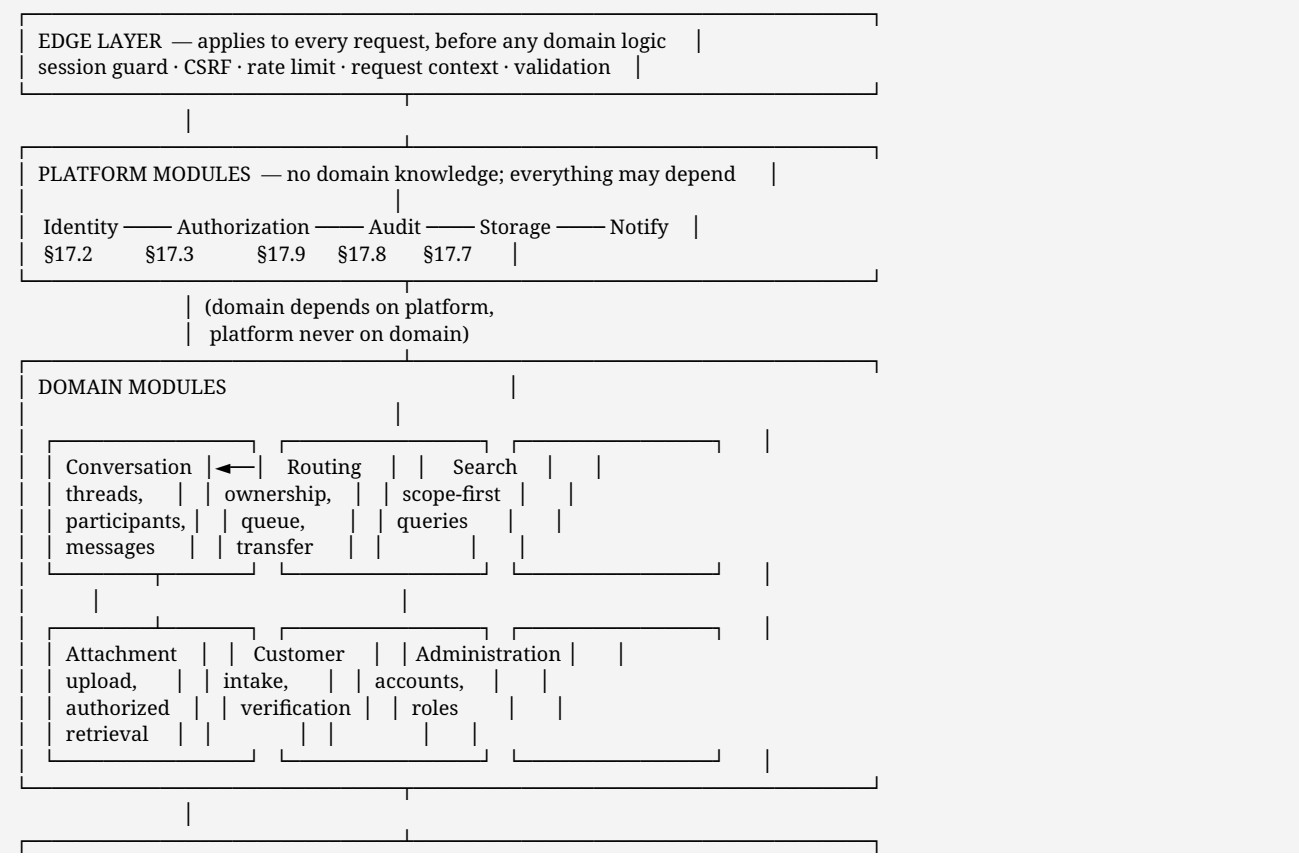
No component's responsibility changes if the technology does. That is the property being claimed here, and §39.4 states which rows would actually shift if the stack were overruled.

---

18. Backend architecture

PROPOSED. How the API would be organised if NestJS (§15.4) is approved. The *structure* below survives a change of framework; the mechanism for enforcing it does not.

18.1 Module map — diagram 11



```
│ DELIVERY MODULES  — read from domain, never written to by it      │
│  Realtime gateway (§20)    ·    Background workers (§29.4)       │
└──────────────────────────────────────────────────────────────────┘

DEPENDENCY RULE — one direction only
edge → domain → platform.  Delivery observes domain events.
A platform module importing a domain module FAILS THE BUILD.
```

18.2 Module responsibilities and ownership

The column that matters is the last one. Business logic living in two modules is business logic that will diverge.

Module	Owns	Depends on	Must never
Identity	Principals, organisation attributes, teams, the directory, the provider interface (§17.2)	Data store	Know what a conversation is
Authorization	Permissions, roles, scopes, the decision in §26.3	Identity	Read message content, or decide *business* ownership
Audit	The append-only ledger and the separately-retained request context	Data store	Be depended upon by any domain module's correctness
Storage	Object put and get, driver selection	—	Decide who may read a file
Notify	Adapter selection, the outbox, delivery attempts	Identity (contact channels)	Block a write
Conversation	Threads, participants, messages, read state, view state, lifecycle transitions	Authorization, Audit, Storage	Decide *who* an incoming conversation goes to
Routing	Ownership records, the queue, claim, transfer, escalation, cover	Conversation, Identity, Notify, Audit	Write message content
Attachment	Upload validation, metadata, authorized retrieval	Conversation, Storage, Authorization, Audit	Serve bytes without a conversation check
Customer	Intake, verification, the customer principal	Identity, Conversation, Authorization	Appear in any employee-scoped query result
Search	Scope resolution, then query	Authorization, Identity	Filter results after querying (§30.2)
Administration	Accounts, role assignment, deactivation	Identity, Authorization, Audit	Confer message read access (**FR-AUTHZ-7**)
Realtime	Connections, rooms, subscribe-time authorization	Authorization, Conversation	Hold state that exists nowhere else (**FR-RT-1**)

The boundary that carries the most weight: *Conversation* owns content and *Routing* owns ownership. Keeping them apart is what makes **P-03** ("participation is not ownership") a structural property rather than a convention. A single module owning both would make it trivially easy to let a participant reply to a customer, which is exactly what **D-04a** proposes to forbid.

18.3 How boundaries are enforced

Mechanism	What it prevents
Explicit module imports and injected providers	A module reaching into another's internals without declaring it
A dependency-direction test in the build	Platform importing domain; domain importing delivery
Each module exposing a service interface, never its data access	Two modules writing the same collection
One data-access layer per module, private to it	Ownership of a shape becoming ambiguous

This is the difference between a modular monolith and a monolith. Without the build-time check it is a naming convention, and naming conventions decay. §39 lists this as a decision needing approval because it imposes a real constraint on how the team works.

18.4 Where authorization is enforced — and where it is not

```
REQUEST
  |
  ▼
[1] EDGE GUARD —— is there a valid, unrevoked session?
  |             → 401 if not.  This is AUTHENTICATION only.
  |               It says nothing about what may be read.
  ▼
[2] ROUTE GUARD — does this principal hold the permission this
  |             operation requires, in principle?
  |             → 403 if not.  Coarse: catches "customer calling
  |               an employee-only operation".
  ▼
[3] OBJECT CHECK — THE ONE THAT MATTERS.
  |             Load the conversation AND authorize it in a single
  |             operation (§17.4). Message reads take the object
  |             from that operation — never an id from the caller.
  |             → 403, identical response to "does not exist".
  ▼
[4] FIELD FILTER — serialisation strips what this principal may not
                see (internal notes for a customer; participant
                roles for a customer).
                A LAST LINE OF DEFENCE, NOT A SECURITY BOUNDARY.
```

Why step 3 is stated so emphatically. §38 records that the reference platform treated membership as authorization, and an early prototype of this product reproduced the defect: every member held a global read permission, and a non-participant successfully read a conversation. The fix was structural — participation grants *that conversation* and nothing else — and step 3 is where that fix lives.

Why step 4 is explicitly not a boundary. If a customer's entitlement is only enforced at serialisation, then any new endpoint that forgets to serialise correctly leaks. Field filtering is defence in depth; the decision is made at step 3.

18.5 Modular monolith versus multiple services — PROPOSED: modular monolith

> ***Status change from v1.0.*** *Version 1.0 stamped this *"Decided — architecture"*. It is downgraded here to* ***PROPOSED****, because it is a technical judgement the CTO should be free to overrule.*

	Modular monolith — proposed	**Multiple services**
Deployables	One (plus one contingent webhook receiver, §37.2)	Six to ten
Cross-module call	A function call	A network call that can fail
Message + its audit record	One process, one failure domain	Two processes, needs compensation
Atomic claim (**FR-ROUTE-3**)	A single conditional update	Distributed coordination
Scaling	Whole application together	Per service
Boundary enforcement	Build-time test	Physical, unavoidable
Operational cost	One pipeline, one log stream, one dashboard	Multiplied, plus tracing
Team fit	Small team	Needs more people than this has

Why the monolith is proposed. No requirement in Part I demands independent scaling or independent availability of any module. **NFR-SCL-1** puts the design point at roughly 200 employees, and customer volume is unknown (**D-13**) — deciding a service topology against an unknown load is guessing. Meanwhile three operations in Part I are transactionally related and cheap in one process: a message and its audit record, an assignment and its notification, a claim and its ownership record.

The honest argument against. Boundaries enforced by a test can be relaxed by editing the test. A service boundary cannot. The counter is that a boundary you can move cheaply is an asset while the boundaries are still being learned — and they are.

What would change the recommendation: the realtime layer developing genuinely different scaling characteristics (§33.2), or a WhatsApp answer to **D-01** requiring a separately-available public webhook receiver — which is one extra process, not a decomposition.

18.6 Background work — PROPOSED

Job	Trigger	Why not inline
Notification delivery	Outbox rows pending (§29.4)	An external provider must never delay a message write (**P-05**)
Conversation state sweep	Schedule	`resolved → closed` on window expiry (§21.4) is time-driven
Waiting-beyond-standard detection	Schedule	Drives the alert in **FR-REP-1**
Request-context expiry	Store-level TTL	Retention (§31.4) must not depend on a job having run

In-process scheduled workers are proposed, not a separate worker service. Two reasons: the work is small, and every job here is idempotent, so a duplicate run from two instances is harmless. **The exception is noted honestly** — if more than one instance runs the scheduled sweeps, they need either a leader or an idempotency guard per run; §33.2 names that as part of the multi-instance step.

---

19. Frontend architecture

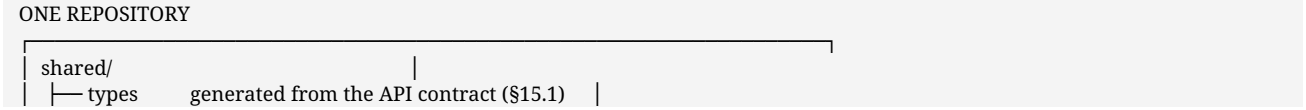
PROPOSED. Consistent with **D-16** (separate customer surface, still awaiting confirmation). If D-16 is answered "one surface with role-based hiding", §19.2 is the section that changes.

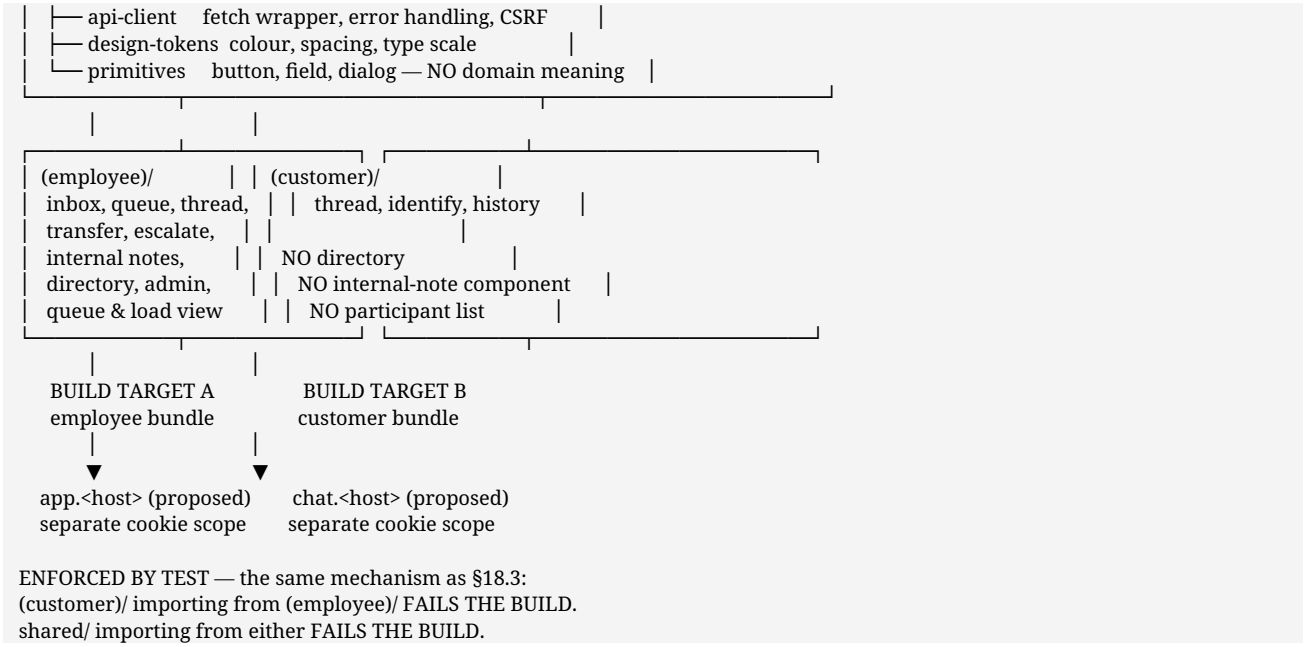
19.1 Two surfaces, and why they are genuinely different

	Employee surface	Customer surface
Mental model	A **workload** — many conversations, queues, load	A **conversation** — one thread
Primary device	Desktop, tablet secondary (**NFR-MOB-1**)	**Phone first** (**NFR-MOB-2**)
Session	Working-period length	Bounded, shorter (**UC-C05**)
Navigation	Application shell, client-side routing, persistent	Near-linear; often a single view
First paint	Behind a login, warm	Cold, first-time, possibly poor connection
Directory access	Yes	**Never**
Internal notes	Visible, and visually unmistakable	**Must not exist in the delivered bundle**

That last row is the architectural point. It is not enough for the customer's *screen* not to show internal notes — the customer's **build** should not contain the components that render them.

19.2 One project, two builds — diagram 12





ENFORCED BY TEST — the same mechanism as §18.3:
(customer)/ importing from (employee)/ FAILS THE BUILD.
shared/ importing from either FAILS THE BUILD.

Why one project rather than two. The alternative — two repositories — duplicates the API client, the generated types and the design tokens, or forces them into a published internal package. For a small team that is real recurring cost. One project with an enforced import boundary gets the isolation without the duplication, and the boundary is checked by the same kind of test that guards the backend modules.

Why two builds rather than one. A single bundle ships employee code to the customer's browser. Even with correct runtime checks, the *code* is there to be read, and the isolation then rests entirely on runtime behaviour. Separate builds mean the customer bundle cannot leak what it does not contain.

Separate hostnames are proposed so cookie scope, CSP and proxy rules differ per surface. Hostnames are a **PRODUCTION DECISION** (§37.1).

19.3 Session handling per surface

	Employee	Customer
Cookie	HttpOnly, SameSite-restricted, scoped to the employee host	Same properties, scoped to the customer host
Read by JavaScript	Never — it is HttpOnly	Never
Session state in the client	Only "authenticated: yes/no" plus the principal's display identity and permission set	Only "verified: yes/no"
Server-side rendering	Session read server-side for the initial shell	Session read server-side for the initial thread
On 401	Redirect to sign-in, preserving intended destination	Show *the session ended*, offer re-verification (**UC-C05**)
On 403	Show a refusal that does not disclose existence (§27.3)	Same

A specific hazard, named because the framework makes it easy. With server-side rendering, data fetched on the server is serialised into the page. Anything fetched for a customer page is therefore *in the customer's HTML*. So the rule is: **customer pages fetch only through customer-scoped operations** — never a shared operation that returns a fuller object and relies on the component not to render the extra fields. This is §18.4 step 4 restated for the frontend: field filtering is not a boundary.

19.4 Conversation and message rendering

Concern	Approach
Thread list	Paged on last activity, cursor-based (**FR-MSG-3**)
Message page	Newest page first, oldest-first within the page; older pages fetched on scroll-up
Message identity	Server-assigned id is the render key. **Never** an array index — a live insert would re-key every row
Optimistic send	Rendered immediately in a *pending* state, reconciled by server id, marked failed on error with a retry. Never silently dropped
Internal note	One shared component, one code path, **layered distinction**: border, background, persistent text label, icon. **Not colour alone** (**NFR-ACC-3**)
Internal composer	A visibly distinct mode that **does not silently reset after send** (**UC-E03**), because a reset that goes unnoticed is how the next message goes to the customer
Attachments	Metadata rendered from the message; bytes fetched through the authorized endpoint (§28.4), never a direct storage URL
Long threads	Virtualised list; the page size is already bounded server-side

19.5 Realtime state in the client

The client rule follows **FR-RT-1**: realtime is additive.

```
MOUNT ──► fetch state over HTTP (the source of truth)
      │
      ├──► subscribe to the conversation room
      │
      ▼
EVENT ARRIVES
  │
  ├─ known id already present? ──► ignore (duplicate, §20.5)
  ├─ is it the next in sequence? ──► apply
  └─ is there a gap?             ──► RE-FETCH the page, do not guess
      │
      ▼
DISCONNECT ──► show a quiet degraded indicator; input stays ENABLED
      │   (sending over HTTP keeps working — NFR-AVL-2)
      ▼
RECONNECT  ──► RE-FETCH, then resume. No replay. No event log.
```

Never derive a message's existence from an event alone. An event says *something changed* and the authoritative read follows. This is what makes the missed-every-event case correct after a reload.

19.6 Unread, read and notification state

State	Where it lives	Why
Unread counts	Server, recomputed on load; not a live feed (§20.2)	Cheap, and never wrong after a refresh
Read marking	Fired on **thread open**, debounced	**FR-READ-4** forbids a write per scroll event
Notification list	Server-owned; the client polls or receives an event	Actionable items only (§29.2)
Draft	**Client-only in V1** — the customer's own browser, never posted on their behalf (**UC-C05**)	Server-side drafts are FUTURE (§42)

19.7 Permission-aware interface

Permissions arrive with the session and shape what is rendered. Two rules keep this honest:

1. **The interface hides what the API would refuse.** A control the principal cannot use is not rendered disabled with an explanation — it is absent. A disabled *Escalate* button tells a user that escalation exists and they are not trusted with it, which is information without a purpose.
2. **The interface is never the check.** Every operation is authorized server-side (§18.4 step 3). A hidden control is a courtesy to the user, not a security measure. If the two ever disagree, the server is right.

19.8 Accessibility, as an architectural requirement

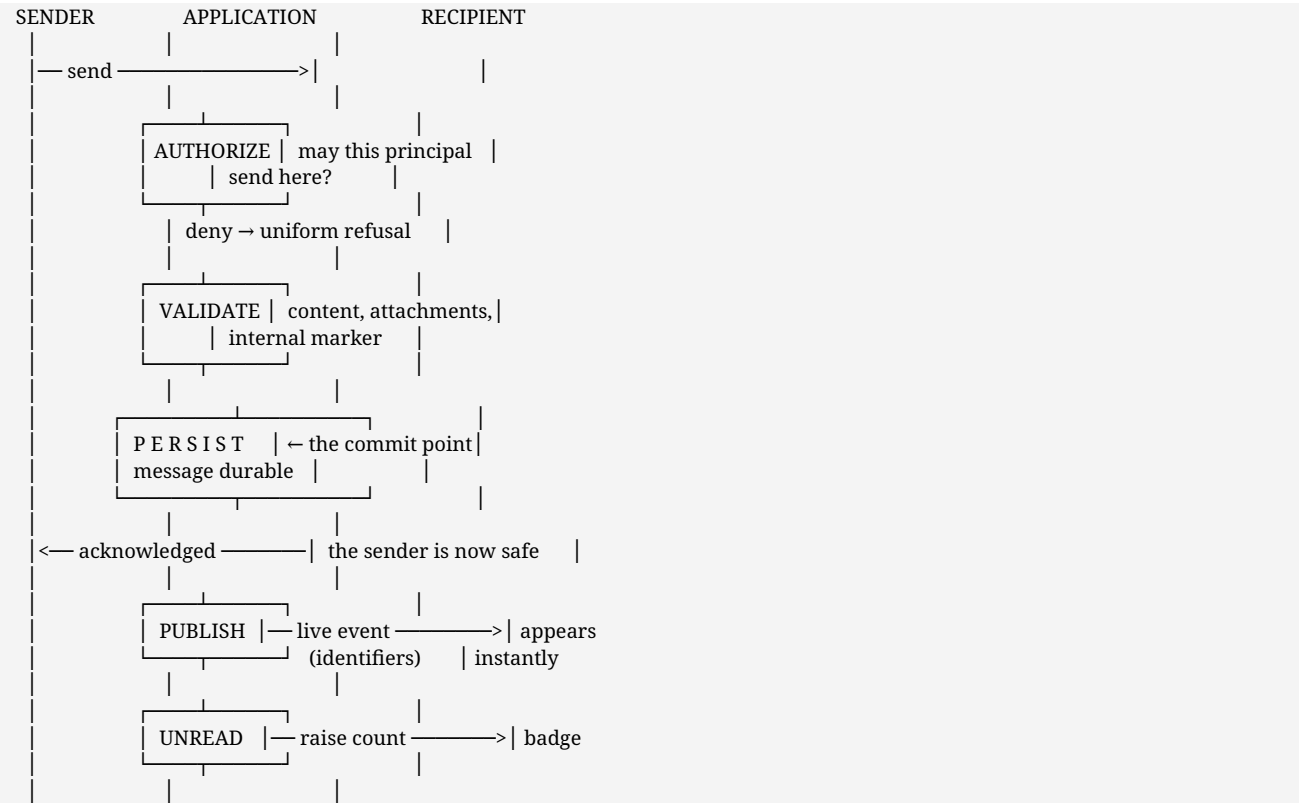
Listed here rather than left to implementation, because one item is a **security** requirement.

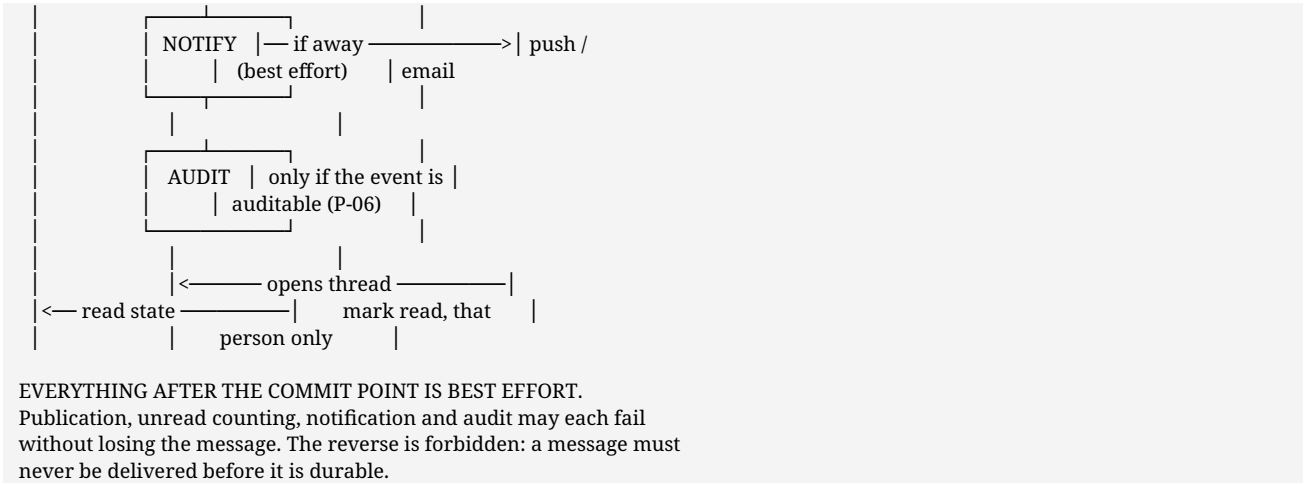
Requirement	Consequence for the architecture
NFR-ACC-3 — internal-note distinction not colour-only	The distinction is composed in one shared component; a text label is part of the *data* presentation, not a style
NFR-ACC-2 — full keyboard operability	Headless primitives (§15.3) rather than hand-built menus and dialogs
NFR-ACC-4 — screen-reader announcement of new messages	The message list is a live region; announcements are throttled so a busy thread does not become unusable
NFR-MOB-2 — customer mobile-first	The customer surface is designed at phone width first, not scaled down from desktop

---

20. Realtime architecture

20.1 Message flow — diagram 8





EVERYTHING AFTER THE COMMIT POINT IS BEST EFFORT.
Publication, unread counting, notification and audit may each fail without losing the message. The reverse is forbidden: a message must never be delivered before it is durable.

20.2 Which events justify realtime, and which do not

P-04 applied honestly. A live channel is a cost in connections, correctness and debugging.

Event	Live?	Reason
New message in an open conversation	**Yes**	Without it, this is not a chat product
Conversation state change (assigned, resolved, reopened)	**Yes**	Someone is waiting on the outcome
New conversation arriving in a queue	**Yes**	A queue that needs refreshing is a queue that grows
Ownership change	**Yes**	Both parties must know immediately
Typing indicator	**Yes**	Cheap, and expected by every user of any chat product
Internal note added	**Yes**, staff only	Staff coordinate in the moment
Unread counts	**No**	Counting, not a live feed. Recomputed on load
Read receipts	**On thread open only**	Not a continuous stream
Directory changes	**No**	Rare; a page load suffices
Search results	**No**	Request and response
Audit events	**No**	Queried, never streamed
Notification preference changes	**No**	Immediate to the actor only
Presence / availability	**Conditional**	**Only** if **D-05** is answered "the system routes around absence automatically". Otherwise it is a feature with no use case (§12.4)

Six live, six not, one contingent on a business decision.

20.3 Realtime is additive

Rule	Consequence
No state exists only in an event	A client that missed every event is correct after a reload
Recovery is by re-reading, not replay	No event log to replay, no ordering guarantees to maintain
Events carry identifiers	A subscriber without established entitlement receives a pointer, not content
Authorized at subscribe **and** invalidated at session end	Server-side, not merely visually

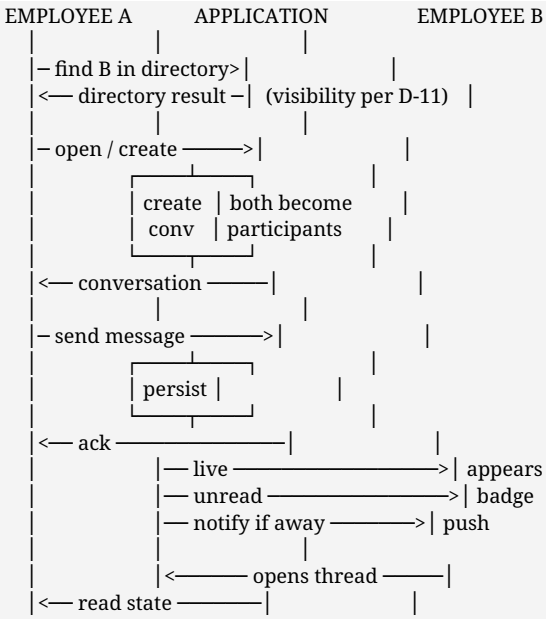
This is what keeps the application usable when realtime degrades (**NFR-AVL-2**): messages still send and threads still read.

20.4 Notification discipline

Notify only what someone must act on. Everything else is an unread count.

Notified	Not notified
A customer conversation assigned to you	Every message in a busy internal group
A conversation waiting beyond a service standard	Someone typing
Escalation to your function	A directory change
Transfer into or out of your ownership	Read receipts
Cover needed on your team	Your own actions
A customer replied to a thread you own	Internal notes — **never** to the customer
Your role or access changed	—

20.5 Employee-to-employee flow — diagram 3



NOT AUDITED — an ordinary internal message (P-06).
Participant changes ARE audited, because they change who may read prior history.

20.6 Transport mechanism — PROPOSED

Aspect	Proposal
Transport	Socket.IO over WebSocket (**ADR-05**)
Connection scope	One connection per authenticated browser tab
Grouping	**Rooms, one per conversation** — the room *is* the authorization unit
Room join	Authorized at join time against §26.3, exactly as an HTTP read would be
Room leave	On unsubscribe, on session end, and on participant removal
Payload	**Identifiers and minimal metadata. Never message bodies** to a subscriber whose entitlement for that content is not established (**FR-RT-4**)
Multi-instance	**Not in V1** (single process). Requires a shared adapter channel —

Aspect	Proposal
	the Redis trigger in §33.2

20.7 Every event, and its mechanism

The table the architecture actually needs. **Publisher** is the module that emits it; **transport required** answers whether the product breaks without a live channel.

Event	Publisher	Receives	Transport required?	Fallback if disconnected	Authorization	Ordering	Duplicates	Idempotent?
Message created	Conversation	Conversation room	**No** — the message is durable and readable over HTTP	Re-fetch the page on reconnect	Room join	Server sequence per conversation; client applies in order or **re-fetches on a gap**	Client discards a known id	Yes — id is the key
Internal note created	Conversation	Room, **staff participants only**	No	Re-fetch	Room join **plus a staff-kind check on publish** — never fanned to a customer connection	As above	As above	Yes
Message delivery state	Conversation	Sender	No	Recomputed on read	Sender only	Latest wins	Harmless	Yes
Read receipt	Conversation	Room, **internal threads only**	No	Recomputed on thread open	Room join	Latest wins	Harmless	Yes
Typing	Conversation	Room, others only	No	Simply absent — no consequence	Room join	**None — ephemeral, expires on a timer**	Harmless	N/A, stored nowhere
Conversation assigned	Routing	New owner, previous owner, team room	No	Visible in the owner's list on load	Room join + team scope	Ownership record is authoritative; the event is a hint	Discard on known version	Yes
Ownership transferred	Routing	Both parties, team room	No	As above	As above	As above	As above	Yes
Conversation state changed	Conversation	Room	No	Read on load	Room join	State version on the conversation	Discard older version	Yes
Participant added / removed	Conversation	Room, and the affected principal	No	Read on load	Room join. **On removal, the connection is forced out of the room**	Latest wins	Harmless	Yes
Queue	Routing	Team room	No	Queue read	**Team scope**	Unordered	Discard	Yes

Event	Publisher	Receives	Transport required?	Fallback if disconnected	Authorization	Ordering	Duplicates	Idempotent?
arrival				on load	**at join**, not conversation participation	— it is a set, not a sequence	known id	
Notification created	Notify	The recipient's personal room	No	Notification list on load	Personal room, self only	Unordered	Discard known id	Yes
Presence / availability	—	—	**Conditional**	—	—	—	—	—

Presence remains conditional on a business decision. It is built **only** if **D-05** is answered "the system routes around absence automatically". If the answer is "a named backup" or "the lead reassigns", a human decides and no presence tracking exists (§20.2). It is not a feature in its own right.

Every row answers "no" to transport required. That is the design working: **NFR-AVL-2** holds because no event carries state that HTTP cannot also produce.

20.8 Ordering, gaps and duplicates

```
PER-CONVERSATION SEQUENCE
Each event on a conversation carries a monotonic sequence for that
conversation. Global ordering is NOT attempted and is not needed —
nothing in the product compares two conversations' timelines.

CLIENT RULE
┌──────────────────────────────────────────────────────────────────┐
│ event.seq == last_seen + 1  ──► apply                            │
│ event.seq <= last_seen      ──► DISCARD (duplicate/replay)       │
│ event.seq >  last_seen + 1  ──► GAP: re-fetch the page.          │
│                          Never interpolate.                      │
└──────────────────────────────────────────────────────────────────┘

WHY NO REPLAY BUFFER
Re-fetching is O(one page) and always correct. A replay buffer
would need retention, ordering guarantees and its own
authorization check on every buffered event — three new ways to
be wrong, to save one HTTP request.
```

20.9 Reconnection

```
DISCONNECT
  │
  ├─ client: quiet degraded indicator. INPUT STAYS ENABLED.
  │        Sending continues over HTTP (NFR-AVL-2).
  │
  ├─ server: connection released. NOTHING is buffered for it.
  │
  ▼
RECONNECT (backoff with jitter, so a deploy does not produce a
        synchronised reconnect storm — monitored in §32.3)
  │
  ├─ [1] RE-VALIDATE THE SESSION.
  │      A session revoked while disconnected must NOT be
  │      honoured on reconnect. This is the hole that a
  │      "resume the old connection" design would open.
  │
  ├─ [2] RE-AUTHORIZE EVERY ROOM JOIN.
  │      Participation may have been removed while away.
  │
```

```
├─ [3] RE-FETCH state for open views.
│
└─ [4] resume live delivery.
```

20.10 Realtime authorization — the hole this closes

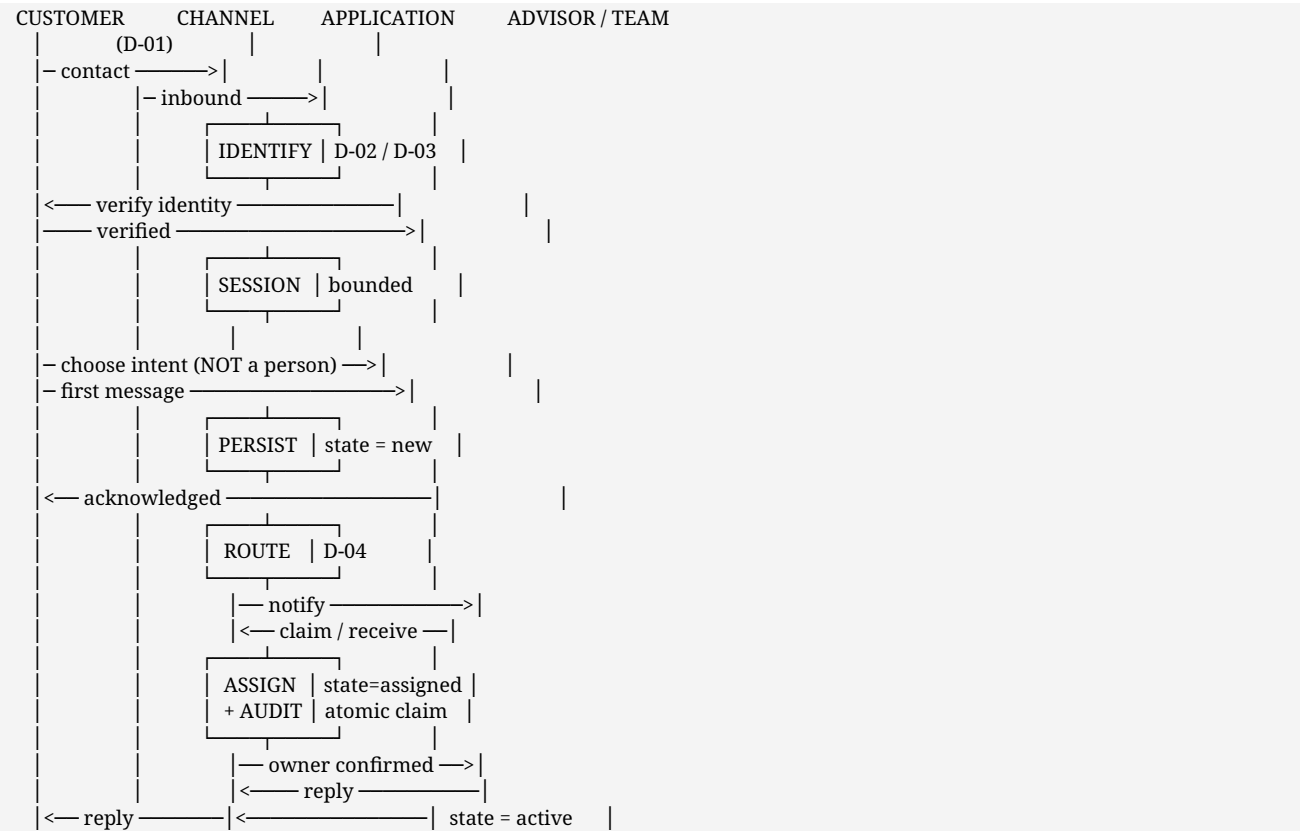
Rule	The failure it prevents
Authorize at **join**, with the same decision as an HTTP read	A room subscription becoming a back door around §18.4 step 3
Invalidate server-side at session end	A socket outliving its session — an authorization hole, not an inconvenience
Re-authorize on **every reconnect**	A revoked session resuming, or a removed participant continuing to receive
Force removal from the room when participation ends	Continued delivery to someone whose access was revoked
Publish-time kind check on internal notes	An internal note reaching a customer connection — **the worst leak in the product**
Identifiers, not bodies (**FR-RT-4**)	A misrouted event disclosing content

§38 records that the reference platform's realtime layer did not close its channels on session end. That is why this is stated as six explicit rules rather than left to implementation.

---

21. Customer journey, intake and routing

21.1 Customer to employee — diagram 4



21.2 Routing and assignment — why hybrid

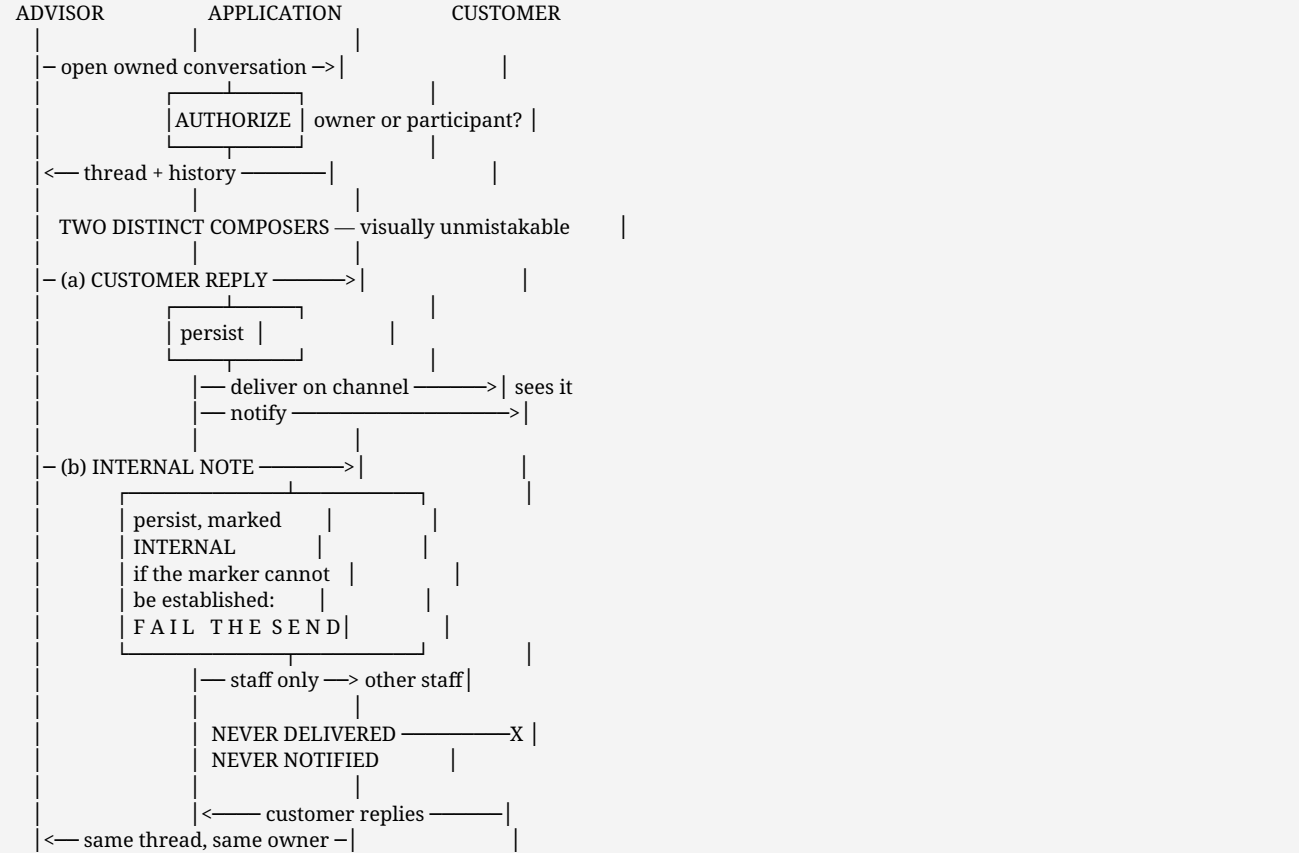
> ***The decision diagram that stood here is superseded by §21.8**, which adds the two steps version 2.0 lacked: the business-calendar check (§23.3) and category-based team selection (§21.6). The department-shape analysis below is unchanged and is what §21.6 and §21.8 both rest on.*

Why hybrid is proposed (D-04). The departments are different shapes, and one model would be wrong for most of them:

Department	Shape	Needs
Fresh Sales (63)	Queue	Speed to first response; no prior relationship exists
Retention, Renewals (~40)	Relationship	The advisor the customer already knows
Support (18)	Queue with continuity	Fast pickup, same person if the issue continues
Claims (4), Grievance (1)	Case	The right function, with complete history

Options considered: **direct to a named person** — rejected, the customer would be managing our org chart and absence becomes their problem; **team-assigned** — workable, and the fallback if hybrid is rejected; **pure queue** — rejected for Renewals, it discards the relationship that is the department's entire value; **hybrid** — proposed.

21.3 Employee to customer — diagram 5



21.4 The two lifecycles, made explicit — diagram 9 (revised)

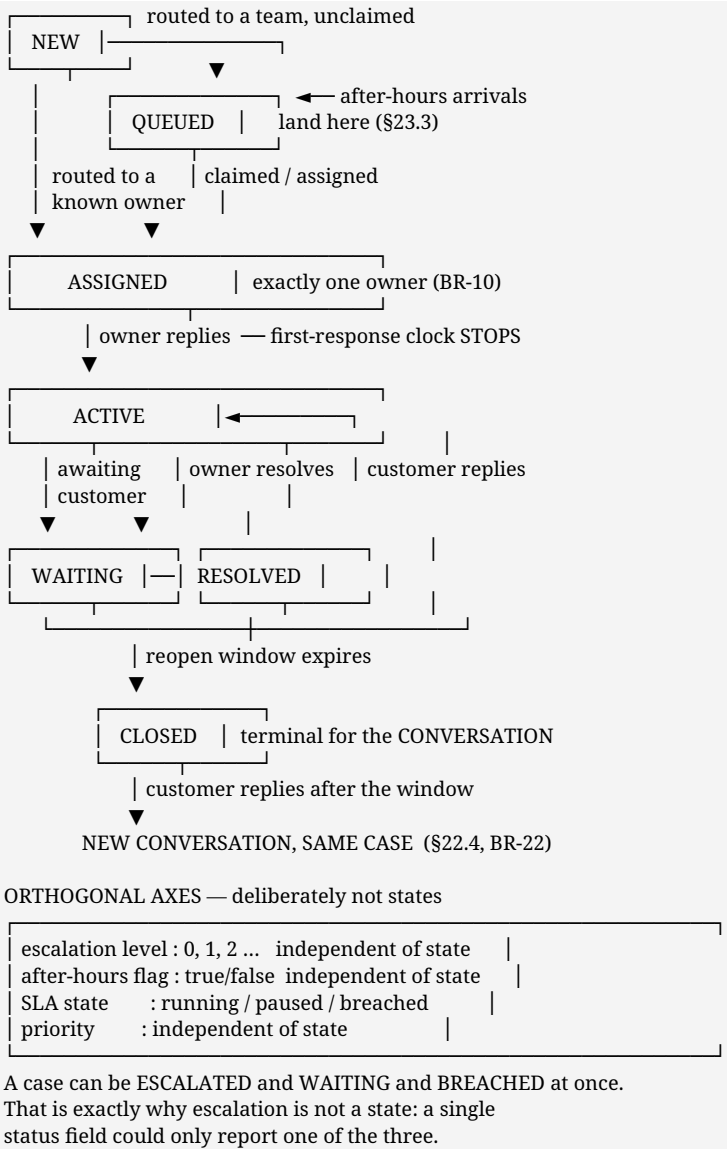
> ***Supersedes version 2.0's §21.4.** That version defined six customer states and then listed `any → reassigned` and `any → escalated` in its transitions table — neither of which is a state. **What changed:** `queued` is added; transfer and escalation move to an events table; escalation additionally becomes a level on the case. Nothing was removed except the two mislabelled rows.*

> ***Conflict resolved — K-02.*** *Version 2.0 defined six customer states and then listed* ***any → reassigned*** *and* ***any → escalated*** *in the transitions table — but neither is a state.* ***queued*** ***is added*** *(routed to a team, nobody has claimed it), which version 2.0 lacked and after-hours requires.* ***Transfer and escalation become events****, and escalation additionally becomes a* ***level*** *on the case.*

Internal employee chat — no business lifecycle

Property	Value
States	**None.** A thread exists and stays open indefinitely
Resolve / close	**Does not exist** (**D-15**)
Archive / mute	**Personal view state**, belonging to one person, never a property of the conversation
SLA	**None.** Colleagues are not measured on reply time (**P-08**)
Service case	**None.** Internal chat has no case

Customer service conversation — a real lifecycle



ORTHOGONAL AXES — deliberately not states

```
escalation level : 0, 1, 2 ...   independent of state
after-hours flag : true/false  independent of state
SLA state        : running / paused / breached
priority         : independent of state
```

A case can be ESCALATED and WAITING and BREACHED at once.
That is exactly why escalation is not a state: a single
status field could only report one of the three.

State transitions in detail

Carried from version 2.0 and extended for queued. The *customer notified* column is the one that matters most: it is the difference between keeping someone informed and exposing internal handling.

From → To	Actor	Reason required	Customer notified	Audited
— → new	Customer	—	Acknowledged	Yes
new → queued	System, when no owner can be assigned, or **after hours** (§23.3)	—	**Acknowledged, with no response-time promise**	**Yes**
new → assigned	System or advisor	—	Optional	**Yes**
queued → assigned	Advisor claims, or a lead assigns	—	Optional	**Yes**
assigned → active	Owner	—	Yes (the reply itself)	No
active → waiting	System, awaiting the customer	—	No	No
waiting → active	Customer replies	—	No	No
active → resolved	Owner or lead	**Yes — the outcome**	**Yes, with the outcome**	**Yes**
resolved → active	Owner or customer (reopen in window)	Yes, if staff-initiated	Yes	**Yes**
resolved → closed	System, on reopen-window expiry	—	No	Yes
closed → *(new conversation)*	Customer replies after the window	—	No — it simply continues	**Yes** (**BR-22**, same case §22.4)

Nothing in this table notifies a customer of internal handling. No transition tells them about SLA state, escalation, priority or who was tried first (§27.16).

Events, not states

Event	Effect	Reason required	Audited
Claim	queued → assigned; sets current owner	No	**Yes**
Assign	Sets current owner	No	**Yes**
Transfer	Changes current owner. **State does not change** — the case is still being worked	**Yes**	**Yes**
Cover	A colleague answers; **current owner does not change** (§21.9 A/D)	No	**Yes**
Escalate	**Raises escalation level**; may change owning team. State unchanged	**Yes**	**Yes**
Reassign on departure	Changes current owner **and** designated employee (**BR-13**)	**Yes**	**Yes, must-succeed**
Reprioritise	Changes priority. State unchanged	**Yes**	**Yes**
SLA breach	Sets a flag with a timestamp; may raise escalation (**D-25**)	—	**Yes**

Why this is better than adding states. "Work in progress" is assigned or active — two values, and a transfer does not add a third. If transferred were a state, every query for open work would have to remember to include it, and the first one that forgot would hide a customer.

21.5 The customer intake journey — diagram 18

The primary customer-facing V1 journey, PROPOSED. This is the path the overwhelming majority of customer conversations will take, so it is stated end to end before anything is decomposed.

```
CUSTOMER visits the CoverYou website
   │
   ▼
Opens StarLink chat  (embedded widget on the CoverYou site)
   │
   ▼
"How can we help you?"
   │
   ▼
Selects a CATEGORY                  ── D-17 (the list)
   │                                D-18 (the structure)
   ▼
Optional SUB-CATEGORY / intent
   │
   ▼
IDENTITY ESTABLISHED                ── D-02 (method)
   │                                D-03 (source)
   ▼
ROUTING ENGINE                      ── §21.7
   │  designated employee? available?
   │  team open?  (§23.3)
   ▼
CONVERSATION ASSIGNED  ──or──  QUEUED
   │
   ▼
EMPLOYEE ⇄ CUSTOMER in realtime     ── §20
   │
   ├──► TRANSFERRED   (event, not a state — §21.4)
   ├──► ESCALATED     (level, not a state — §21.4)
   │
   ▼
RESOLVED  (with an outcome the customer is told)
   │
   ▼
CLOSED    (on reopen-window expiry)
   │
   ▼
REOPENED within the window  ──►  same conversation, same case
after the window            ──►  NEW conversation, SAME case (§22.4)
```

Two properties of this journey worth stating plainly. The customer is asked for **one thing** — what they need — and never for a form, a reference number or a person's name. And **identity is established after the category**, so a customer who abandons at the category step has disclosed nothing.

21.6 Category and sub-category model

> ***Conflict resolved — K-03.*** *Version 2.0 used the word *intent* informally ("choose an intent or topic", "team queue by intent").* ***Category and sub-category are the concrete realisation of that intent.*** *The terminology is unified here; no earlier decision is overturned.*

Categories are data, not code. They are seeded and administrable, because the business will change them and a deployment should not be required to do so. The category tree, its depth and whether sub-categories are mandatory are **D-17** and **D-18**.

Illustrative categories — PROPOSED, not approved

The brief offers these as examples and explicitly states they are not permanently approved. They are recorded here as a **starting point for D-17**, mapped to the teams the directory actually contains.

Category *(PROPOSED)*	Plausible sub-categories *(PROPOSED)*	Candidate team	Existing evidence
Sales	New policy · quotation · product question	Fresh Sales (63)	§7.2 — queue-shaped, no prior relationship
Renewals	Renewal due · premium change · lapse	Retention (21), Renewals (19)	§7.2 — relationship-shaped
Existing policy / service	Update details · documents · endorsement	Support (18)	§7.2 — queue with continuity
Claims	New claim · claim status · documents	Claims (4)	§7.2 — case-shaped
Grievance	Complaint · escalation	Grievance (1)	§7.2 — **one person; a real single point of failure**
Other / not sure	—	Triage, then reroute	Prevents a wrong-category dead end

"Other / not sure" is architecturally important, not a filler row. Without it a customer who cannot classify their own problem either abandons or picks wrongly, and a wrong category means wrong routing, wrong team and a wrong clock.

What the category determines

Determines	How
Candidate team	Category → team mapping, held as data (**D-17**)
Which business calendar applies	The team's calendar (§23.1)
Which SLA targets apply	Per category or per team (**D-22**, **D-23**)
Default priority	Category → priority default (**D-24**)
Whether a designated employee is consulted	Relationship-shaped categories only (§21.7)

A category is never a promise about who replies. It selects a team and a clock.

21.7 Designated employee versus current owner

The distinction the routing model turns on, and version 2.0 implied it without naming it.

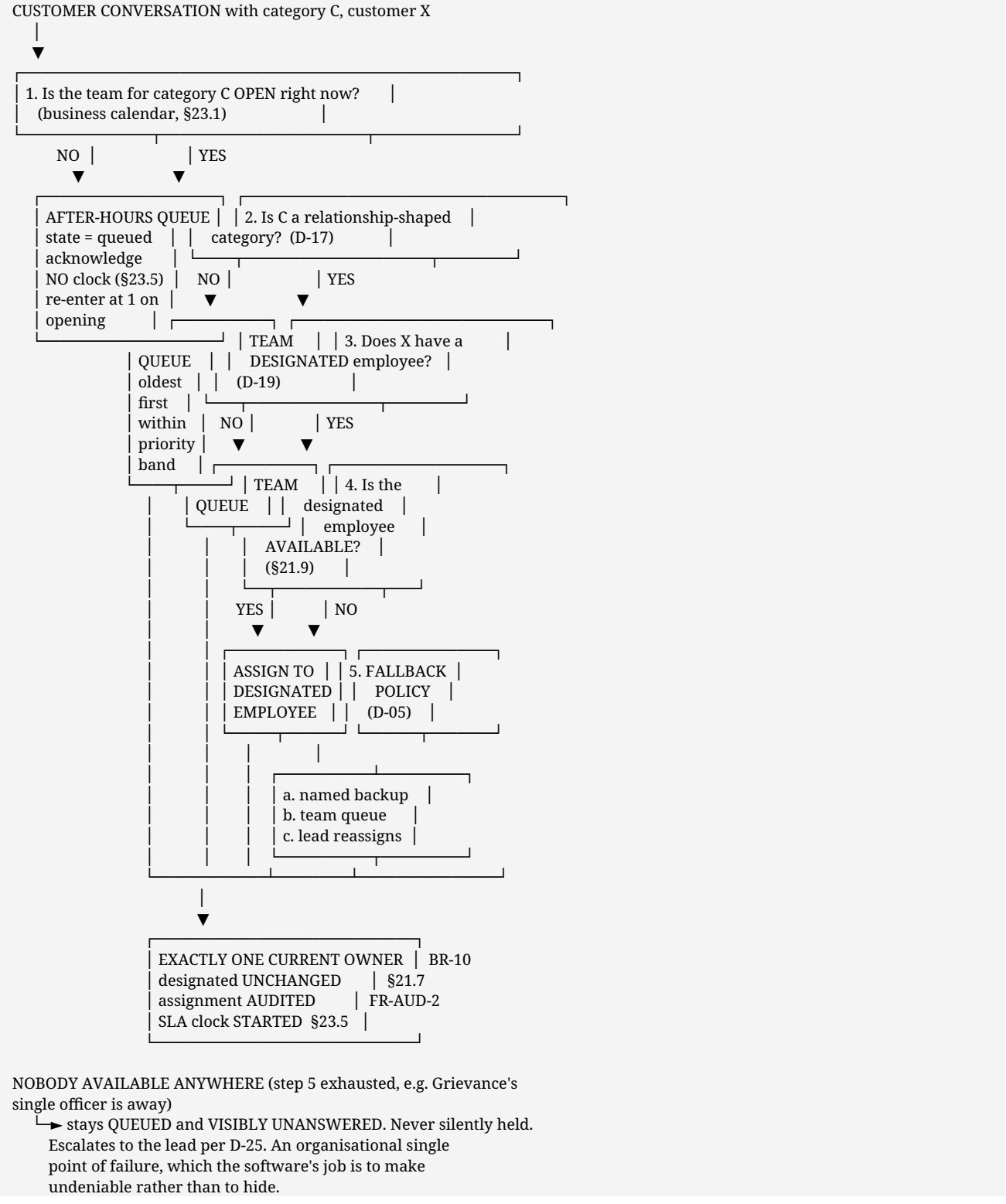
	Designated employee	Current owner
Meaning	The customer's relationship advisor — *who should normally handle them*	Who is accountable **right now**
Lives on	The **case** (§22.3), and derived from customer history	The **case**, with append-only assignment history
Changes	Rarely — a book transfer, a departure, a business decision	Often — claim, transfer, cover, escalation
Source	**D-19** — CRM, policy system, prior conversation history, or manual assignment	The routing engine and staff action
Exists for every customer?	**No.** A new prospect has none	**Yes**, once assigned
Customer sees it	**Never** (§22.5)	Only as "who you are speaking to"

```
THE TWO ARE INDEPENDENT — and that is the point.

Designated = A        Current owner = A     normal
Designated = A        Current owner = B     A is away; B is covering
Designated = A        Current owner = A     A is back; B's cover is history
Designated = none     Current owner = C     prospect, C claimed from queue
Designated = A (left) Current owner = D     reassigned on departure (BR-13)

COVER DOES NOT DESTROY THE RELATIONSHIP.
A conversation covered by B while A is on leave still has
designated = A, so the next contact returns to A.
```

> ***Conflict resolved — K-04.*** *This appears to contradict* ***BR-02*** *("a customer may not start a conversation with a named individual"). It does not.* ***The customer chooses a category; the system chooses the person.*** *A customer cannot request an individual, and cannot see who their designated employee is. BR-02 stands unchanged.*

21.8 The routing decision tree — diagram 19

No random selection at any point. Every branch is deterministic and inspectable.



Step 2 is where §21.2's department-shape analysis is applied. Fresh Sales is queue-shaped, so a prospect goes to the queue and the fastest advisor takes it. Renewals is relationship-shaped, so the customer's own advisor is tried first. **Which categories are relationship-shaped is part of D-17.**

21.9 Employee availability and unavailability

> ***Preserved from version 2.0, and load-bearing:*** *a dropped realtime connection* ***must never*** *mean an employee is unavailable.* ***Ownership and presence are separate concepts*** *(§34.5). A network hiccup must not reassign anybody's work.*

What "available" means

Signal	Counts toward availability?	Why
Team's calendar is open (§23.1)	**Yes**	Nobody is available outside their working window
Account is active	**Yes**	A deactivated account cannot own work (**BR-13**)
Declared absence — leave, off shift	**Yes**	An explicit, human-entered fact
Explicit "unavailable" status set by the employee or a lead	**Yes** (**D-05**)	A deliberate declaration
At or over a workload ceiling	**D-05**	Optional; needs a business position
Realtime connection present	**NO — never**	A phone entering a lift is not leave
Recent activity	**NO**	Being quiet is not being absent

Availability is declared or derived from the calendar. It is never inferred from a socket.

The five cases

	Case	Current owner	Designated employee	Fallback	Customer told?	Audited
A	**Short absence** — a day, a shift, a meeting	**Unchanged** — the owner keeps it	Unchanged	Surfaced to the team as *needing cover*; a colleague may answer under team scope	**No** — they simply get an answer	**Yes** — cover is a read they would not otherwise have
B	**Extended leave** — beyond a threshold (**D-05**)	**May change** to a backup or the queue	**Unchanged**	Per **D-05**: named backup · auto-return to queue · lead reassigns	**No**	**Yes**
C	**Departure / deactivation**	**MUST change**	**Must be reassigned** (**D-19**)	Lead or admin reassigns. **Every owned case is surfaced at deactivation** (**FR-EMP-3**)	**No** — but the new owner introduces themselves	**Yes, must-succeed**
D	**Unavailable during working hours** — in a meeting, on a call, at a ceiling	**Unchanged**	Unchanged	Same as A. **The team covers; ownership does not move**	**No**	**Yes**
E	**No employee available for the team at all**	None yet	Unchanged	Stays **queued and visibly unanswered**. Lead alerted per **D-25**	**Acknowledged**, with no response-time promise (§23.2)	**Yes** — the queued state and its age

Case C is the one that silently loses customers. A case owned by a deactivated principal is unreachable work, so §32.3 monitors *cases owned by an inactive principal* with a target of **zero** and §32.4 alerts on any non-zero value.

Case D is the one most often got wrong. The temptation is to treat "in a meeting" as unavailability and move ownership. Moving ownership for a two-hour meeting churns the relationship for no benefit — the team covers, the owner returns, and the cover is history.

How the customer is informed

Situation	What the customer sees
Cover by a colleague (A, D)	A reply from a named person. **The internal mechanics are not exposed** (**FR-TRAN-5**)
Reassignment (B, C)	A reply from a named person, who may introduce themselves. **No "your advisor has left"** unless the business chooses to (**D-20**)
Escalation	Where appropriate, that a specialist is now helping. **Never the escalation level or reason** (§22.5)
Nobody available (E)	An acknowledgement that the message is received and queued (§23.2)

The customer is never shown internal routing information — not the queue, not the fallback path, not who was tried first (§11.7).

---

22. Service case metadata — conversation-centric, not ticketing

PROPOSED. This section answers a question a CTO will certainly ask: *does StarLink need a ticketing system?* The answer proposed here is **no — and also not pure chat either.**

22.1 The question, stated fairly

Customer service platforms cluster around two shapes, and both are defensible.

	Option A — traditional ticketing	Option B — pure chat
Customer experiences	A ticket with a reference number, a form, a status page	A conversation
Employee experiences	A queue of tickets with fields and status	A list of threads
Strength	Operational discipline. Nothing is lost, everything is measurable	Immediacy. Low friction. Feels like talking to a person
Weakness	**Feels like filing a complaint, not getting help.** Slow, formal, and the customer chases a reference	**No operational visibility.** Nobody can say what is waiting, who owns it, or whether it is late
Fit for CoverYou	Wrong for Fresh Sales (63 people) — a prospect will not file a ticket to ask about a premium	Wrong for Claims and Grievance — a complaint with no owner and no clock is a compliance exposure

Neither shape fits, because the two ends of this business need different things. Sales needs the immediacy of chat; Claims and Grievance need the accountability of ticketing.

22.2 Recommendation — Option C

> ***PROPOSED: a conversation-centric model with service metadata attached.***
>
> ***The customer experiences a conversation.*** *No form, no reference number to quote, no status page to refresh. They open the website chat, choose what they need, and talk to a person.*

> ***Employees and management get the operational metadata*** *— category, priority, owner, assignment history, SLA state, escalation history — carried alongside the conversation and* ***invisible to the customer****.*

This is not a compromise between A and B. It is the observation that **the operational discipline of ticketing is valuable to us and worthless to the customer**, so it should exist without being imposed on them.

This is explicitly not a standalone ticketing product. There is no ticket form, no separate ticket inbox, no customer-facing ticket portal, and no second application to log into. §22.7 states what would have to change before that were true.

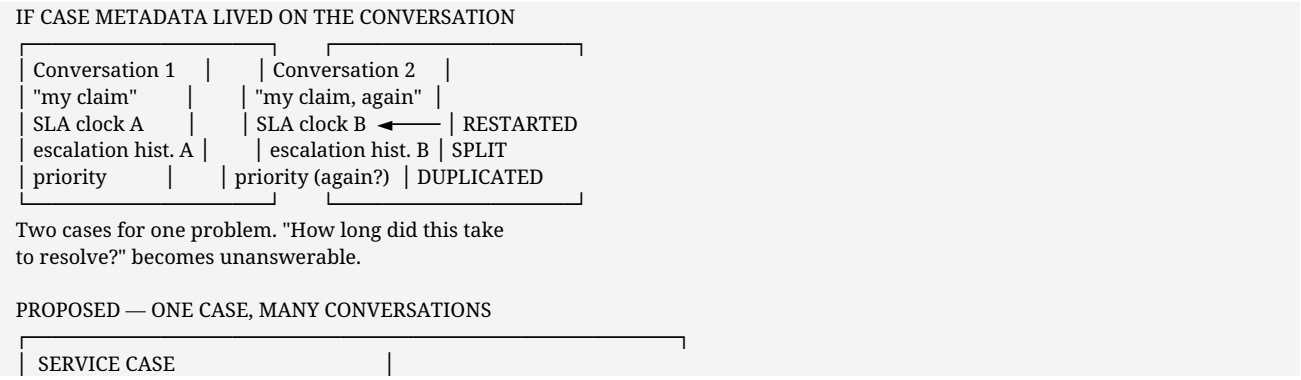
22.3 The Service Case

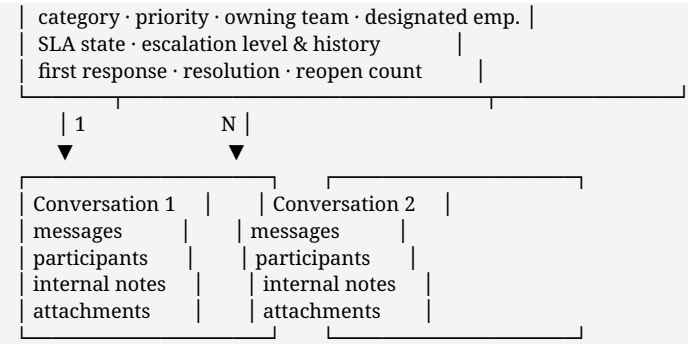
Naming is a decision (D-28). This document uses **Service Case**, or *case*. The brief offered "Customer Service Work Item"; v2.0 established no term, so nothing is being overridden.

Attribute	Purpose	Visible to customer?
Case reference	Internal identifier. **Whether a customer-quotable number exists is D-27**	**D-27**
Customer	The principal the case belongs to	Implicitly
Category / sub-category	What the customer said they needed (§21.5)	Yes — they chose it
Priority	Operational urgency. **Definitions are D-24**	**No**
Owning team	The team accountable	No
Designated employee	The customer's relationship advisor, if any (§21.6)	No
Current owner	Who is accountable *right now* (§21.6)	Only as "who you are speaking to"
Assignment history	Append-only record of every owner and why it changed	**No**
Internal state	new · queued · assigned · active · waiting · resolved · closed (§21.9)	**No — mapped to a customer vocabulary (D-26)**
Escalation level	0 = none. A separate axis, not a state (§21.9)	**No**
Escalation history	Who escalated, when, why	**No**
SLA state	Clocks and breach flags (§23.5)	**No**
First-response timestamp	When a human first replied	No
Resolution timestamp	When it was resolved, and the outcome	Outcome only
Reopen count	How often it came back	No
Linked conversations	**One or more** (§22.4)	Their own only

22.4 Why the case is separate from the conversation — diagram 15

This follows from a rule v2.0 already established. BR-22 says that after the reopen window expires, a customer reply creates a **new conversation** with prior history linked. So one customer issue can already span two conversations.





ONE clock. ONE escalation history. ONE resolution time.
A reopen JOINS the case rather than starting a second one.

Consequence	Why it matters
The SLA clock is not restarted or duplicated by a reopen	§23.5 — otherwise resolution time is meaningless
Escalation history is continuous across reopens	A repeatedly-reopened complaint is exactly what Grievance needs to see
"How many times did this come back?" is answerable	A reopen count on the case; impossible if each reopen is a fresh record
D-27 becomes a display question, not a structural one	The case already has an identifier; whether the customer is shown one is a product choice
Growth to case management needs no migration	§22.7

The cost, stated honestly. The employee view needs the case *and* the conversation — one extra read. §24.9 lists the case fields denormalised onto the conversation so the queue view still needs no join, and that list grows from five entries to nine.

22.5 Who sees what

	Customer	Owner	Team member	Team lead	Compliance
Conversation messages	Own	✅	Participation only	**D-11**	Via audit
Internal notes	**Never**	✅	If participant	**D-11**	Via audit
Category / sub-category	✅ (chose it)	✅	✅	✅	✅
Priority	**Never**	✅	✅	✅	✅
Internal state	**Never** — mapped (**D-26**)	✅	✅	✅	✅
SLA state and breach	**Never**	✅	✅	✅	✅
Escalation level and history	**Never**	✅	✅	✅	✅
Assignment history	**Never**	✅	Own entries	✅	✅
Designated employee	**Never**	✅	✅	✅	✅
Audit records	**Never**	**No**	**No**	**No**	✅ only

The row that carries the most risk is SLA state. A customer who can see "first response breached" has been handed a grievance the company generated itself. §27.16 makes this a fail-closed serialisation rule, not a UI convention.

22.6 What this is *not*

Not	Why
A ticket form	The customer picks a category and types. Nothing else is asked of them
A customer ticket portal	They have their conversation, and their own history (**D-08**)
A second application	One employee surface, one customer surface (§19)
A workflow engine	No configurable multi-step approval chains
An asset or knowledge base	Out of scope (§12.5)
A replacement for policy administration	Out of scope (§6.2)

22.7 How it could grow into case management — FUTURE

Recorded so the CTO can see the path exists without it being built now.

Capability	What it would need	Why the current shape allows it
Cases not started by a conversation (a phone call logged by an advisor)	A case created without a conversation	The case is already the parent; conversations are already optional children
Sub-cases or linked cases	A case-to-case relationship	Additive; no conversation change
Structured resolution codes	A vocabulary on the resolution	The resolution field already exists
Multi-step workflow per category	A workflow definition per category	Category is already an entity (§21.5)
Customer-visible case portal	A customer-facing status view	**D-26** already defines the safe vocabulary
Cross-case reporting and trend analytics	A reporting surface	**FUTURE** (§42) — SLA *state* is V1, SLA *analytics* is not

Nothing above is proposed for V1. The point is that each is an addition rather than a migration — because the case exists as its own entity from the first record.

---

23. Working hours, after-hours and SLA

PROPOSED. Version 2.0 mentioned working hours exactly once, about planned maintenance, and had no SLA model at all. This section supplies both, because a customer-facing product that cannot say *when someone will reply* has no operating discipline.

> ***Conflict resolved — K-01.*** *§12.4 and §42 list *"analytics beyond queue, load and SLA state"* as future.* ***SLA state tracking and escalation are V1****, because they drive routing and notification, which are already V1.* ***SLA reporting, trend analytics and dashboards remain FUTURE.*** *Those two lines were amended rather than left to contradict this section.*

23.1 The business calendar

A new configuration entity. **Every value in it is a stakeholder decision (D-21).**

Attribute	Purpose
Scope	Per **team**, not company-wide — Claims and Fresh Sales need not share hours
Working days	Which days the team operates
Working window	Open and close time per working day
Holidays	Non-working dates
Timezone	One timezone per calendar. Explicit, never inferred from a server

Attribute	Purpose
	clock
Exceptions	One-off closures or extended hours

Why per team. §21.7 routes by category to a team, so "are we open" is a question about the **team that would receive this**, not about the company. A customer contacting Claims at 20:00 may be after hours even if Support is still open.

No default hours are proposed. Inventing "10:00–19:00, Monday to Saturday" would be inventing a business fact. The architecture reads whatever the calendar says.

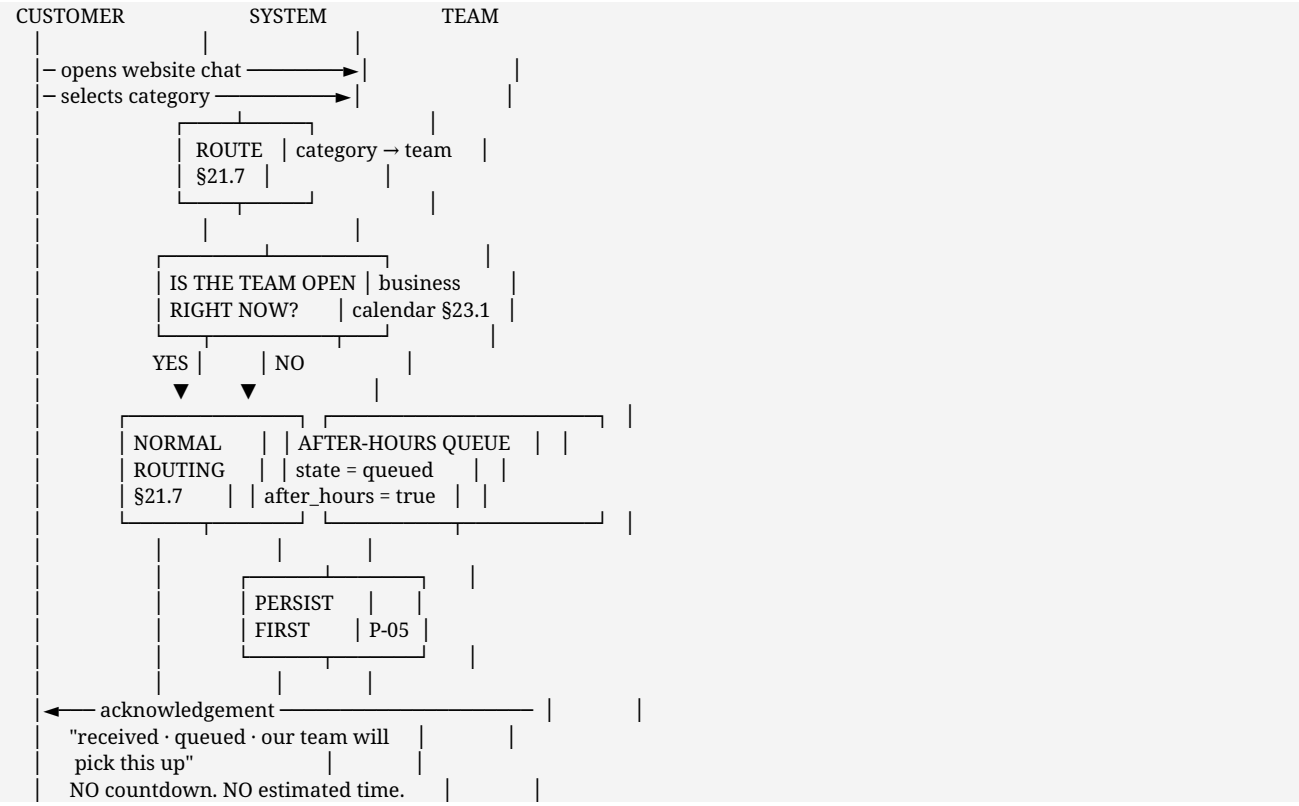
23.2 Three different promises — and they must not be confused

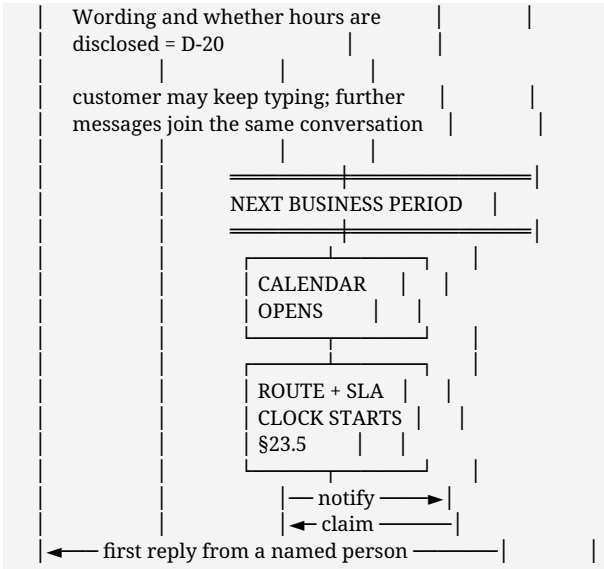
Model	What it promises	What it requires	Honest cost
24×7 SLA	A human responds within the target **at any hour**	Staffing outside business hours — night shifts or an outsourced desk	The expensive one. **A 24×7 target without 24×7 staffing is a promise that breaches every night**
Business-hours SLA	A human responds within the target **of elapsed working time**	Staffing during declared hours; a calendar to compute against	The realistic one for most teams
After-hours acknowledgement	The message is **received, stored and queued** — with no response-time promise	Nothing but honesty	Free, and the minimum acceptable behaviour

Which model applies to which team is D-23. The target values are D-22. The architecture supports all three; it invents none of them.

> ***The rule that matters more than the model.*** *After hours, the customer is told their message is **received and queued**. They are **never** shown a countdown, an estimated response time, or anything implying a human is reading it. A false promise at 23:30 is worse than no promise.*

23.3 After-hours flow — diagram 16





INVARIANT — nothing here is lost, and nothing is silently held.
An after-hours conversation is a QUEUED conversation with a flag.
It appears in the queue view (FR-ROUTE-4) and is counted by the
unassigned metric (§32.3) exactly like any other waiting work.

23.4 Ordering when the queue opens

A real operational question: overnight arrivals all become routable at once.

Rule	Reason
Oldest arrival first **within a priority band**	The customer who waited longest is served first, but a P1 claim does not queue behind routine enquiries
After-hours arrivals do **not** jump ahead of business-hours work already waiting	Otherwise the backlog inverts every morning
A designated employee's customers still route to them (§21.6)	The relationship survives the overnight gap
The opening surge is **visible** in the queue view before anyone claims	Leads can redistribute rather than discover it

Priority bands are D-24. Without them, "oldest first" is the whole rule.

23.5 The SLA model

Three clocks, each with a distinct question.

Clock	Starts	Stops	Question it answers
First response	The customer's first message becomes routable — **immediately if open, at opening if after hours**	A **human** replies on the customer-visible channel	*Did anyone answer this person?*
Resolution	Same as first response	The case reaches resolved	*Did we finish?*
Escalation	On escalation	The receiving function makes first contact	*Did the specialist pick it up?*

An internal note does not stop the first-response clock. Only a message the customer can actually see counts, or the metric measures internal activity rather than service.

Clock behaviour — diagram 17

BUSINESS-HOURS SLA (D-23 per team) — the clock PAUSES when closed

```
═══════════════════════════════════════════════════════════════

Team open 10:00 — 19:00      Example: first-response target 30 min

Case A — customer messages 18:55
┌──────────────────────────────────────────────────────────────┐
│ 18:55 message · clock starts, 5 min elapse before close      │
│ 19:00 CLOSE  · clock PAUSES with 25 min remaining            │
│ ~~~ overnight: NO elapsed time counted ~~~                   │
│ 10:00 OPEN   · clock RESUMES with 25 min remaining           │
│ 10:20 reply  · total elapsed 25 min · MET                    │
└──────────────────────────────────────────────────────────────┘
The 15 overnight hours are not counted, because nobody was
rostered. Counting them would breach every evening enquiry.

Case B — customer messages 23:30
┌──────────────────────────────────────────────────────────────┐
│ 23:30 message · PERSISTED · acknowledged · queued            │
│                CLOCK NOT STARTED                             │
│ 10:00 OPEN   · routed · CLOCK STARTS NOW                     │
│ 10:25 reply  · elapsed 25 min · MET                          │
└──────────────────────────────────────────────────────────────┘
The customer is NOT told "25 minutes" at 23:30. They are told
the message is received and queued.

24x7 SLA (only where the business staffs it) — clock never pauses
═══════════════════════════════════════════════════════════════
23:30 message · clock starts · 30 min target · 00:00 BREACHED
Correct only if someone is actually rostered at midnight.
```

Rule	Statement
The clock is computed, never stored as a countdown	Elapsed working time is derived from start, stops and the calendar — so a calendar correction fixes history rather than leaving it wrong
Pause and resume are calendar-driven	No job needs to have run for the arithmetic to be right
Timezone is explicit on the calendar	Never inferred from a server clock
A holiday added retrospectively re-derives correctly	Same property as the above
Breach is a **flag with a timestamp**, not a deletion of the target	*Late, and by how much* must stay answerable
Waiting on the **customer** stops the resolution clock	Otherwise we are measured on their response time (**D-22** confirms whether this holds)

23.6 Escalation on breach

```
FIRST-RESPONSE CLOCK RUNNING
   │
   ├── target × warning threshold reached (D-25)
   │      └──► notify the OWNER.  Nothing else. Quiet nudge.
   │
   ├── target reached
   │      └──► flag BREACHED + timestamp
   │           notify OWNER and TEAM LEAD
   │           appears in the lead's queue view
   │
   └── breach + escalation threshold (D-25)
          └──► RAISE ESCALATION LEVEL
               route per the escalation policy
               AUDITED (FR-AUD-2 · ownership category)

THE CUSTOMER IS NOT NOTIFIED OF ANY OF THIS.
A breach is our failure to manage, not news to deliver.
(§22.5 · §27.16)
```

Thresholds are D-25. Automatic escalation on breach is PROPOSED, not assumed — some businesses prefer a lead to decide. The architecture supports both; §27.16 keeps either invisible to the customer.

23.7 What SLA is in V1, and what is not

In V1	Not in V1
Clocks computed against the business calendar	SLA dashboards and trend reporting — **FUTURE** (§42)
Breach flags with timestamps	Per-agent SLA scorecards — **forbidden by P-08**, not merely deferred
Warning and breach notification to owner and lead	Customer-visible SLA or countdown — **forbidden** (§22.5)
Automatic escalation on threshold (**D-25**)	Predictive breach forecasting
Queue depth and oldest-waiting age (**FR-REP-1**)	Cross-team benchmarking
Per-case first-response and resolution timestamps	Contractual SLA credits

The line being held. SLA state exists to get a customer answered and to tell a lead when that is not happening. It is **not** an instrument for measuring individuals — **P-08** applies with full force, and §46 records it as a rule that must not be traded away.

23.8 Availability of the platform versus availability of people

Easy to conflate, and expensive if conflated.

	Platform availability	Team availability
Governed by	**NFR-AVL-1**	The business calendar (§23.1)
After hours	**The platform stays up.** Customers can send; messages persist	Nobody is rostered
If it is down	An incident	Normal, expected, and communicated
Customer experience	Explicit service-unavailable state (§34.1)	Acknowledgement and a queued conversation

The platform is 24×7 even where no team is. A customer must always be able to reach us; what varies is when a human replies.

23.9 Do the new requirements justify Redis? — K-05, re-evaluated

The brief asks that Redis appear only where genuinely justified. Version 2.0 marked it **FUTURE**. This section adds SLA clocks, an after-hours queue and a business calendar, so the verdict was re-examined rather than assumed to still hold.

New requirement	Does it need shared ephemeral state?	Why not
SLA clock evaluation	**No**	Elapsed time is **computed on read** from the calendar (**NFR-PRF-7**), not accumulated in a counter. The sweep is a scheduled query over a **partial index of running clocks only** (§24.13)
After-hours queue	**No**	It is a conversation state plus a flag in the database. A queue that must survive a restart belongs in the database, not in a cache
Business calendar	**No**	Configuration, read rarely, cacheable **in process**
Opening-surge routing	**No**	A query ordered by priority band and arrival time (§23.4)
SLA countdown in the employee interface	**No**	Derived client-side from the case's start time and target

Verdict unchanged: Redis is not required for V1. The trigger remains exactly what §33.2 states — the first moment a second API instance holds realtime connections, at which point a shared pub/sub channel becomes **required**, not optional.

The honest caveat. If the SLA sweep ever needs to run more often than the partial index can serve cheaply, or if warning notifications need de-duplicating across instances, that is the same multi-instance trigger arriving from a different direction. It is one trigger, not two.

---

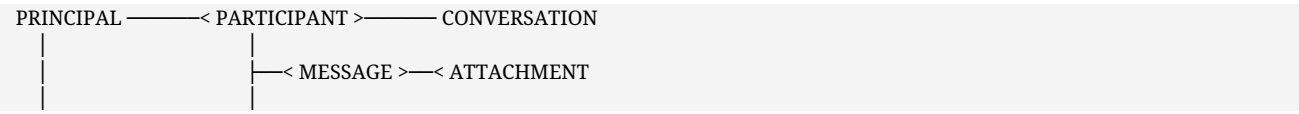
24. Data architecture

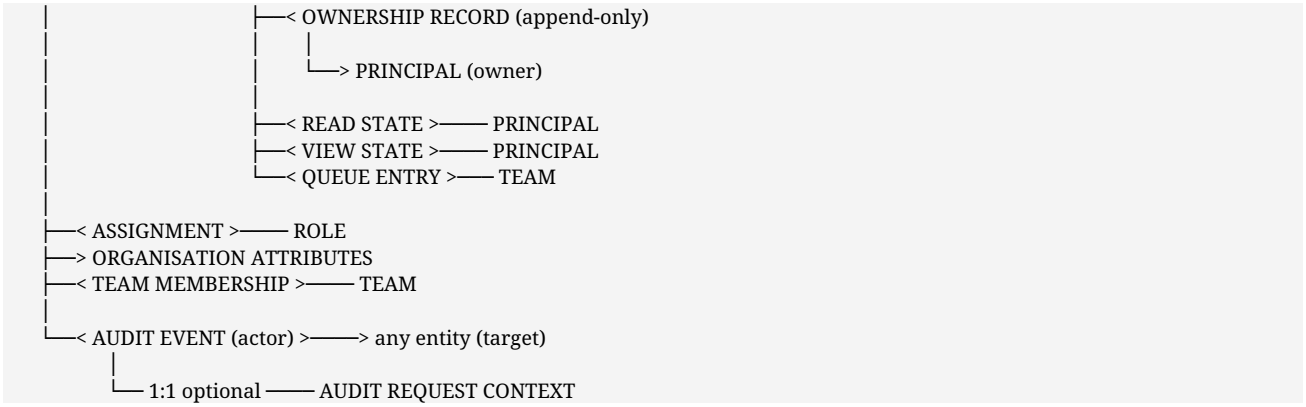
Conceptual: entities and relationships, not production schemas. Storage design follows sign-off, and premature schemas become decisions nobody remembers making.

24.1 Entities

Entity	Holds	Owning component	Notes
Principal	One actor: employee, customer, or service	Identity	One identifier is the only join key
Organisation attributes	Department, sub-department, reporting line	Identity	HRMS-sourced later; absent means absent
Team	A named group used for routing and scope	Identity	Not the same thing as a department
Role	A named set of permissions	Authorization	Which roles exist is **D-11**
Assignment	A role held by a principal within a scope, optionally time-boxed	Authorization	Expiry read from the clock
Conversation	A thread: type, state, title, activity	Conversation	Internal has no state (**D-15**)
Participant	A principal in a conversation, with role, added-by, added-at, optional expiry	Conversation	Rich from the first record (§24.5)
Message	Immutable content, author, time, internal marker	Conversation	Correction is a new message
Attachment	File metadata and storage reference	Attachment	Reference derived on read
Ownership record	Who owned a customer conversation, from when, why it changed	Routing	**Append-only**
Queue entry	An unassigned conversation awaiting claim	Routing	Atomic claim
Read state	Per principal, per conversation	Conversation	Personal
View state	Archive, mute	Conversation	Personal — **never** conversation state
Notification	A pending or delivered alert	Notification	Actionable only
Audit event	Actor, action, target, time, outcome	Audit	Append-only
Audit request context	Address, agent, correlation identifier	Audit	**Separate store, own retention**
Customer record	The minimum needed to identify and contact	Identity	Scope depends on **D-03**

24.2 Relationships





Reading the important edges. A principal reaches a conversation's content through **participation** or through an **assignment whose scope covers it** — never through department membership alone (BR-29). Ownership is a separate relationship from participation, which is what makes **P-03** expressible in the model rather than merely stated in prose. Audit references the actor and the target but is **not** referenced by them: nothing in the domain depends on audit, so audit can be retained, restricted and queried independently of everything else.

24.3 Data that is append-only

Data	Why nothing may be overwritten
Messages	The record must be trustworthy enough to argue from
Ownership records	"Who was responsible on the day" must stay answerable
Audit events	An audit trail that can be edited is not an audit trail
Participant changes	Determines who *could* have read what, historically

24.4 Indexing intent

Not schema, but a stated requirement, because getting it wrong is the reference platform's measured failure (§38):

- Message pages are read by conversation, ordered by time **with a tiebreaker on record identity**, and must be served by a compound index that keeps records-examined proportional to records-returned.
- Conversation lists are read per principal, ordered by last activity.
- Audit is queried by actor, by target, and by period.
- Request context expires automatically on its own retention.

24.5 Shapes decided now, because they cannot be fixed cheaply later

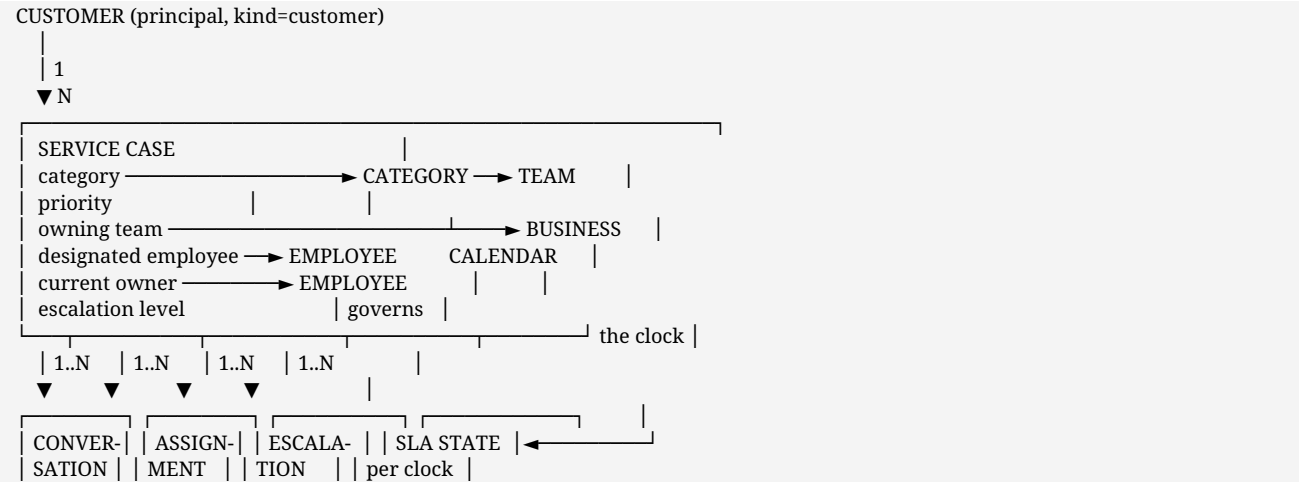
Decision	Why now
Participants carry role, added-by, added-at and optional expiry from the first record	A flat identifier list must be migrated to grant anyone a role or a time-boxed view. Richer costs nothing today
One attachment field, no singular alias	A dual shape is reconciled on every read forever, because historical documents never get migrated
Principal identifier is the sole join key	Anything else means a rename orphans history
Ownership is a **record**, not a field on the conversation	A field cannot express history, and history is what handover disputes need
Audit request context is separate from the first record	Splitting it later means rewriting an append-only store, which is a contradiction

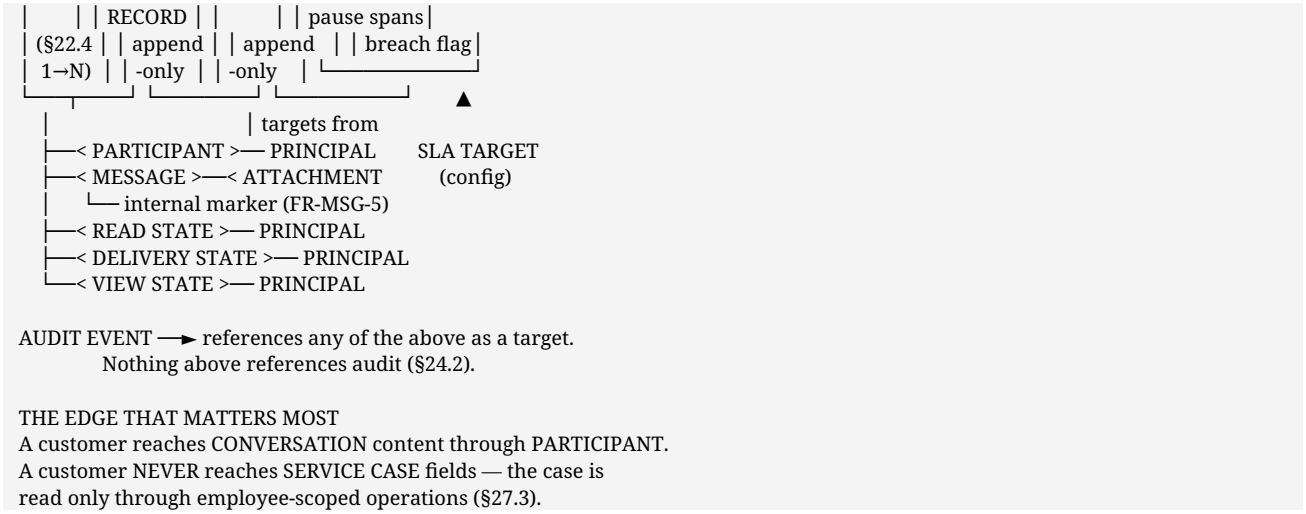
24.11 Entities added by the case, category and SLA model

Entity	Purpose	Owner	Lifecycle	V1
Service Case	The unit of customer service work: category, priority, owning team, designated employee, current owner, SLA state, escalation level and history, resolution (§22.3)	Routing	Created on first customer contact; never deleted; **survives conversation closure**	✅
Category	A node in the intake taxonomy, with its team mapping, default priority and whether it is relationship-shaped (§21.6)	Identity/admin	Seeded, then administrable. **Data, not code**	✅
Business Calendar	Working days, window, holidays, timezone and exceptions, **per team** (§23.1)	Identity	Configuration; versioned so a correction re-derives history	✅
SLA Target	The target per category or team, per clock, and whether it is business-hours or 24×7 (**D-22**, **D-23**)	Routing	Configuration	✅
SLA State	Per case per clock: started-at, stop-at, pause spans, breach flag and breach time	Routing	Runs with the case; terminal on stop	✅
Escalation	A record: level, from, to, by whom, why, when	Routing	Append-only	✅
Assignment record	Who owned the case, from when, why it changed, whether it was cover or a transfer	Routing	**Append-only** — extends v2.0's ownership record with a cover/transfer discriminator	✅
Designated-employee link	The customer→advisor relationship, and where it came from (**D-19**)	Identity	Rare change; **must be reassigned on departure** (BR-13)	✅

Internal Note is not a new entity. It remains a message carrying an internal marker (**FR-MSG-5**), because making it a separate entity would create a second path to conversation content and therefore a second place to get authorization wrong.

24.12 Relationships with the case model — diagram 20





24.13 Case-model indexes and query patterns

Collection	Index	Serves
cases	owning team + state + priority + created ascending	The queue view: oldest first within a priority band (§23.4)
cases	current owner + state	"my work", and the **inactive-owner alert** (§32.3)
cases	designated employee + customer	Routing step 3 (§21.8)
cases	customer + created descending	The customer's own case history
cases	**SLA breach flag + state**	The lead's breach view (§23.6)
cases	**SLA next-deadline ascending** *(partial: running clocks only)*	The scheduled sweep that raises warnings and breaches — **a partial index, so the sweep touches only live clocks**
cases	after-hours flag + owning team	The opening-surge view (§23.4)
conversations	case reference	Every conversation of a case (§22.4)
escalations	case + level ascending	Escalation history
assignment records	case + effective-from descending	Ownership history (**BR-14**)

The partial index on running clocks is the one worth naming. Without it, the SLA sweep scans every case ever created; with it, the sweep touches only cases with a live clock. This is the same documents-examined-versus-returned discipline that §38.2 records as a measured failure on the reference platform, applied before it becomes one here.

24.14 Denormalised fields — the list grows

The case is separate (§22.4), so the queue view would need a join. Four fields are duplicated onto the conversation to avoid it, taking the total from five to nine. Every one is a consistency risk, so the list stays exhaustive.

Field	Duplicated onto	Source of truth	Refreshed when
Author display name	message	principal	**Never** — a message keeps the name in force when sent
Owner display name	conversation	principal	On rename, and on ownership change
Last activity time	conversation	newest message	On every send
Last message preview	conversation	newest message	On every send
Participant count	conversation	participants array	On participant change
Case reference	conversation	case	Never — immutable once linked

Field	Duplicated onto	Source of truth	Refreshed when
Category	conversation	case	On recategorisation
Priority	conversation	case	On reprioritisation
Internal state	conversation	case	On every state transition

Deliberately *not* denormalised: SLA state, escalation level and breach flags. They change on a clock rather than on an action, so copying them would mean a write per tick — and they are the fields that must never reach a customer (§27.16), so keeping them on the case alone means a customer-facing conversation read cannot leak them by accident.

24.6 Additional entities the technical design requires

Four entities absent from the conceptual model in §24.1 because they are mechanism rather than domain.

Entity	Purpose	Owner	Lifecycle	V1
DeliveryState	Per recipient per message: queued, delivered to the channel, failed. Distinct from **ReadState** — delivery is the system's fact, reading is the person's	Conversation	Created on send; terminal on delivery or permanent failure	Customer messages only. Internal messages need no delivery state
NotificationOutbox	A pending external delivery with attempt count and next-attempt time (§29.4)	Notify	Pending → delivered, or → dead-letter	✅
Draft	Unsent composer content	—	—	**Client-only in V1.** No server entity. Server-side drafts are FUTURE (§42)
Reaction	An emoji response by a principal to a message	Conversation	Created and deleted freely	**FUTURE** (§42) — no use case demands it

Draft and Reaction are listed precisely to record that they are not being built. A customer's in-flight message stays in their own browser and is **never posted on their behalf** (**UC-C05**), which is a privacy property, not a limitation.

24.7 Indexes and query patterns

Stated as architecture because §38 records a **measured** failure here, not a hypothetical one: 20,000 documents examined per page instead of 301 when the compound index was absent.

Collection	Index	Serves	Why this shape
messages	conversation + created descending + **id descending**	The message page (**FR-MSG-3/4**)	The id tiebreak is what makes paging correct under concurrent insertion. **Without it, a row can repeat or be skipped across pages**
messages	conversation + internal marker	Filtering internal notes out of a customer read	Keeps the customer path from scanning staff content
messages	**text index** on body	Search (§30)	Intersected with the scope set, never queried alone
conversations	participant principal + last-activity descending	The caller's thread list	Serves the dominant employee read
conversations	team + state + created ascending	The queue (**FR-ROUTE-4**)	Oldest-waiting first
conversations	owner + state	"my conversations", and the inactive-owner alert (§32.3)	The metric that must read zero

Collection	Index	Serves	Why this shape
conversations	customer principal + last-activity	Customer's own history, and relationship lookup for routing	One index serves both
ownership records	conversation + effective-from descending	Ownership history (**BR-14**)	Append-only, read newest-first
read state	principal + conversation	Unread counts	Unique on the pair
audit	actor + time · target + time · action + time	The four forensic queries (§31.5)	Separate indexes; these queries do not combine
audit request context	correlation id · **expiry**	Join to the ledger; automatic ageing	**Store-level TTL — retention must not depend on a job having run**
attachment metadata	conversation · binding state + created	Authorized retrieval; expiring unbound uploads	§28.6
outbox	state + next-attempt	The worker's poll	The only hot query in the notification path
principals	username unique · active + display name · manager	Sign-in, directory, reporting scope	Scope resolution walks the manager edge

The one query that must be monitored, not merely indexed

documents examined ÷ documents returned for the message page should sit near 1. Drift means a lost index or a changed query shape, and §32.3 alerts on it. This is the earliest available warning of the exact regression §38 measured.

24.8 Where MongoDB's trade-offs actually bite

Named specifically, with the mitigation, rather than left as a general caveat.

Trade-off	Where it bites	Mitigation
No multi-document atomicity by default	Claim-and-assign touches the conversation and creates an ownership record	The claim is a **single-document conditional update** on the conversation; the ownership record follows. If the second write fails, the conversation is claimed but unrecorded — detected by the inactive-owner and unassigned metrics (§32.3), and correctable. Where genuine atomicity is required, an explicit transaction on the replica set
No joins	The queue view needs conversation state *and* owner display identity	**Controlled denormalisation**: the owner's display name is duplicated onto the conversation, refreshed on rename. §24.9 lists every duplicated field
Weaker text search	Message search ranking (§30.3)	Accepted for V1; trigger recorded in §30.4
Schema discipline in the application	Any shape that varies	§24.5 fixes the unfixable shapes before first write
Unbounded array growth	Participants on a very large group	Bounded by product limits, not by the database. Announcements use kind-scoped visibility (§26.3) rather than 200 participant entries

24.9 Denormalised fields — the complete list

Every duplicated field is a consistency risk, so the list is short and exhaustive.

Field	Duplicated onto	Source of truth	Refreshed when
Author display name	message	principal	Never — **a message is a**

Field	Duplicated onto	Source of truth	Refreshed when
			historical record and keeps the name as it was. The principal id is the join key
Owner display name	conversation	principal	On rename, and on ownership change
Last activity time	conversation	newest message	On every send
Last message preview	conversation	newest message	On every send
Participant count	conversation	participants array	On participant change

Author display name is deliberately never refreshed. A message showing the name in force when it was sent is the correct behaviour for a record someone may later have to argue from — and the principal id is what anything correlates on (**FR-EMP-1**).

24.10 Proposed database separation

Database	Collections	Why separate
`starlink_identity`	principals, organisation attributes, teams, roles, assignments	It is the seam the HRMS will cross (§17.2) — a separate database makes that boundary visible
`starlink_chat`	conversations, participants, messages, read and view state, ownership records, queue, delivery state, notifications, outbox, attachment metadata	The domain
`starlink_audit`	audit ledger, audit request context	Different access control, different retention, and ideally a **write-restricted database role** (§31.5)

All three names are PROPOSED. The runtime guard refuses any database whose name does not begin `starlink_` (§35.4), which is what makes cross-product isolation a property rather than a promise.

---

25. API architecture

PROPOSED, and conceptual. Operations are named to show architectural completeness and to make the authorization surface countable. **No request or response schemas** — those follow sign-off, and premature schemas become decisions nobody remembers making.

25.1 Shape

Aspect	Proposal	Reason
Style	REST over HTTPS, versioned at the path root	The client surface is narrow and known; see §14.2 for why GraphQL is rejected
Versioning	A single version prefix; additive change preferred over new versions	Two consumers only, both first-party
Transport for reads and writes	HTTP	Realtime is additive (§20.3), never the only path
Serialisation	JSON	—
Errors	A small closed set of shapes; **the same body for "not permitted" and "does not exist"** (§27.3)	Existence must not be disclosed
Paging	Opaque signed cursor; never offset (**FR-**	Correct under concurrent insertion

Aspect	Proposal	Reason
	MSG-3)	
Idempotency	A client-supplied key on message send and on claim	§34.7
Two surfaces	**Separate operation sets**, not one set with role filtering	§18.4 step 4 — filtering is not a boundary

The customer operation set is separate and small. Every customer-reachable operation is enumerable on one page, which is what makes it reviewable. An operation absent from §25.4 is not reachable by a customer.

25.2 Employee surface — operations by domain

Authentication and session

Operation	Purpose
POST sign-in	Verify a credential, issue a session (§26.2)
POST sign-out	Invalidate the session
GET me	The current principal, permission set, and preferences — one call the shell depends on
POST change own credential	Self-service; audited

Directory and principals

Operation	Purpose
GET directory	Active principals, subject to visibility (**D-11**)
GET principal	One principal's display identity and organisation attributes
GET teams	Teams the caller may route to or see

Conversations

Operation	Purpose
GET conversations	The caller's threads, cursor-paged on last activity
POST conversations	Create an internal 1:1 or group
GET conversation	One thread's metadata and participants — **the object check of §18.4 step 3 lives here**
PATCH conversation	Title; and lifecycle transitions for customer threads (resolve, reopen)
GET queue	Unassigned conversations for the caller's teams

Participants

Operation	Purpose
POST participants	Add — exposes prior history, and the interface must warn first (**BR-07**). Audited
DELETE participant	Remove — ends future access, does not un-read (**BR-09**). Audited

Messages

Operation	Purpose
GET messages	One page, newest-first, cursor-paged (**FR-MSG-3/4**)
POST message	Send. Carries the internal marker; **fails if the marker cannot be established** (**FR-MSG-6**)
POST read	Mark read for the caller only, debounced (**FR-READ-4**)
POST typing	Ephemeral; stored nowhere

Assignment and routing

Operation	Purpose
POST claim	**Atomic** — a second claimant is told it is taken, not shown an error (**FR-ROUTE-3**)
POST assign	Assign to a named principal. Audited
POST transfer	Move ownership, **reason required**. Audited
POST escalate	Route to a function, original owner retained as participant. Audited
POST cover	Answer under team scope while the owner is away (**UC-E07**). Audited
GET ownership history	The append-only record — who owned this, when, and why it changed

Attachments

Operation	Purpose
POST attachment	Upload; validated server-side (§28.2)
GET attachment	**Streamed through the application after a conversation check. Audited** (**FR-ATT-5**)

Search

Operation	Purpose
GET search	Scope resolved **before** query (§30.2). **Audited with the term** (**FR-SRCH-3**)

Notifications

Operation	Purpose
GET notifications	The caller's actionable items
POST notifications read	Acknowledge
GET/PATCH preferences	Channel and frequency preferences

Administration

Operation	Purpose
GET/POST/PATCH accounts	Manage principals
POST role assignment	**Audited as must-succeed** — whoever assigns roles could otherwise grant themselves anything (**FR-ADM-2**)
POST deactivate	Ends sessions, blocks sending, **surfaces owned conversations for**

Operation	Purpose
	reassignment (**FR-EMP-3**)
GET audit	Restricted to the compliance role (**FR-AUD-6**)

Oversight

Operation	Purpose
GET queue metrics	Depth and waiting time (**FR-REP-1**)
GET load	Per team and per owner, **for capacity** — never a ranking (**FR-REP-3**)

25.3 Customer surface — the complete set

Deliberately short. **This table is the customer's entire reachable API.**

Operation	Purpose	Notes
POST identify	Begin verification	Method depends on **D-02**
POST verify	Complete verification, issue a bounded session	**Audited**
POST conversation	Start a thread by **intent**, never by naming a person (**BR-02**)	Channel depends on **D-01**
GET conversations	The customer's own threads only	
GET conversation	One own thread	Internal notes and participant roles **absent from the response**, not hidden in the client
GET messages	One page of an own thread	**Read is audited** (**UC-C04**)
POST message	Reply	
POST read	Own read state	
PATCH conversation reopen	Inside the window (**D-08**)	
POST attachment	**Only if D-07 permits**	AV scanning required (§28.2)
GET attachment	Own thread's attachments only	Audited

Not present, and not reachable: directory, other customers, participant lists, internal notes, staff receipts, search (**D-08**), export (**D-08**), queue, load, audit, administration.

25.4 Authorization at the API boundary

Every operation above is annotated in implementation with the permission it requires, so the set of operations and the set of permissions can be **diffed.** An operation with no declared permission fails the build rather than defaulting to open — the same fail-closed posture as **FR-AUTHZ-3**.

25.5 What the API does not do

Not	Why
Expose a generic query interface	Every query shape is a scope check to get right (§14.2)
Return storage URLs	**FR-ATT-1** — bytes stay behind the application
Accept a conversation id as authority	§18.4 step 3 — the object is loaded and authorized together
Offer bulk export	**D-08**, and it is an exfiltration surface
Stream audit	§31.5 — queried by a restricted role, never streamed

26. Authentication and authorization

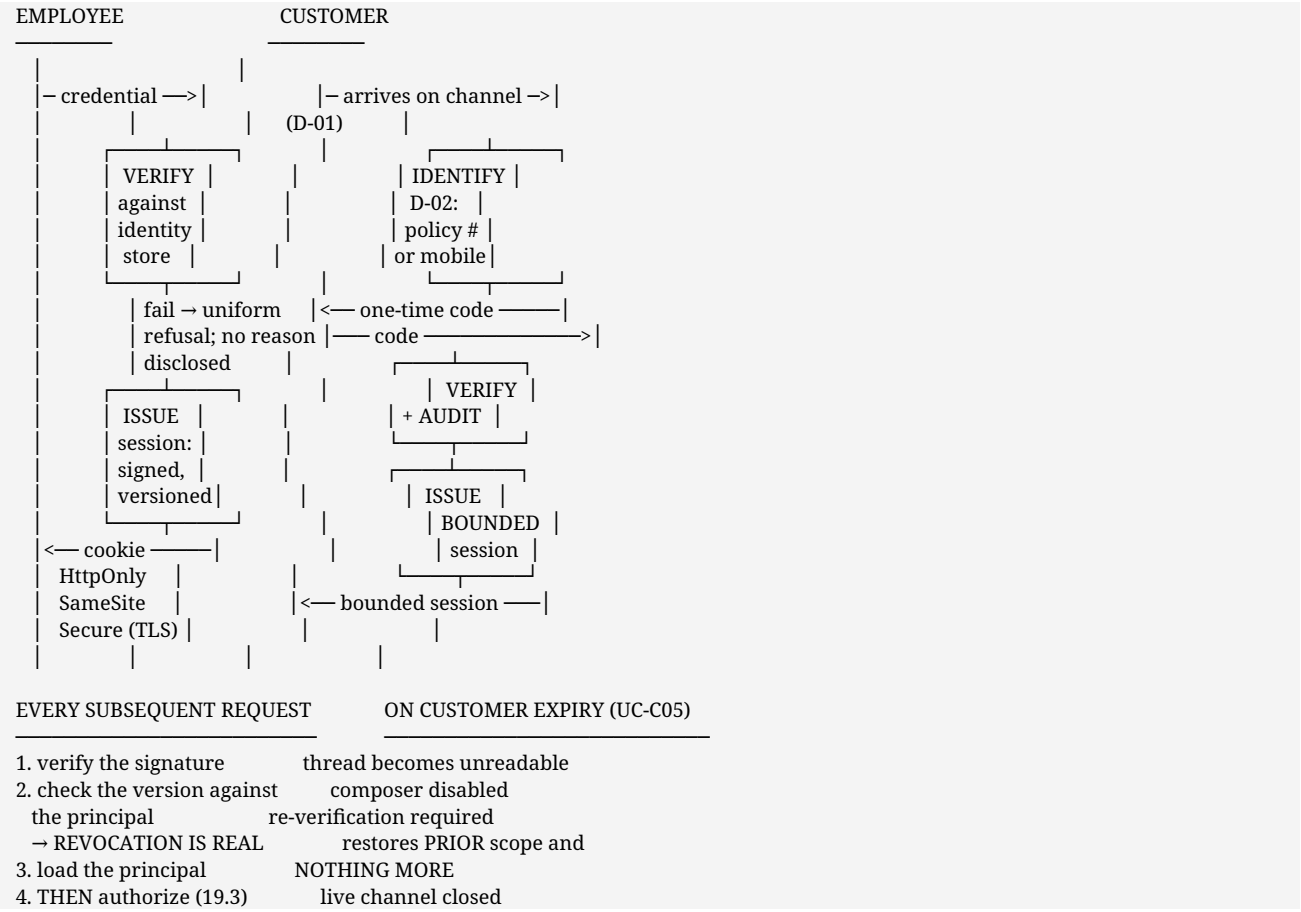
Separated deliberately, because conflating them is the defect class that produces the widest breaches.

	Question	Answers with	Failure mode
Authentication	Who are you?	A verified principal identity	Impersonation
Authorization	What may you do?	Allow or deny, per action, per object	Over-broad access

26.1 Two populations, two flows, no shared credential path

	Employee	Customer
Identity source	StarLink's store in V1; HRMS later (§17.2)	**D-03 — undecided**
Method	Credential, then a signed server-issued session	**D-02 — proposed: verified session**
Session lifetime	Appropriate to a working period	**Bounded, shorter**
On expiry	Re-authenticate	Re-verify; restores prior scope only (UC-C05)
Revocation	By session version, effective next request	Same, plus account closure
Directory access	Yes	**Never**

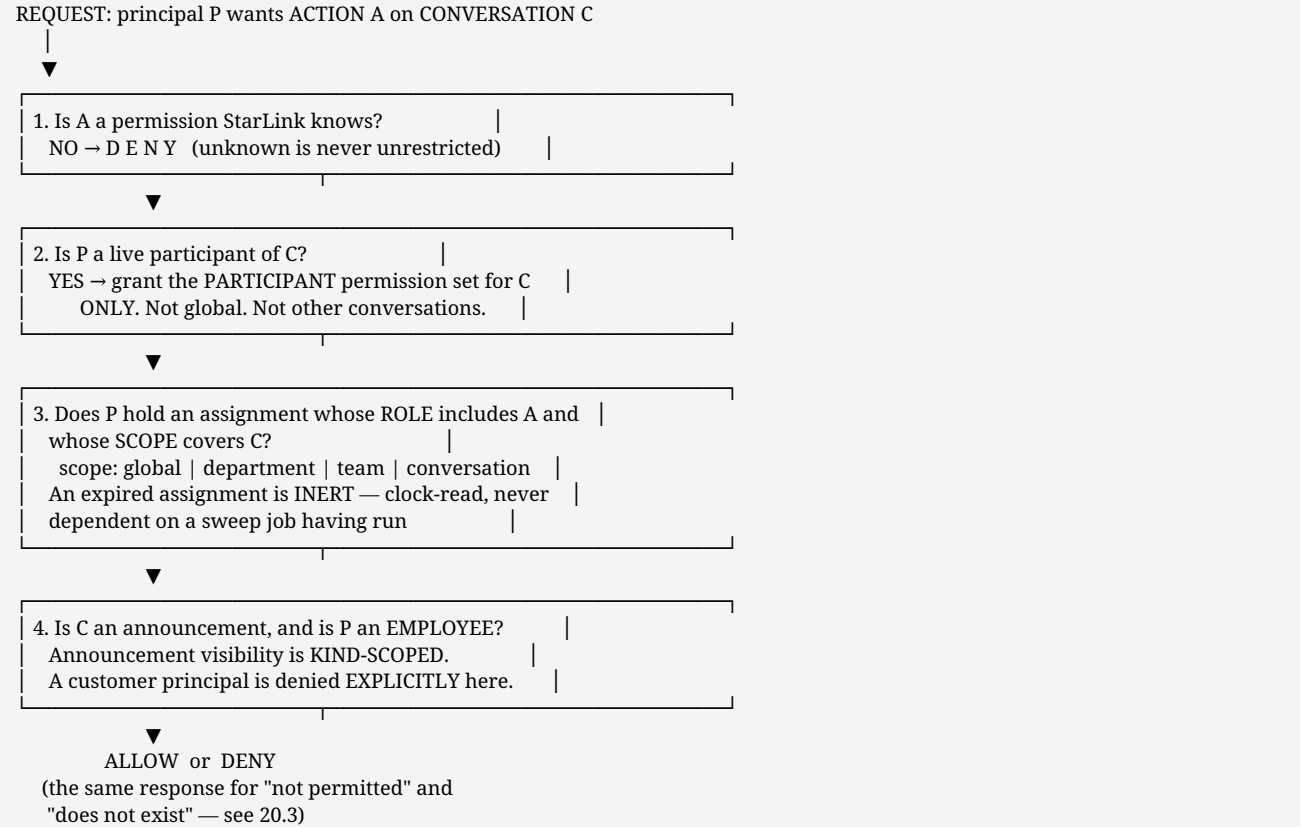
26.2 Authentication flow — diagram 7



```
                    SERVER-SIDE
```

Why a version check on every request. A signed session needs no lookup to be *read* — but revocation would then be impossible. Checking the version against the principal makes deactivation and role change effective on the **next request**, not at a cache expiry. A revocation that lags is a revocation that did not happen when somebody believed it had.

26.3 Authorization model



Step 4 exists because of a specific rejected pattern. The reference platform had a broadcast flag that bypassed membership without consulting any scope. Carried into a product where customers are principals, that flag would expose company announcements to customers. §38 records it as REJECT; this step is where the rejection is enforced.

26.4 Session expiry and revocation

Trigger	Effect	Audited
Employee session expires	Re-authentication required	No — a clock, not an act
Customer session expires	Thread unreadable, composer disabled, re-verify (UC-C05)	No
Re-verification succeeds	Prior scope restored, nothing widened	**Yes**
Role changed	Effective on the next request	**Yes** — must-succeed
Account deactivated	Sessions end; sending blocked; history readable to the entitled; owned conversations surfaced for reassignment	**Yes**
Customer account closed	All access ends	**Yes**

---

27. Security architecture

27.1 Session and transport

Server-issued signed sessions, revocable by version · HttpOnly, so script cannot read the cookie · SameSite-restricted, so a cross-site request carries nothing · Secure whenever TLS is present · CSRF defence on every state-changing request · **uniform failure responses**, because distinguishing "wrong password" from "no such account" tells an attacker which half they got right and tells a legitimate user nothing they can act on.

27.2 Authorization hardening

Control	Rationale
Unknown permission denied	Fail closed
Participation is conversation-scoped	The single most valuable containment property in the model
Expiry read from the clock	No job's failure silently extends access
Absent attributes are absent, never blank	Prevents one blank matching another and granting something
Announcement visibility kind-scoped	§26.3 step 4
Administration confers no read	Whoever manages roles is not thereby a reader

27.3 Customer isolation

The hardest boundary in the product, and the one worth testing hardest.

Control	Effect
Customer principals are a distinct kind, checked explicitly wherever kind matters	No accidental inclusion in employee-scoped results
Scope resolved **before** the query, never filtered after	A forgotten filter leaks; an absent scope returns nothing
Identical response for "not permitted" and "does not exist"	Existence is not disclosed
No customer-addressable path to the directory, to other customers, or to staff structure	§11.7
History from before verification is unreadable	A hijacked session inherits nothing
Realtime events carry identifiers, not bodies	A misrouted event discloses nothing

27.4 Attachment security

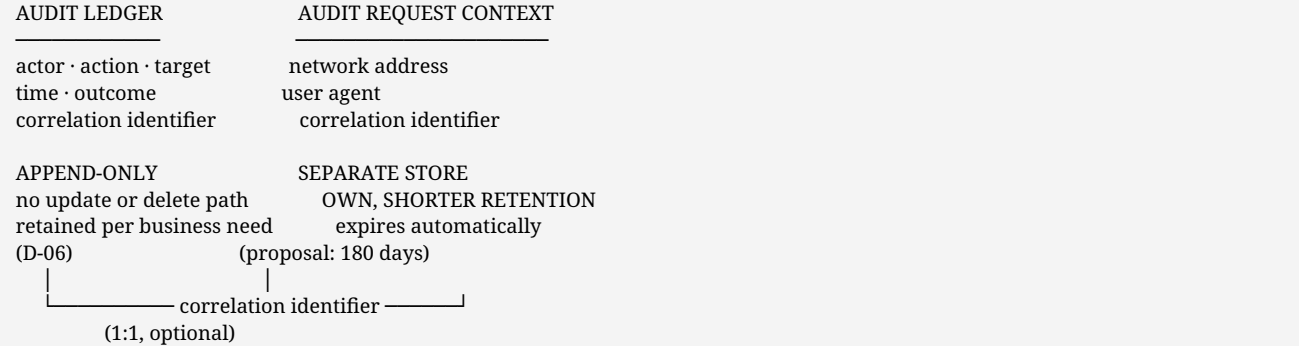
Authorization through the conversation, never through URL knowledge · server-side type and size limits · response headers that prevent inline execution in the browser · references derived on read · **download audited** · customer uploads require anti-virus scanning if permitted at all (**D-07** — and the scanning cost is part of that decision, not an implementation detail).

27.5 Rate limiting and abuse

Surface	Why
Authentication and customer verification	Credential and code guessing
Customer intake	The only unauthenticated-adjacent surface, and an abuse target
Search	Expensive, and an exfiltration tool
Attachment upload	Storage exhaustion
Realtime subscribe	Connection exhaustion

27.6 Audit and retention

Two stores, one purpose.



Why split. A retention obligation says "delete identifying data after N days". An audit obligation says "the ledger is immutable". Held in one record these requirements are in direct conflict, and one of them gets quietly broken. Split, both are satisfiable: the ledger keeps *what happened*, and the identifying context ages out on its own schedule while the ledger remains intact and still meaningful without it.

Audited: authentication events · authorization grants and revocations · assignment, transfer and escalation · participant added and removed · **privileged reads** of customer history · attachment downloads · searches **with the term** · administrative actions.

Not audited: ordinary internal message sends. Auditing them is surveillance, not accountability (**P-06**, **P-08**).

Failure policy: for a security-critical action, audit write failure **fails the action**.

27.7 Secrets and configuration

Secrets from configuration only, never from source · production refuses to start on a shipped development default · the configuration namespace is StarLink's own and shares nothing with any other platform · no credentials in messages, logs or audit records.

27.8 Threats considered

Threat	Control
Customer sees an internal note	Distinct composer, explicit marker, fail-closed send (UC-E03), separate surface (**D-16**)
Customer sees another customer	Distinct principal kind, scope-before-query, uniform responses
Employee reads a conversation they have no part in	Participation is conversation-scoped; no global member grant
Hijacked customer session inherits history	History from before verification is unreadable
Live channel outlives its session	Server-side invalidation at session end
Attachment retrieved by guessing a URL	Conversation-based authorization
Departing employee retains access	Deactivation ends sessions; owned conversations surfaced
Whoever grants roles grants themselves everything	Role assignment audited as must-succeed
Bulk extraction through search	Scoped, rate-limited, and audited **with the term**
Exfiltration through message forwarding	Not in V1. If built, source and every target are recorded (§12.4)

27.9 The four concepts that must never be conflated

This distinction is the most important thing in the security architecture. Every serious access defect in a product like this comes from collapsing two of these four into one.

Concept	The question it answers	Where it is decided	What it does not grant
Authentication	*Who are you?*	§26.2, edge guard	Nothing. Being signed in grants no read of anything
Authorization	*May you perform this action on this object?*	§26.3, object check	Business authority. A compliance role may query audit and still not own a conversation
Business ownership	*Who is accountable for this customer conversation?*	Routing module (§18.2)	Exclusive read. Participants also read it
Conversation participation	*Are you part of this thread?*	Participants on the conversation	Ownership. **Participation grants reading that conversation and nothing else** (**P-03**)

```
SIGNED IN                  ── says nothing about access
   │
   ├── PARTICIPANT of C    ── may read C.  May NOT reply to the
   │                          customer, reassign, or close.
   │
   ├── OWNER of C          ── accountable. May reply, resolve,
   │                          transfer. Exactly one at a time.
   │
   └── SCOPED GRANT        ── may act across a team or globally,
                              time-boxed, and the read is
                              AUDITED AS PRIVILEGED.

THE FAILURE MODE THIS PREVENTS
An early prototype granted every member a global read permission
because "a member can read conversations" was collapsed into
"a member can read". A test caught a non-participant reading a
conversation. The fix was structural, not a patch.
```

27.10 Input validation

Rule	Reason
Validate at the boundary, on a **declared schema per operation**	An operation with no schema fails the build, the same fail-closed posture as **FR-AUTHZ-3**
Reject unknown fields rather than ignoring them	An ignored field is a field someone will later assume was honoured
Never trust a client-supplied id as authority	§18.4 step 3 — the object is loaded and authorized together
Bound every string, array and upload server-side	A client-side limit is a courtesy
Validate the **cursor signature** before use (**FR-MSG-3**)	An unsigned cursor is a client-supplied database query
Treat every identifier as opaque	A malformed id is a 400, never an internal error

27.11 Output encoding and XSS

Surface	Control
Message bodies	**Rendered as text, never as HTML.** Message content is plain text in V1
Rich text	**FUTURE**, and if it arrives it is a sanitised allow-list of tags produced server-side — never client-side sanitisation, which is bypassable
Links in messages	Detected at render, `rel="noopener noreferrer"`, and **never auto-followed or previewed** — a link preview would make the server fetch a URL a customer chose

Surface	Control
Filenames	Escaped on display; never used as a path (§28.3)
Content Security Policy	Per surface, no inline script, no external script or style origins beyond what is declared
Server-rendered data	§19.3 — customer pages fetch only through customer-scoped operations, so the serialised payload contains nothing extra
Framing	Denied, except where a business decision requires the customer surface to be embeddable — which would be a **D-01** consequence, and a decision with its own risks

27.12 Brute force and credential protection

Control	Detail
Rate limit per identifier **and** per source	Limiting only by source lets a distributed attempt through; limiting only by identifier allows enumeration
Progressive delay, then temporary lock	Lock duration bounded — a permanent lock is a denial-of-service against a real employee
Uniform failure response	Same body, same timing characteristics, for wrong credential and unknown account (§27.1)
Credential storage	A memory-hard password hash with per-credential salt. Never a general-purpose fast hash
One-time codes (customer, **D-02**)	Short expiry, single use, attempt-limited, and **invalidated on success**
Alerting	Failure-rate spike above baseline (§32.4)
Audit	Failures audited; **a burst against one identity is what a takeover attempt looks like** (§31.1)

27.13 WebSocket authorization

Stated separately because it is the control most often forgotten, and §38 records that the reference platform forgot part of it.

Control	Why
Authenticate at the **upgrade** using the same session mechanism as HTTP	A separate realtime auth path is a second implementation to get wrong
Authorize **every room join** with the same decision as an HTTP read	Otherwise the subscription is a back door around §18.4
Close connections server-side when the session ends	A socket outliving its session is an authorization hole
Re-validate session **and** re-authorize rooms on reconnect	A session revoked while disconnected must not resume
Force removal from a room when participation ends	Continued delivery after access was revoked
Publish-time kind check on internal notes	The worst leak available in the product
Connection limit per principal	Exhaustion (§27.5)
Never send bodies to unestablished subscribers (**FR-RT-4**)	A misrouted event discloses nothing

27.14 Secret management

Rule	Detail
Secrets from configuration only	**NFR-SEC-3**, §35.3
Production refuses to start on a shipped default, a short secret, or a non-`starlink_*` database	A misconfigured production system that starts is worse than one that refuses
Never logged, echoed by a health check, or included in an error report	§32.2

Rule	Detail
Distinct secrets per purpose	Session signing and cursor signing are separate keys, so one compromise is not both
Rotation	Session secret rotation invalidates sessions — an accepted, planned event
At rest	The platform's secret store. **A file on a server is not a secret store**
In the repository	Nothing. A committed secret is a rotated secret

27.15 Security review as a gate

Because §15.7 and **ADR-07** commit to in-house session and authorization code, a security review is a **gate before the V1-B customer pilot**, not an optional courtesy (§45 Phase 3). Minimum scope:

1. The object check of §18.4 step 3 on **every** content path, with negative tests for each.
2. Customer isolation — the §27.3 controls, tested adversarially rather than inspected.
3. The internal-note boundary, including the realtime publish path and the fail-closed send.
4. Session revocation timing, including the reconnect path of §20.9.
5. Attachment authorization, including the internal-note attachment case (§28.4 step 4).
6. Rate limits and the uniform-response property under timing observation.

27.16 Internal operational metadata must never reach a customer

The case model (§22) adds fields that did not exist in version 2.0, and every one of them is customer-invisible. Enforced at three layers, because a single layer is a single point of failure.

Layer	Control
Operation	Customer-facing operations read **conversations, never cases** (§27.3). The case is reachable only through employee-scoped operations
Serialisation	A customer-facing serialiser **declares the fields it emits** and drops everything else. **Fail-closed:** an unrecognised field is omitted, never passed through. A new case field is therefore invisible by default rather than visible by accident
Realtime	Publish-time filtering — SLA, escalation and priority changes are published to **staff rooms only** (§20.10). A customer connection is never joined to a staff room

Never sent to a customer, on any path: SLA state, targets, elapsed time, breach flags · escalation level, reason or history · priority · assignment history · designated employee · internal routing information (the queue, the fallback path, who was tried) · internal notes · audit records · employee hierarchy.

The failure this prevents. A customer who sees *first response: breached* has been handed a grievance the company generated itself. It is also the kind of leak that arrives by accident — someone adds a field to a shared response object and it appears on both surfaces. A declared allow-list is why that cannot happen.

27.17 Case-scoped authorization

The case adds a second object to authorize, so the §18.4 object check extends rather than being duplicated.

Rule	Reason
Case access derives from **the conversations it contains**, plus team scope	One authorization model, not two. Participation in any conversation of a case grants reading that case's operational fields — nothing wider

Rule	Reason
Reading a case a principal owns no conversation in requires an **explicit scoped grant**, and is **audited as privileged**	The same treatment as a privileged conversation read (§31.1)
Changing priority, category or escalation requires a permission distinct from replying	An advisor who may answer need not be able to re-prioritise
The queue view returns **cases**, filtered by team scope **before** query	Scope before query (§30.2), applied to the case collection
A customer principal is **denied at the operation layer**, not the field layer	§27.16 — field filtering is defence in depth, never the boundary

27.18 Encryption at rest — NFR-SEC-8

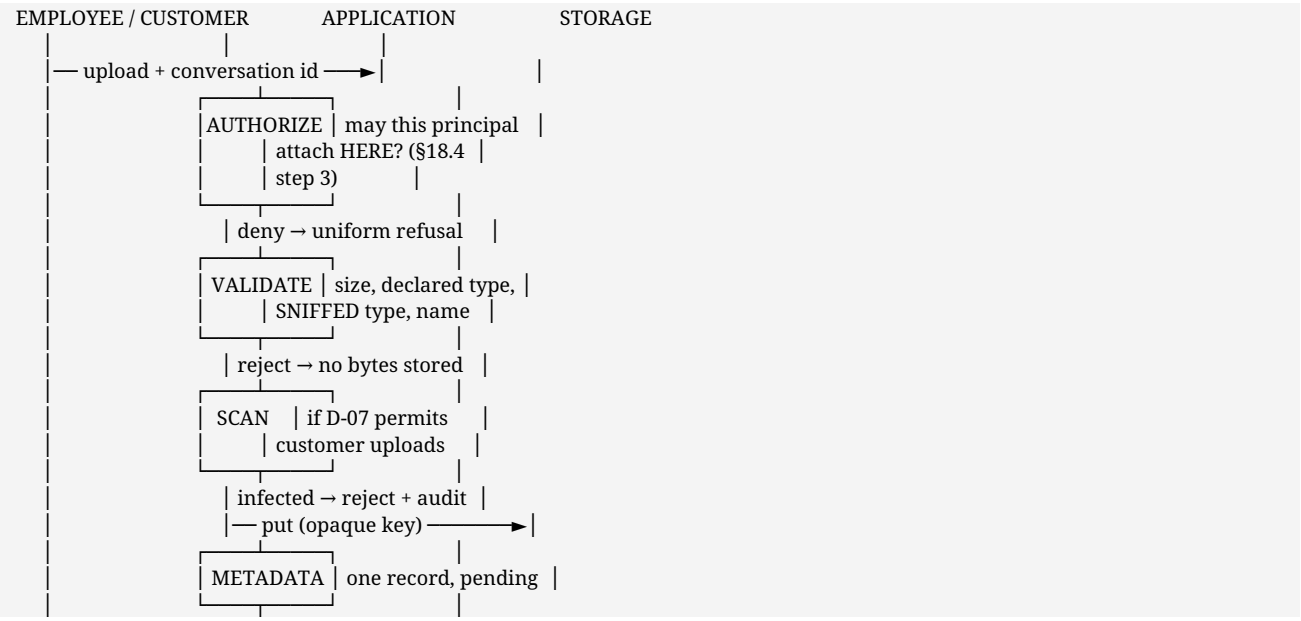
Scope	Position
Database	Encrypted at rest. **PROPOSED** — the mechanism (storage-level or engine-level) is a **PRODUCTION DECISION** (§37.1)
Object storage	Encrypted at rest — attachments include claims documents
Backups	Encrypted. **This is the one most often missed:** an encrypted database with plaintext backups is not encrypted
Audit store	Encrypted, with its own access control (§31.5)
Key management	The platform's key service. Keys never in configuration, never in the repository (§27.14)
What it does **not** protect against	An application-level authorization defect. Encryption at rest defends against a stolen disk, a copied backup or a decommissioned volume — **not** against §18.4 being wrong. Both are required

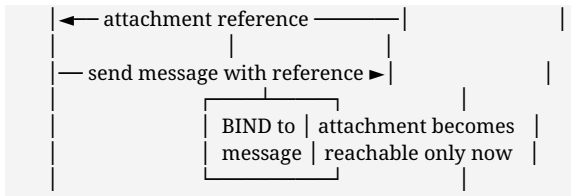
---

28. Storage and attachment architecture

PROPOSED. Claims and grievance work is document work — a claim without its documents is not a claim — so this is a first-class concern rather than a detail of messaging.

28.1 Upload flow





AN UPLOADED-BUT-UNSENT ATTACHMENT IS REACHABLE BY NOBODY.
Binding to a message is what grants access, because access is
derived from the conversation (§28.4). Unbound uploads expire.

28.2 Validation — server-side, always

Check	Rule
Size	Enforced server-side against a configured maximum. A client-side limit is a courtesy
Declared type	Allow-list, never a deny-list
Actual type	**Content sniffed and compared to the declared type.** A mismatch is a rejection — trusting the declared type is how an executable arrives named as an image
Filename	Sanitised; never used as a storage key (§28.3) and never echoed into a path
Byte count	Verified against what was received, not what was claimed
Malware	**Required if D-07 permits customer uploads.** For employee uploads in V1-A, scanning is **FUTURE** — a stated, deliberate gap, because employees are authenticated and attributable while customers are neither to the same degree

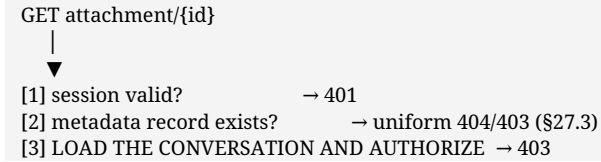
The honest statement on scanning: V1-A ships without malware scanning on employee attachments. That is acceptable only because every upload is attributable to an authenticated principal and recorded. It becomes unacceptable the moment **D-07** admits customer uploads, and §39 lists the scanning provider as a decision with a cost attached.

28.3 Object naming and metadata

Concern	Approach	Why
Storage key	**Opaque, server-generated**, carrying no meaning	A key containing a conversation id, customer name or original filename leaks information and invites enumeration
Original filename	Held in **metadata**, not in the key	Presented on download; never trusted as a path
Metadata record	Owns: id, conversation, uploader, declared and sniffed type, size, created time, scan status, binding state	The metadata is the authority; storage holds only bytes
Storage layout	Flat namespace with a partition prefix	No directory structure to reveal organisation

The reference URL is derived on read, never stored (§38) — a route baked into a document becomes a migration the first time the route moves.

28.4 Download and access authorization



```
   (the same object check as §18.4 step 3 — the attachment's
    access is ENTIRELY derived from its conversation)
[4] is the caller a customer, and is this an internal-note
   attachment?                    → 403, always
[5] AUDIT THE DOWNLOAD (FR-ATT-5)
[6] stream the bytes through the application
```

Why bytes stream through the application rather than via a signed direct URL. Steps 3, 4 and 5 must all happen, and a signed URL handed to a client performs none of them at fetch time — it converts an authorization decision into a bearer token that can be forwarded. **FR-ATT-5** requires the *download* to be the audited event, and a signed URL audits the *issuance* instead.

When a signed URL becomes acceptable: for large-file performance, if it is short-lived, scoped to one object, issued only after steps 1–5 have all passed, and the issuance is what gets audited. That is a **FUTURE** optimisation with a stated cost, not a V1 shortcut.

28.5 Customer versus employee restrictions

	Employee	Customer
May upload	Yes (V1-A)	**Only if D-07 permits**
Type allow-list	Broader — claims documents, images, PDFs	Narrower
Size limit	Configured	Lower
Malware scanning	FUTURE (§28.2)	**Mandatory**
May download	Own conversations, per §28.4	Own conversations only
Internal-note attachments	Visible to staff participants	**Never** — step 4 above

28.6 Retention and deletion

Concern	Position
Retention period	**D-06** — needs Legal. The architecture implements whatever it says
Deletion request	If a deletion obligation exists, it must be reconciled with the append-only ledger — **D-06** again. The split in §31.4 is what makes this tractable
Unbound uploads	Expire on a schedule; they were never reachable
Attachments of a deleted conversation	Follow the conversation's retention, not their own
Storage deletion	The metadata record's removal is authoritative; byte deletion follows and is retried until it succeeds
Backup	**NFR-RCV-1** — backup must cover attachments. A backup that omits them does not restore a claim. This is called out because it is the classic omission

---

29. Notification architecture

PROPOSED. In-app notification is required for V1. Every external channel depends on a business decision that has not been made — **D-01** (customer channel) and **D-12** (what customers are told).

29.1 The governing principle

P-05 — the record is written first, then delivery is attempted. A notification that fails must never mean a message that was not stored. Everything in this section follows from that ordering, and reversing it would be the single worst change anyone could make here.

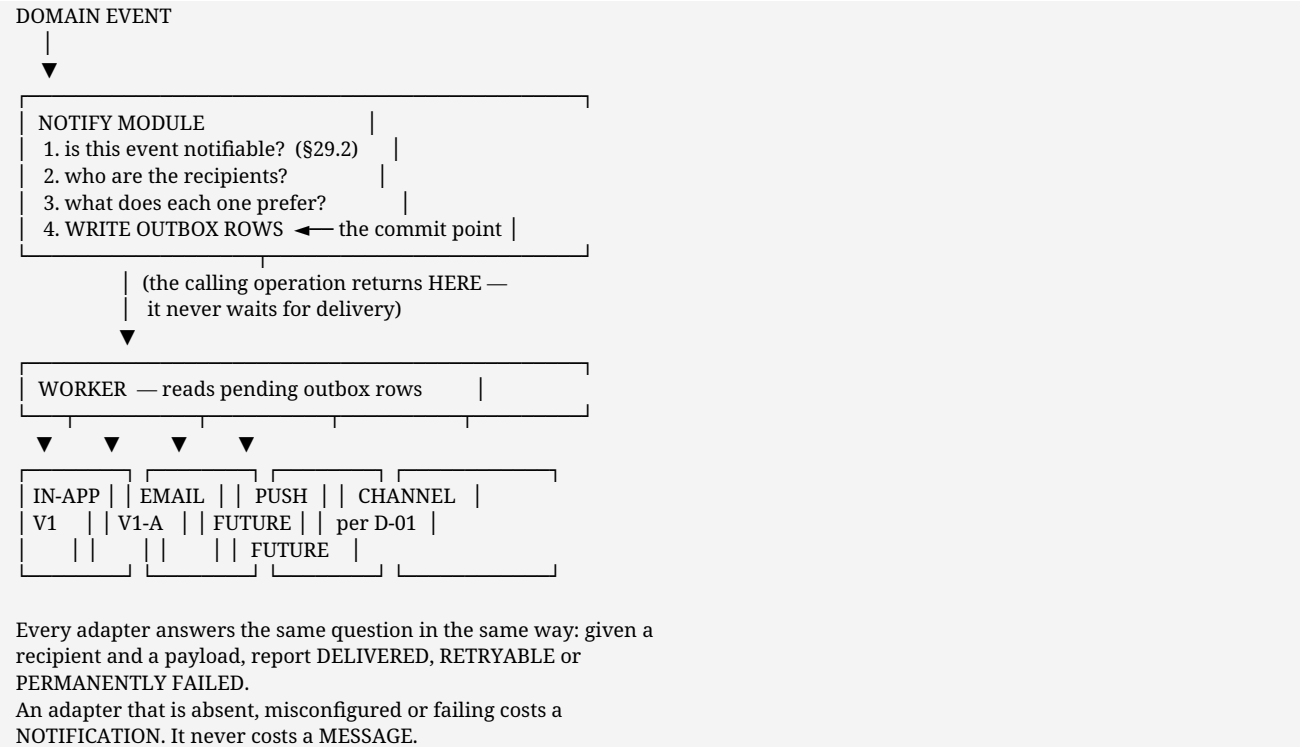
29.2 What is notified, and what is not

Notify only what someone must act on. Everything else is an unread count (§20.2).

Notified	Recipient	Channel
A customer conversation assigned to you	Owner	In-app + external if away
A conversation waiting beyond a service standard	Owner, then lead	In-app + external
Escalation to your function	Receiving officer, lead	In-app + external
Transfer into or out of your ownership	Both parties	In-app + external
Cover needed on your team	Team	In-app
A customer replied to a thread you own	Owner	In-app + external if away
Your role or access changed	Principal	In-app + external
A new conversation in your team's queue	Team	In-app
A customer's conversation was answered / resolved	Customer	**D-12 decides the channel**

Not notified: every message in a busy internal group · typing · read receipts · directory changes · your own actions · **internal notes, to anyone customer-facing, ever** (**BR-26**).

29.3 Adapter model



29.4 The outbox, and why not a broker

Requirement	How the outbox meets it
At-least-once delivery	A row persists until an adapter reports success or permanent failure
Survive a restart mid-delivery	The row is still pending; the worker picks it up
Retry with backoff	Attempt count and next-attempt time on the row
Never block a write	The write commits the row; the worker is asynchronous
Visible failures	Rows exceeding the attempt limit become a dead-letter state, and §32.4 alerts on the count

Why not Kafka or RabbitMQ. A broker is the right answer when many independent consumers need the same event stream, or when volume exceeds what a database-backed queue can carry. StarLink in V1 has **one** consumer — its own worker — and a notification volume bounded by roughly 200 employees plus one pilot department's customers. A broker would add a component to operate, monitor and back up, in exchange for a guarantee the outbox already provides.

Trigger that would change this: fan-out to multiple external consumers, or notification volume where polling an outbox collection becomes the bottleneck. Named here so the decision is revisited on evidence.

29.5 Idempotency and duplicate suppression

Risk	Control
The worker delivers, then crashes before marking the row	Each row carries a stable key; adapters that support idempotency keys receive it. **At-least-once is the honest guarantee** — a rare duplicate email is acceptable, a lost assignment notification is not
The same event notified twice	Deduplicated on (recipient, event, target) within a short window
A storm from one conversation	Coalesced — "3 new messages", not three notifications
Two instances running the worker	Rows claimed by conditional update, so a row is delivered by one worker only

29.6 Preferences and failure

Concern	Position
Preferences	Per principal, per channel, per category. In-app is **not** disableable — it is the unread mechanism
Away detection	No realtime connection for a configured period. Not presence (§20.2) — simply "not currently connected"
Quiet hours	**FUTURE**; needs a business position
Customer preferences and opt-out	**D-12**
Permanent failure (invalid address)	Row dead-lettered, principal flagged for administrative attention. Not retried forever
Provider outage	Rows accumulate as pending and drain on recovery. **Alerted on** (§32.4) — a silent backlog is the failure mode that matters
Provider credentials	Configuration only (§35)

---

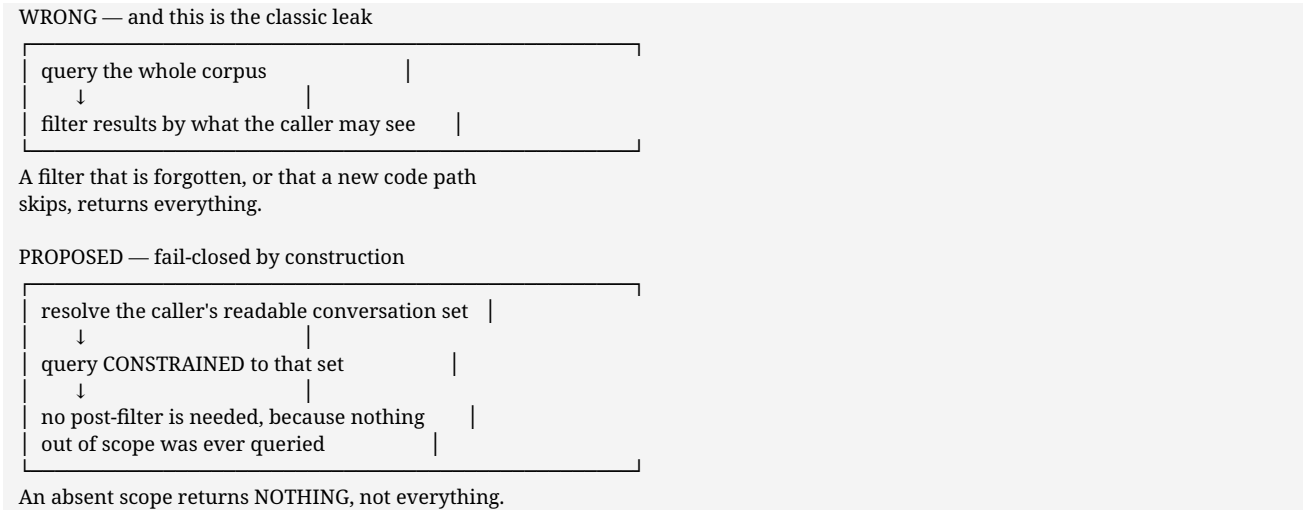
30. Search architecture

PROPOSED: MongoDB text indexes for V1. A dedicated search engine is **FUTURE** with a named trigger.

30.1 What is searched, and by whom

Search	Actor	Scope	V1
Employee directory	Employee	Visible directory (**D-11**)	✅
Conversation titles and participants	Employee	Conversations the caller may already read	✅
Message content	Employee	Same	✅
Customer identity (to find a thread)	Employee	Depends on **D-10** — what customer information an employee may see	⚠️ **D-10**
Own conversation history	Customer	Own threads only	❓ **D-08**
Cross-entity ("everything about X")	Employee	—	**FUTURE**

30.2 The architectural rule: scope before query



FR-SRCH-1 states this as a requirement. It is repeated here because it is the one thing in this section that must not be traded for performance: constraining the query is more expensive than filtering afterwards, and it is the correct expense.

30.3 Is MongoDB sufficient for V1? — an honest assessment

Requirement	MongoDB text index	Verdict
Find a message by a word or phrase, within scope	Text index, intersected with the scope set	**Sufficient**
Return within 2 s (**NFR-PRF-6**) on the V1 corpus	One pilot department plus internal chat is a small corpus	**Sufficient**
Prefix matching for directory lookup	Anchored regex on an indexed field, or a text index	**Sufficient**
Stemming and stop-words	Supported, language-configured	**Adequate**
Relevance ranking	A basic text score only	**Weaker.** Acceptable when results are scope-limited and few
Fuzzy matching / typo tolerance	Not supported	**Gap.** Acceptable in V1; would matter for customer-facing search
Highlighting matched terms	Not provided; would be done in the application	**Acceptable**
Faceting and aggregation across entities	Awkward	**Gap** — and it is a **FUTURE** requirement, not a V1 one

Conclusion: sufficient for V1, with two acknowledged gaps — relevance ranking and typo tolerance — neither of which any Part I use case requires. **UC-E09** asks that the searcher find what they were entitled to find, not that results be optimally ranked.

30.4 The trigger for a dedicated engine — FUTURE

Introduce Elasticsearch or OpenSearch when **any** of these becomes true, and not before:

1. Search latency exceeds **NFR-PRF-6** on a corpus that indexing cannot fix.
2. Relevance ranking becomes a stated requirement — most likely when customer-facing search arrives (**D-08**).
3. Typo tolerance becomes a stated requirement.
4. Cross-entity search is required (§42).

The cost to state alongside it: a second datastore to operate, back up and secure; an indexing pipeline that can fall behind; and — most importantly — **the scope rule of §30.2 must be reproduced inside it.** A search index that does not enforce scope is a complete copy of every conversation with the authorization removed. That is why this is deferred rather than pre-built.

30.5 Indexing and audit

Concern	Position
Index maintenance	Native to the collection; no separate pipeline in V1
Internal notes in employee search	Included — staff participants may read them
Internal notes in customer search	**Excluded absolutely**, if **D-08** admits customer search at all
Search auditing	**FR-SRCH-3** — audited **with the term**. "Someone searched" answers nothing; search is an exfiltration tool as much as a feature
Very short terms	Refused rather than returning everything (**FR-SRCH-5**)
Empty results	"No matches", never "no data" (**FR-SRCH-4**)
Rate limiting	Yes (§27.5) — search is expensive and is the natural bulk-extraction surface

---

31. Audit architecture

PROPOSED. After an incident the question is never "what does the data say now" — it is "who did what, when". This section exists to make that answerable without engineering assistance.

31.1 What is audited

Category	Events
Authentication	Sign-in success and failure · customer verification and re-verification · session revocation
Authorization	Role assigned or removed · scope granted or expired · a **refused** privileged attempt
Ownership	Assignment · claim · transfer (with reason) · escalation (with reason) · cover · reassignment on departure
Participation	Participant added or removed — it changes who may read prior history
Privileged reads	A non-participant reading a customer conversation under a scoped

Category	Events
	grant · a lead's oversight read (**D-11**) · a grievance officer reading escalated history
Attachments	**Download**, not upload (**FR-ATT-5**)
Search	Every search, **with the term** (**FR-SRCH-3**)
Administration	Account created, modified, deactivated · configuration change affecting access
Customer access	Customer reading their own history (**UC-C04**) · account closure
Lifecycle	Resolve (with outcome) · reopen · close

31.2 What is deliberately not audited

Not audited	Why
Ordinary internal message sends	The message *is* the record. Auditing every internal message is surveillance, not accountability (**P-06**, **P-08**)
Reading a conversation you participate in	Participation is already recorded; the read adds nothing
Typing, unread counts, read receipts	Ephemeral or personal state with no accountability value
Session expiry	A clock, not an act
Attachment upload	Already attributable through its message

The line being drawn: audit records the exercise of *authority* — access beyond ordinary participation, and changes to who holds access. It does not record ordinary work.

31.3 Event structure — conceptual

Field	Purpose
Event id	Unique, server-assigned
Timestamp	Server clock, UTC
Actor	The principal who acted
Actor kind	Employee, customer, service, system
Action	A closed vocabulary, not free text — a set that can be enumerated and diffed against §25
Target	The entity acted upon: conversation, principal, attachment, role
Target kind	—
Outcome	Succeeded or refused. **Refusals matter most** — a burst of refused privileged reads is what an attempt looks like
Reason	Where the action requires one (transfer, escalation, resolution)
Correlation id	Links to the request context record, and to related events in one request
Detail	A bounded, structured payload. **Never message content**

Message content never enters the audit ledger. The ledger says *who read what*, not *what it said* — the message store already holds the content, and duplicating it into a record with different retention creates two copies with two policies.

31.4 Two stores — the split that makes retention tractable

AUDIT LEDGER	AUDIT REQUEST CONTEXT
actor · action · target	network address
outcome · reason · time	user agent

```
correlation id                  correlation id

APPEND-ONLY                     SEPARATE COLLECTION
no update path in the application   OWN, SHORTER RETENTION
no delete path in the application   store-level automatic expiry
retained per D-06               proposal: 180 days
    |                               |
    └──────────── correlation id ─────────────┘
                  1:1, optional

THE LEDGER REMAINS MEANINGFUL AFTER THE CONTEXT EXPIRES.
"Who did what, when" survives. "From which address" does not.
```

Why this split is architectural rather than cosmetic. A retention obligation says *delete identifying data after N days*. An audit obligation says *the ledger is immutable*. In one record those are in direct conflict and one of them gets quietly broken — usually the immutability, by a script nobody documents. Split, both are satisfiable, and **D-06** can be answered either way without redesign.

31.5 Immutability, access and failure

Concern	Position
Immutability	No application code path updates or deletes a ledger record. Enforced by the audit module owning the collection exclusively (§18.2) and exposing append and query only
Database-level enforcement	**PROPOSED**: a write-restricted database role for the application, so immutability does not rest on application code alone. Requires the deployment decision in §37.1
Access	A restricted compliance role (**FR-AUD-6**). **Never exposed through ordinary conversation interfaces** (**FR-AUD-7**)
Query patterns	By actor, by target, by action, by period — the four indexes named in §24.4
Write failure	For a **security-critical** action, audit write failure **fails the action** (**FR-AUD-5**). An unaudited role assignment is worse than a refused one
Non-critical write failure	Logged as an operational error and alerted (§32.4). The action proceeds
Retention	**D-06** for the ledger; automatic expiry for the request context
Backup	Independent of conversation retention (**NFR-RCV-4**)
Audit of audit access	Reading the ledger is itself audited. Whoever investigates is also accountable

---

32. Observability

PROPOSED. StarLink must be able to answer "is it working" and "what went wrong" without attaching a debugger to production.

32.1 Operational logs versus audit — a boundary, not a preference

	Operational log	Audit ledger (§31)
Purpose	Diagnose the system	Establish accountability
Audience	Engineers	Compliance, Legal
Retention	Short — days to weeks	**D-06**, long
Mutability	Rotated and discarded freely	**Append-only**
Contains identity	Principal id only	Actor, target, outcome

	Operational log	Audit ledger (§31)
Contains message content	**Never**	**Never**
Where it lives	Log aggregation	Its own restricted store

They are never the same record. An audit event written only to the application log is not an audit trail — logs rotate. A privileged read written only to the log is unanswerable in three months.

32.2 Structured logging

Requirement	Detail
Format	Structured (JSON), one event per line — parseable by aggregation without regex
Correlation id	On **every** line of a request, and shared with the audit record for that request (§31.3)
Always present	Timestamp, level, correlation id, principal id where authenticated, operation, duration, outcome
Never present	Message content · attachment bytes or filenames · credentials, tokens, session values · customer contact details · full request bodies
Levels	Error (needs a human) · Warn (unexpected, handled) · Info (state change) · Debug (off in production)
Refusals	A 403 is logged at Info with the permission that was missing — a burst is a signal (§31.3)

The redaction rule is explicit because it is easy to violate: logging a whole request body for debugging is how message content and customer data end up in a log store with a different retention policy and a wider access list than the message store. **NFR-DAT-5** forbids it.

32.3 Metrics

Area	Metrics
API	Request rate, error rate, latency distribution per operation. Latency as percentiles — a mean hides the tail that **NFR-PRF-1** targets
Realtime	Connected clients, connect and disconnect rate, **reconnect rate**, rooms per connection, events published, publish failures
Database	Query latency, slow queries, connection pool utilisation, **documents examined versus returned**
Notification	Outbox depth, delivery attempts, successes, retries, **dead-lettered count**
Storage	Upload and download counts, rejected uploads by reason, scan failures
Business-operational	Queue depth, conversations waiting beyond the standard, unassigned count, conversations owned by inactive principals

Two of these are unusual and deserve explanation.

Documents examined versus returned is monitored because §38 records the measured failure it detects: 20,000 documents examined per page instead of 301 when a compound index is missing. A ratio drifting away from ~1 means an index has been lost or a query has changed shape — it is the earliest possible warning.

Conversations owned by inactive principals is monitored because **BR-13** says ownership by a deactivated account is invalid. This metric should be **zero**. A non-zero value is unreachable customer work, which is §32.4's highest-priority alert.

32.4 Alerting — what a human must be told about

Alert	Threshold	Why it matters
Conversations owned by an inactive principal	**> 0**	Unreachable customer work (**BR-13**)
Customer waiting beyond the service standard	Business-defined	The product's core promise
Notification dead-letter count rising	Any sustained rise	Notifications silently not arriving
Outbox depth growing without draining	Sustained	Provider outage or a stuck worker
Audit write failures	**> 0**	Accountability is degraded (**FR-AUD-5**)
Authentication failure rate spike	Above baseline	Credential attack (§27.5)
Refused privileged-read spike	Above baseline	Someone probing for access
Realtime reconnect storm	Above baseline	Instability, or a deploy going wrong
Documents-examined ratio degrading	Sustained	A lost index
Database replica health	Any degradation	§34.1
Error rate or latency breaching NFR targets	**NFR-PRF-1/2/3**	—

Deliberately not alerted: individual 403s (expected, and logged) · individual notification retries (designed for) · realtime disconnections (normal on mobile) · unread counts.

32.5 Health checks

Check	Reports	Used by
Liveness	The process is running	Restart supervision
Readiness	Database reachable, configuration valid, storage reachable	Whether to route traffic here
Dependency	Per-dependency status, degraded rather than binary	Operators and the status view

Readiness must fail if configuration is invalid, not just if the database is down. A process running with a missing secret and serving errors is worse than one that refuses to accept traffic — this pairs with the production refusal in §35.3.

32.6 Error tracking and tracing

Concern	Position
Unhandled errors	Captured with correlation id and stack, **grouped** so one incident is one entry, not ten thousand
Redaction	The same rules as §32.2 apply to error payloads — an error report is a log with wider distribution
Client-side errors	Captured from both surfaces, tagged by surface
Distributed tracing	**FUTURE.** One process means a correlation id in structured logs answers the same questions. It becomes valuable at the multi-process step in §33.2
Provider	**Open** — self-hosted or managed, a **PRODUCTION DECISION** (§37.1)

---

33. Scalability and performance

PROPOSED. The design point is roughly 200 employees growing to 500 (**NFR-SCL-1**). **Customer concurrency is unknown — D-13** — and this section refuses to guess it.

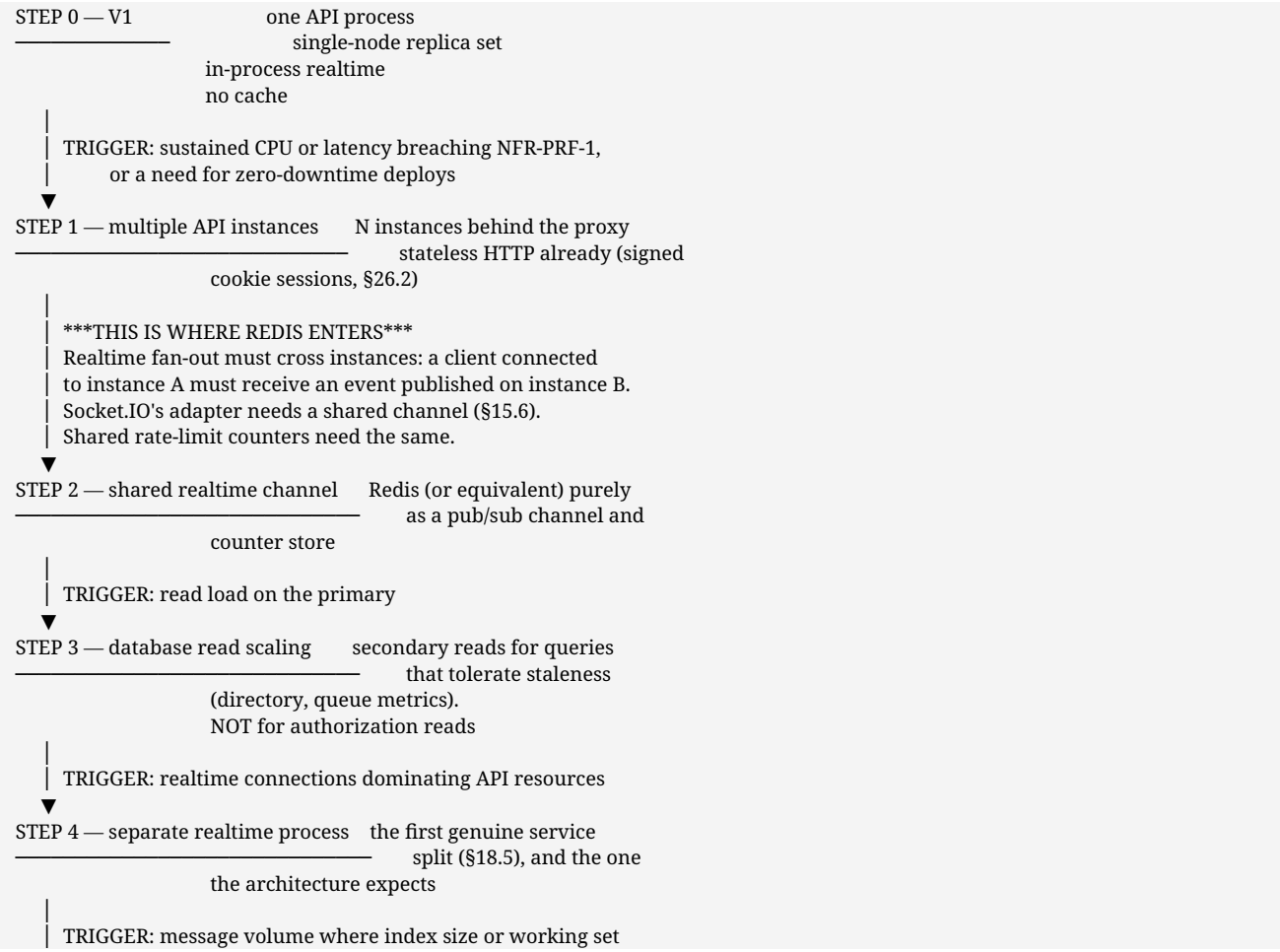
33.1 What V1 actually needs

Concern	V1 position	Why it is enough
API instances	**One** is sufficient	~200 employees, one pilot department. One process handles this comfortably
Database	**Single-node replica set**	Replica set is required for transactions, not for load
Realtime	**In-process**, single instance	Connection count is bounded by headcount
Cache	**None**	§14.2 — nothing to cache that a query does not serve fast enough
Background work	In-process scheduled workers (§18.6)	Small, idempotent jobs
Search	MongoDB text indexes (§30)	Small corpus

This is the honest V1 answer: almost nothing needs to scale yet. Stating that plainly is more useful than pre-building for load that may not arrive.

33.2 The scaling path, with triggers

Each step names what must become true before it is taken. **The order matters** — the realtime layer is expected to need attention first, and it is the step that introduces Redis.



```
   |          exceeds memory
   ▼
STEP 5 — archival tiering          cold conversations to cheaper
                                          storage, still readable.
                    BEFORE sharding, which is a
                    much larger commitment
```

Redis is not in V1 because Step 1 has not happened. When it does, Redis is required — not optional — and knowing that now is why §15.6 names Socket.IO's adapter dependency rather than discovering it under load.

33.3 The performance properties that are structural

These are not tuning; they are consequences of decisions already made, and losing them is a regression.

Property	Mechanism	Why it is structural
Bounded documents examined per message page	Compound index on conversation + time + record identity	§38's measured result: 301 examined versus 20,000. Monitored in §32.3
Cursor paging, never offset	Signed opaque cursor (**FR-MSG-3/4**)	Offset paging degrades linearly and is wrong under concurrent insertion
One read for a conversation and its participants	Participants nested (§15.5)	Every authorization decision needs both at once
Read state not written per scroll	Debounced marking (**FR-READ-4**)	Otherwise the busiest thread generates the most writes
Realtime carries identifiers, not bodies	**FR-RT-4**	Bounds event size regardless of message size
Notification delivery off the write path	Outbox (§29.4)	A slow provider cannot slow a message

33.4 Where the architecture would strain first

Named honestly, so nobody is surprised.

Pressure point	Symptom	Response
Realtime connections	Memory and file descriptors on one process	Step 2, then Step 4
Queue view with ownership	No join available (§15.5 trade-off)	Controlled denormalisation (§24.5); a materialised view if it stops sufficing
Search on a growing corpus	Latency breaching **NFR-PRF-6**	§30.4 trigger
Attachment throughput	Bytes streaming through the application (§28.4)	The scoped time-limited URL noted there, with its audit cost accepted
Unbounded conversation growth	A single thread with very many messages	Paging already bounds reads; archival is Step 5
Fan-out to a large announcement audience	One publish, ~200 recipients	Fine at this size. Batched publish if the company grows substantially

33.5 Not required for V1 — stated so it is not built by reflex

Horizontal API scaling · Redis · read replicas · sharding · a CDN for application assets (static asset caching at the proxy suffices) · a dedicated search engine · a message broker · distributed tracing · container orchestration · multi-region anything.

Every one of these has a trigger above or in §14.2. None is refused permanently; none is justified now.

---

34. Failure and resilience

PROPOSED. Architectural behaviour under failure, not implementation. The governing rule throughout: **degrade visibly, never silently.** A user who is told the system is degraded can act; a user shown a successful-looking screen that did nothing cannot.

34.1 The database is unavailable

Effect	Nothing works. Conversations cannot be read or written; authorization cannot be resolved
Behaviour	Readiness fails (§32.5), so the instance stops receiving traffic. Clients see an explicit service-unavailable state, **never an empty conversation list**
The rule that matters	**An empty result and a failed query must never render identically.** A customer shown "no conversations" when the database is down concludes their history was deleted
Realtime	Connections are closed rather than held open against a dead backend, so clients enter their normal reconnect path
Recovery	Clients re-fetch. No local state was authoritative (§19.5), so there is nothing to reconcile
Mitigation	Replica set with automatic failover; connection retry with backoff; monitored replica health (§32.4)

34.2 The realtime connection drops

Effect	Live updates stop. **Nothing else**
Behaviour	A quiet degraded indicator. **Input stays enabled** — sending continues over HTTP (**NFR-AVL-2**)
Recovery	Reconnect with backoff, then **re-fetch and resume**. No replay, no event log (§19.5)
Why this is cheap	Realtime is additive (**FR-RT-1**). No state exists only in an event, so a missed event costs latency, not correctness
Server side	Connections for an ended session are closed **server-side** (§27.1) — a socket outliving its session is an authorization hole, not a convenience

34.3 A notification provider fails

Effect	External notifications are delayed. **Messages are unaffected**
Behaviour	Outbox rows stay pending and retry with backoff (§29.4). In-app notification and unread counts are unaffected because they are not provider-dependent
Visibility	Outbox depth and dead-letter count are alerted on (§32.4). **A silent notification backlog is the real failure mode** — worse than the outage
Permanent failure	Dead-lettered, principal flagged for administrative attention, not retried indefinitely
Why it cannot cascade	**P-05** — the record was written before delivery was attempted

34.4 Object storage fails

Effect	Attachments cannot be uploaded or downloaded. **Conversations continue to work**
Upload	Fails explicitly, before the message is sent. The user keeps their message and can retry — **a message is never sent claiming an attachment that does not exist**
Download	Explicit error naming the attachment as temporarily unavailable, not a broken image or a silent blank
Metadata	Metadata lives in the database, so the conversation still shows *that* a file exists — its absence is legible rather than mysterious
Recovery	Retry. Unbound uploads expire (§28.6)

34.5 An employee disconnects or goes offline

Effect	They stop receiving live events
Behaviour	Unread counts accrue server-side. External notification fires if they remain unconnected past the threshold (§29.6)
Owned conversations	**Unaffected. Ownership is not presence.** A disconnected owner still owns their conversations — cover is triggered by *unavailability policy* (**D-05**), never by a dropped socket
Why that distinction is load-bearing	Treating a lost connection as unavailability would reassign conversations every time someone's network hiccups

34.6 A customer session expires

Effect	Thread unreadable, composer disabled (**UC-C05**)
Behaviour	The screen says **the session ended** — never implying the conversation is gone. Re-verification restores the **prior scope and nothing more**
In-flight draft	Held in the customer's own browser only, restored after re-verification, **never posted on their behalf**
Unread staff reply	Waits. The notification still went out
Realtime	Channel closed server-side at session end
Audit	Expiry not audited (a clock). **Re-verification audited** — a burst against one identity is what a takeover attempt looks like

34.7 A duplicate request arrives

Cause	A retry after a timeout where the first attempt actually succeeded — common on mobile
Message send	A client-supplied idempotency key; a repeat returns the **original** message rather than creating a second
Claim	Naturally idempotent — a single-document conditional update. The second attempt is told it is taken (**FR-ROUTE-3**)
Read marking	Idempotent by nature
Notification delivery	At-least-once; duplicates suppressed on a short window (§29.5). **A rare duplicate email is acceptable; a lost assignment notification**

	is not
Realtime events	Client discards events whose id it already holds (§19.5)

34.8 An assignment or ownership operation fails

Effect	The most consequential failure in the product, because a conversation with no owner is unanswered customer work
Behaviour	The operation is atomic at the ownership record. It either takes effect or it does not — **there is no partial assignment**
If the audit write fails	The operation **fails** (**FR-AUD-5**). An unattributable ownership change is not acceptable
If notification fails afterwards	The assignment stands; the notification retries (§34.3). The owner also sees it in their own list
The safety net	A conversation that ends up unassigned appears in the queue as **visibly waiting** (**FR-ROUTE-4**), and unassigned count is alerted on (§32.4). Nobody waits invisibly
On departure	**BR-13** — ownership by an inactive principal is invalid, and §32.3 monitors for it with a target of zero

34.9 Summary — the invariants that survive every failure above

1. A message acknowledged as sent **is durable**.
2. Nothing is delivered to anyone not entitled to it, even mid-failure.
3. **An empty result never looks like a failed query, and a failed query never looks like an empty result.**
4. No customer or conversation is left waiting **invisibly**.
5. The audit ledger never silently loses a security-critical event.
6. Losing realtime costs latency, never correctness.

---

35. Configuration and environment

PROPOSED. Configuration is where an independence claim is either kept or quietly broken, so it is stated as an architectural boundary rather than an operational detail.

35.1 The naming boundary

Every StarLink setting carries the `SL_` prefix. There is exactly one prefix and no fallback to any other product's variables.

This is not a style preference. A fallback chain — read `SL_MONGO_URL`, and if absent try something else — is precisely how a "standalone" system acquires a dependency nobody declared. One prefix, no fallback, and a build-time test that fails if a foreign prefix appears anywhere in the application.

35.2 Configuration surface — conceptual

Variable	Purpose	Default
SL_ENV	Environment name; drives production-only behaviour	dev
SL_MONGO_URL	StarLink's own database connection	Local replica set
SL_DB_IDENTITY	Identity database name	starlink_identity
SL_DB_CHAT	Conversation database name	starlink_chat
SL_DB_AUDIT	Audit database name	starlink_audit
SL_SESSION_SECRET	Session signing key	**None in production**
SL_CURSOR_SECRET	Paging cursor signing key (**FR-MSG-3**)	**None in production**
SL_API_PORT	API listen port	Development default
SL_WEB_EMPLOYEE_ORIGIN	Employee surface origin — CORS and cookie scope	Development default
SL_WEB_CUSTOMER_ORIGIN	Customer surface origin (§19.2)	Development default
SL_STORAGE_DRIVER	local or an object-storage driver	local
SL_STORAGE_BUCKET / SL_STORAGE_REGION / SL_STORAGE_KEY / SL_STORAGE_SECRET	Object storage, when the driver needs them	—
SL_STORAGE_MAX_BYTES	Upload ceiling (§28.2)	Configured
SL_NOTIFY_TRANSPORTS	Enabled adapters (§29.3)	inapp
SL_NOTIFY_EMAIL_*	Email adapter settings	—
SL_SCAN_ENDPOINT	Malware scanning, if **D-07** permits customer uploads	—
SL_AUDIT_CONTEXT_TTL_DAYS	Request-context retention (§31.4)	180
SL_RATE_LIMIT_*	Per-surface rate limits (§27.5)	Configured
SL_REALTIME_ENABLED	Allows realtime to be disabled outright	true
SL_LOG_LEVEL	Operational log level (§32.2)	info

SL_REALTIME_ENABLED exists to make **NFR-AVL-2** testable: the application must remain fully usable with realtime off, and a flag is how that gets exercised rather than asserted.

35.3 Rules

Rule	Reason
Secrets come from configuration only, never from source	**NFR-SEC-3**
Production refuses to start on a shipped development secret, on a secret below a minimum length, or on a database name outside starlink_*	A misconfigured production system that starts is worse than one that refuses
Every setting has a working development default **except secrets**	A clean checkout must start (§15.11); a shipped secret default would mean the variable could never legitimately be unset
Configuration differs between environments by **values only, never by code path**	An environment-conditional code path is a path that is never tested where it runs
Readiness fails on invalid configuration	§32.5
Configuration is validated **once at startup**, all problems reported together	An operator should not fix one setting per restart
No secret is logged, echoed by a health check, or included in an error report	§32.2

35.4 Proposed database naming

Database	Holds	Status
starlink_identity	Principals, organisation attributes, teams, roles, assignments	**PROPOSED**
starlink_chat	Conversations, participants, messages, read and view state, ownership records, queue, notifications, outbox, attachment metadata	**PROPOSED**
starlink_audit	The audit ledger and request context (§31.4)	**PROPOSED**

Why three rather than one. Identity is separated because it is the boundary the HRMS will eventually cross (§17.2) — a separate database makes the seam visible. Audit is separated because it wants different access control, different retention, and ideally a write-restricted database role (§31.5). Chat is everything else.

The runtime guard: the application refuses to open any database whose name does not begin starlink_. That single check is what makes "it cannot reach another product's data" a property rather than a promise.

35.5 The NorthStar prohibition, stated as configuration

StarLink must not depend on NorthStar runtime services, databases, secrets, environment variables, gateway, session, realtime hub, storage, or audit.

Prohibited	How configuration prevents it
Reading a NS_-prefixed variable	One prefix, no fallback (§35.1), verified by a build-time test
Connecting to a northstar_* database	The starlink_ runtime guard (§35.4)
Calling a NorthStar service	No NorthStar host or port appears in any configuration key
Sharing a session secret	SL_SESSION_SECRET is StarLink's own; a cookie signed elsewhere is **unreadable**, not merely rejected
Using NorthStar's storage or audit	Own drivers and own databases

Acceptance requirement: StarLink starts, authenticates, creates a conversation, sends and reads messages, delivers realtime, notifies and audits **with NorthStar completely stopped** — from a checkout in which NorthStar is not present at all. This belongs in the build, not in a reviewer's goodwill.

35.6 Environments

Environment	Character	Notes
Development	Single-node replica set, seeded data, local storage driver, in-app notification only, realtime on	**Never production data** (**NFR-DAT-6**)
Staging	Mirrors production topology, synthetic data	**Required before the V1-B customer pilot** (§45)
Production	Replicated database, object storage, TLS, real adapters	Refuses to start on a development default

---

36. Integrations and dependencies

36.1 Classification

STARLINK OWNED — built and operated as part of StarLink; no external party involved.

Component	Note
Application (all components of §17)	One deployable
Identity store	The V1 identity source of record
Conversation and message store	The core asset
Audit store	Separate and restricted
Realtime channel	In-process in V1 (§37.3)
Attachment storage	Local in development; object storage in production
Both web interfaces	Separate surfaces, shared foundation

SHARED INFRASTRUCTURE — company-provided, not StarLink-specific. StarLink must not assume exclusive use.

Item	Requirement
Hosting and network	Own ports and own namespace; no shared runtime with another platform
TLS termination and certificates	Required in production
Backup infrastructure	Must cover conversations, messages, **attachments** and audit (NFR-RCV-1)
Monitoring and log aggregation	StarLink emits; the platform collects
Company DNS	A distinct hostname per surface, if **D-16** is confirmed

EXTERNAL SYSTEM — third parties, each introducing availability, cost and compliance obligations.

System	Status	Introduced by
Customer messaging channel	**D-01 — UNDECIDED.** If WhatsApp: a third party holds conversation data, template rules and a service window apply, and a business-verified number is required	**D-01**
Email delivery	Needed for employee notification	§17.7
Push notification service	Needed if mobile push is required	**D-12**
Anti-virus scanning	Required if customer uploads are permitted	**D-07**
One-time code delivery (SMS)	Needed if verification uses a mobile code	**D-02**

FUTURE INTEGRATION — anticipated and designed for, not built in V1.

System	Direction	Prepared for by
HRMS	Inbound identity	The provider interface (§17.2) — one component changes
Customer / policy system	Inbound customer identity	**D-03**
CRM framework	Outbound conversation history	Stable principal and conversation identifiers
Compliance reporting	Outbound audit	Queryable audit (§27.6)
Single sign-on	Inbound employee authentication	Authentication separated from authorization (§26)

36.2 NorthStar appears on none of those lists

StarLink must never	Enforcement
Import NorthStar code	Boundary test in the build
Read NorthStar configuration	Own configuration namespace only
Connect to a NorthStar database	Refuses any database outside its own namespace
Call a NorthStar service	No NorthStar host or port in configuration
Use NorthStar's session, gateway, realtime hub, storage or audit	Own implementation of each (§17)

Acceptance requirement. StarLink must start, authenticate a user, create a conversation, send and read messages, deliver realtime, notify and audit **with NorthStar completely stopped.** This is testable, and the test belongs in the build rather than in a reviewer's goodwill.

36.3 Dependency risk

Dependency	Risk	Mitigation
Customer channel (**D-01**)	Highest in the document — third-party rules, cost, data residency	Adapter-based, and the decision is taken before intake is built
HRMS timing	StarLink may ship first	Provider interface; StarLink is its own source of record in V1
Customer identity source (**D-03**)	StarLink could become the customer master by default	Raised as a decision rather than allowed to happen
Anti-virus scanning (**D-07**)	Recurring cost	Feature existence is a decision, not an assumption
Object storage	Availability	Attachments degrade independently of conversation

---

36.4 Integration technology, per classification

Integration	Proposed mechanism	Status
Email notification	Adapter over a transactional provider; provider unnamed	**PROPOSED**, V1-A
Push notification	Adapter; requires a service and mobile decision	**FUTURE**
Customer messaging channel	Adapter. **If WhatsApp, an inbound webhook receiver as a separate process** (§37.2)	**DEPENDS ON D-01**
Malware scanning	Adapter over a scanning service or sidecar	**DEPENDS ON D-07**
One-time code delivery	Adapter over an SMS or email provider	**DEPENDS ON D-02**
HRMS identity	A second implementation of the §17.2 provider interface	**FUTURE**, prepared for
Customer / policy system	Read-only adapter behind the Customer module	**DEPENDS ON D-03**
CRM framework	Outbound: conversation history by stable identifier	**FUTURE**
Single sign-on	Replaces the authentication step only; authorization is untouched (§26)	**FUTURE**
Object storage	Driver interface (§28)	**PROPOSED**
Log aggregation, metrics, error tracking	Standard emission; platform collects (§32)	**PROPOSED** signals, **OPEN** tooling

Every external integration is an adapter behind an interface StarLink owns. No provider name appears anywhere in the domain modules — which is what allows **D-01**, **D-02**, **D-03** and **D-07** to be answered without touching conversation logic.

---

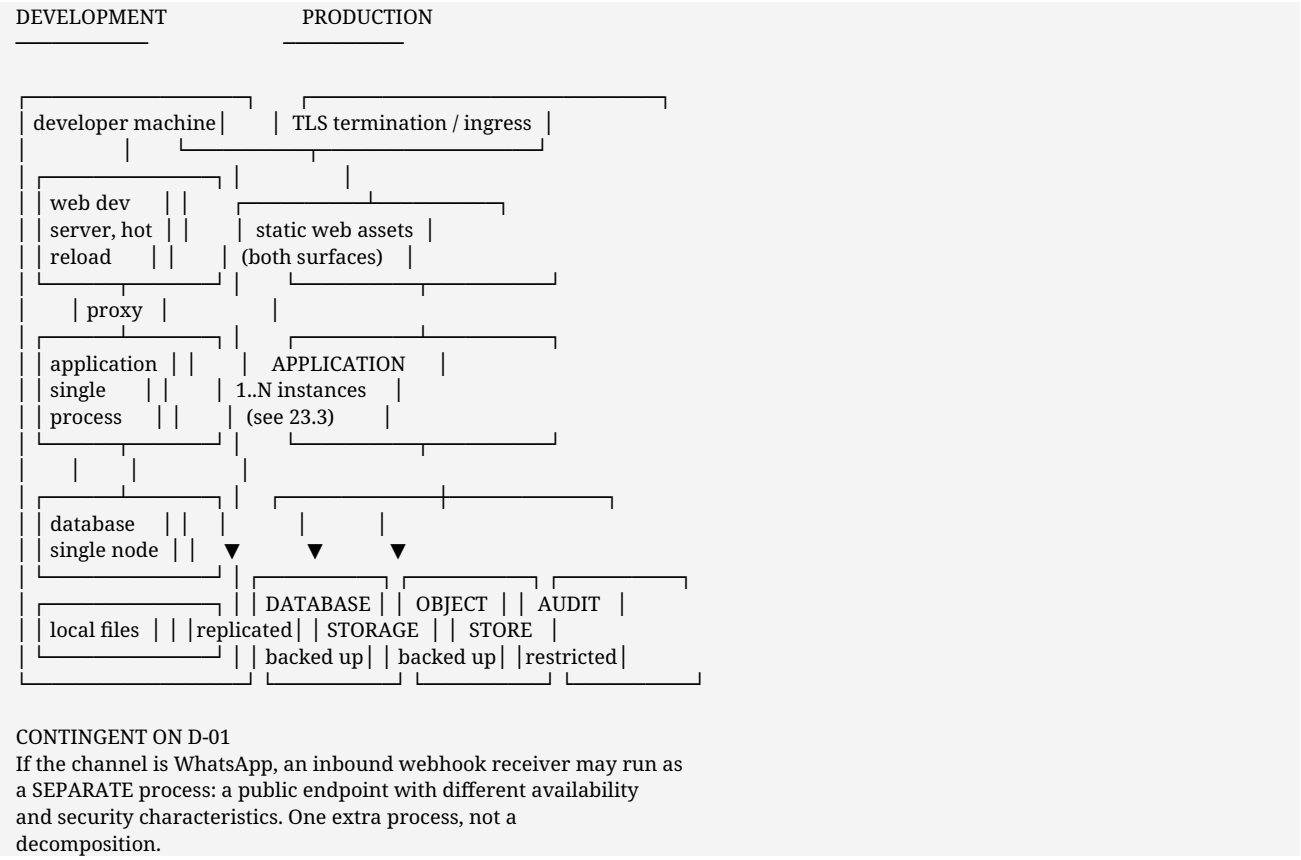
37. Deployment architecture

37.1 Three concerns, deliberately separated

Conflating these is how "we run it this way locally" becomes an accidental production commitment.

Concern	Statement	Status
Architectural requirement	Own datastore namespace; no runtime dependency on another platform; realtime degradable; authorization before content on every path	**ARCHITECTURAL REQUIREMENT** — not negotiable without changing the product's guarantees
Proposed	One deployable with enforced module boundaries (**ADR-04**); reverse proxy in production; Docker packaging	**PROPOSED** — open to being overruled
Development convenience	How a developer runs it locally: ports, seeded data, local file storage, single-node database	**Not an architectural commitment**
Production deployment	Instance count, hosting, TLS termination, backup schedule, scaling trigger	**Requires infrastructure sign-off — outside this document**

37.2 Deployment view — diagram 10



37.3 Scaling path

Stated here so the first scaling decision is not made under pressure.

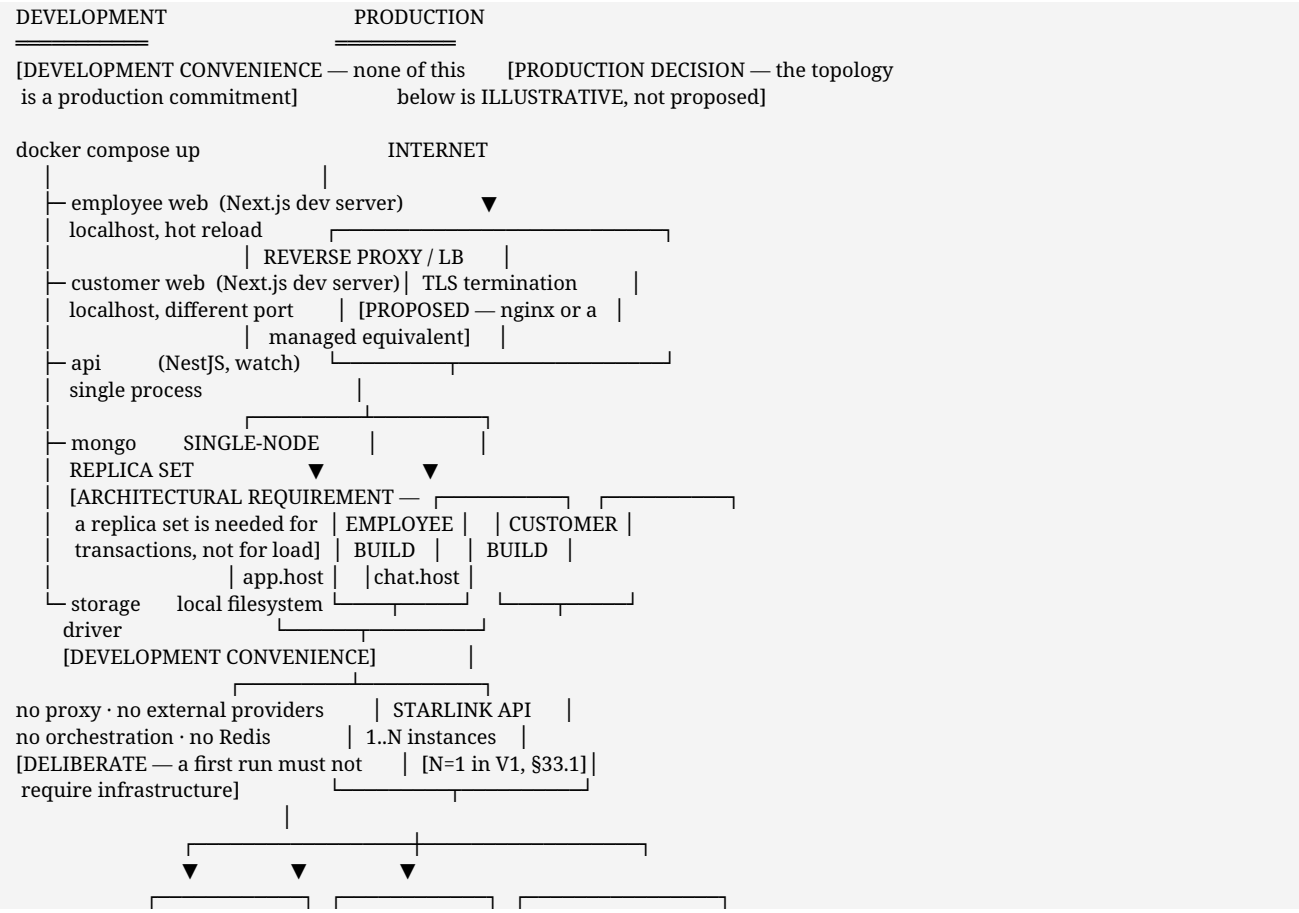
Trigger	Response
Read load rises	Multiple application instances behind the ingress
Realtime connections exceed one process	**The first component expected to need attention.** Move fan-out to a shared channel between instances; conversation and message handling are unaffected, because realtime is additive (§20.3)
Message volume grows	Indexing is already the design constraint (§24.4); archival tiering before any decomposition
Attachment volume grows	Object storage scales independently
Customer concurrency exceeds employee-scale assumptions	**D-13 is the missing input.** The architecture must not assume the two are comparable

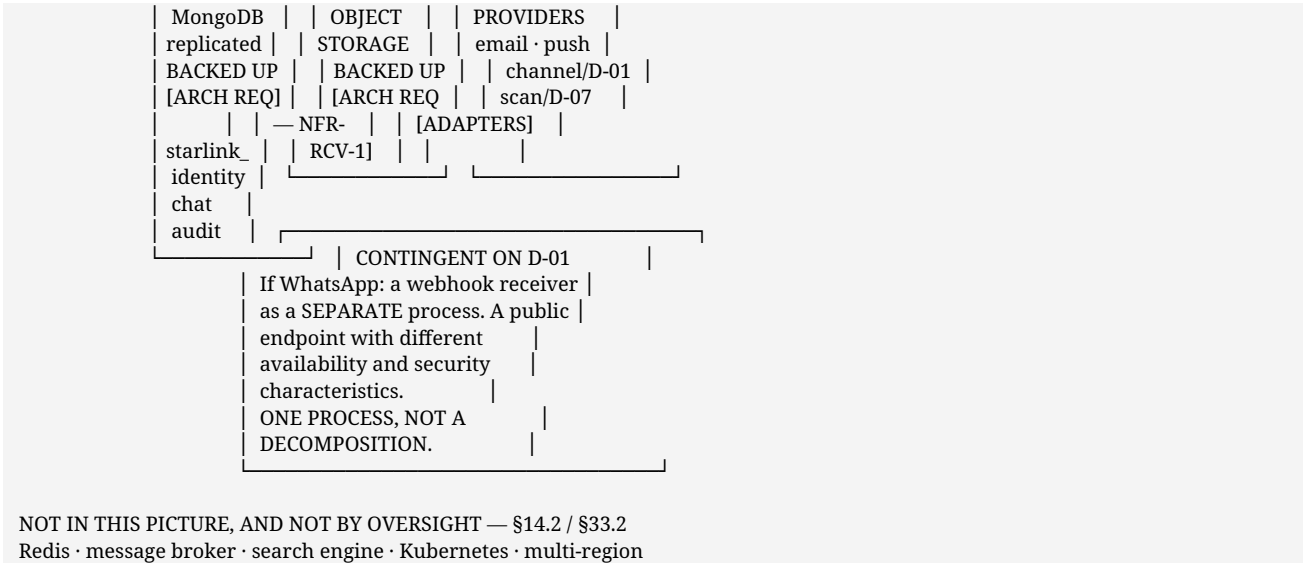
37.4 Environments

Development — single node, seeded, **never production data** (NFR-DAT-6) · staging — mirrors production topology, required before the V1-B customer pilot · production. Configuration differs by values only, never by code path, and production refuses to start on a development default (§27.7).

37.5 Deployment view with technology — diagram 14

Read the labels. They are the point of this diagram: the same picture contains architectural requirements, proposals, development conveniences and open production decisions, and conflating them is how a Compose file becomes a production commitment.





37.6 The four labels, applied

Label	Meaning	Examples in this document
ARCHITECTURAL REQUIREMENT	Cannot change without changing the product's guarantees	Own starlink_* databases refusing others · authorization before content on every path · realtime additive and degradable · backup covering attachments and audit · replica set for transactions · no runtime dependency on another platform
PROPOSED	Engineering recommends; the CTO may overrule	The whole of §14 · one deployable with enforced boundaries (**ADR-04**) · reverse proxy in production · Docker packaging · three database names
DEVELOPMENT CONVENIENCE	How a developer runs it. **Carries no production meaning**	Compose · dev-server ports · local storage driver · single-node database · seeded data
PRODUCTION DECISION	Genuinely open; outside this document's authority	Instance count · hosting · orchestration or not · hostnames · TLS certificate management · backup schedule and retention · observability tooling · scaling triggers being acted on

The trap this labelling exists to avoid: a development Compose file quietly becoming the production topology because nobody said it was not. **T-17 in §39 is marked OPEN for exactly this reason** — this document does not propose a production deployment model.

37.7 Release and rollback

Concern	Position	Label
Deployment artefact	A container image built once, promoted between environments	**PROPOSED**
Configuration	Injected per environment; values only, never code paths (§35.3)	**ARCHITECTURAL REQUIREMENT**
Zero-downtime deploy	Requires more than one instance — the trigger in §33.2 step 1	**PRODUCTION DECISION**
Database changes	Additive first; a shape that cannot be added is a §24.5 problem, decided before first write	**ARCHITECTURAL REQUIREMENT**
Rollback	The previous image, plus the rule that a release must not make a non-additive data	**PROPOSED**

Concern	Position	Label
	change it cannot reverse	
Realtime during deploy	Clients disconnect and reconnect with backoff and jitter (§20.9). **Reconnect storms are monitored** (§32.3)	**ARCHITECTURAL REQUIREMENT**
Staging	Mirrors production topology; **required before the V1-B customer pilot**	**ARCHITECTURAL REQUIREMENT**

---

38. NorthStar reference analysis

NorthStar is an existing internal platform with a working chat implementation. It was studied in depth. This section exists to show what that study produced, and to be explicit that a working implementation is not the same thing as a correct design for a different product.

38.1 Product and design lessons

Area	NorthStar approach	StarLink decision	Verdict	Reason
Session	Signed cookie, HttpOnly, SameSite-restricted	Same pattern, re-derived, plus **version-based revocation**	**KEEP + IMPROVE**	The pattern is sound; a version check makes revocation real rather than eventual
CSRF	Header-based defence on state-changing requests	Same approach	**KEEP**	Simple, effective, no token plumbing
Service routing	Explicit allow-list; unlisted routes refused	Same discipline for external adapters	**KEEP**	Fail-closed by construction
Message paging	Cursor over creation time and record identity, with a compound index	Same, with an **opaque signed cursor**	**KEEP + IMPROVE**	**Measured:** with the compound index, 301 records examined per page; without it, **20,000 examined and a blocking sort** — 184 ms first page and 1,219 ms cursor page, against 4 ms and 11 ms. Signing the cursor stops it becoming a client-supplied query
Conversation membership	Flat list of member identifiers	Participants with role, added-by, added-at, optional expiry, from the first record	**IMPROVE**	A flat list must be migrated before anyone can hold a role or a time-boxed view. Richer costs nothing at the start
Attachment shape	Two shapes, singular and plural, reconciled on every read	**One** shape, no alias	**IMPROVE**	The reconciliation is correct and permanent, because historical documents were never migrated. StarLink has no history to protect
Attachment URLs	Route strings stored on documents	Reference **derived on read**	**IMPROVE**	A stored route is a migration the first time the route moves
Realtime	In-process hub holding connections per user	Same for V1, plus **subscribe-time authorization and session-end invalidation**	**KEEP + IMPROVE**	Adequate at this scale; the missing piece was closing the channel server-side when the

Area	NorthStar approach	StarLink decision	Verdict	Reason
				session ends
Read state	Separate per-user read records	Same	**KEEP**	Correct separation of personal state from conversation state
Notifications	Stored first, delivered after	Same, promoted to an explicit principle (**P-05**)	**KEEP**	The right ordering, worth stating so it is not accidentally reversed
Audit	Present for some actions; **privileged reads not recorded**	Audit privileged reads, downloads and searches; **not** ordinary internal messages	**IMPROVE**	"Who read this customer's history" is the question an incident asks, and it was unanswerable
Audit retention	One record including identifying context	**Split**: append-only ledger plus separately-retained request context	**IMPROVE**	Resolves the direct conflict between a retention obligation and ledger immutability
Broadcast visibility	An everyone flag bypassing membership, consulting **no scope**	**Kind-scoped** visibility; customer principals denied explicitly (§26.3 step 4)	**REJECT**	In a product where customers are principals, this flag exposes company announcements to customers. The most dangerous pattern found
Authorization	Membership treated as authorization; a member effectively held broad read	**Permission → role → scope**; participation grants that conversation only	**REJECT**	This is the defect class that produces "any authenticated user can read anything". A prototype of the pattern was built during evaluation and a test caught precisely that — a non-participant reading a conversation successfully
Identity coupling	Chat assumed the platform's own identity model	Identity behind a **provider interface** (§17.2)	**REJECT**	Identity moves to the HRMS. A coupled design needs re-architecture; an interface needs a new adapter
Customer surface	None — an internal tool with no external audience	A separate customer surface, built from customer use cases	**NOT APPLICABLE**	NorthStar had no customers, so it offers no guidance here. This is the part of StarLink with no precedent to borrow, and therefore the part carrying the most design risk
Deployment	Multiple services behind a gateway	Single deployable with enforced module boundaries	**REJECT for V1**	The service split addressed a problem StarLink does not have, at a cost StarLink cannot yet absorb

38.2 Technical mapping — pattern by pattern

The table in §38.1 records product and design lessons. This one is narrower and more concrete: **the specific technical pattern, what StarLink does instead, and where in this document it lives.**

NorthStar pattern	StarLink equivalent	Verdict	Why	Where
Signed cookie session, no version check	Signed cookie **plus a version checked against**	**IMPROVE**	Revocation must be effective on the next	§26.2, ADR-06

NorthStar pattern	StarLink equivalent	Verdict	Why	Where
	the principal every request		request, not at a cache expiry	
Header-based CSRF defence	Same	**KEEP**	Simple, effective, no token plumbing	§27.1
Allow-list route proxy, unlisted refused	Same fail-closed discipline for external adapters	**KEEP**	Fail-closed by construction	§36.4
Cursor paging over `(created_at, _id)`	Same, **with an HMAC-signed opaque cursor**	**IMPROVE**	An unsigned cursor is a client-supplied database query	§24.7, **FR-MSG-3**
Compound index on conversation + time + id	Same, **and monitored**	**KEEP + IMPROVE**	The measured failure: **301 documents examined per page with it, 20,000 and a blocking sort without.** 4 ms versus 184 ms first page; 11 ms versus 1,219 ms cursor page. Now an alert, not a hope	§24.7, §32.3
Flat `member_ids` array	**Participants with role, added-by, added-at, optional expiry** from the first record	**IMPROVE**	A flat list must be migrated before anyone can hold a role or a time-boxed view. Richer costs nothing at the start	§24.5, **FR-CONV-2**
Dual attachment shape reconciled on every read	**One** shape, no alias	**IMPROVE**	The reconciliation is correct and permanent because history was never migrated. StarLink has no history to protect	§24.5
Attachment route strings stored on documents	Reference **derived on read**	**IMPROVE**	A stored route is a migration the first time the route moves	§28.3
In-process realtime hub, connections per user	Same shape for V1, **plus subscribe-time authorization and server-side close at session end**	**KEEP + IMPROVE**	Adequate at this scale. **The missing piece was closing the channel when the session ended** — an authorization hole	§20.10, §27.13
Realtime authorization implemented separately from HTTP	**One** session and permission path, shared by both	**REJECT**	Two implementations of an authorization check means two chances to get it wrong, and they diverge	§18.4, ADR-02
Separate per-user read-state records	Same	**KEEP**	Correct separation of personal state from conversation state	§24.1
Notification stored first, delivered after	Same, **promoted to a named principle**	**KEEP**	The right ordering; naming it stops it being accidentally reversed	**P-05**, §29.1
Audit present for some actions; **privileged reads not recorded**	Privileged reads, downloads and searches-with-terms audited; **ordinary internal messages not**	**IMPROVE**	"Who read this customer's history" is the question an incident asks, and it was unanswerable	§31.1, §31.2
Audit as one record including identifying context	**Split**: append-only ledger + separately-retained request context	**IMPROVE**	Retention and immutability held in one record conflict, and immutability is what quietly breaks	§31.4, ADR-11
`everyone` flag bypassing membership, consulting **no scope**	**Kind-scoped** visibility; customer principals denied **explicitly**	**REJECT**	In a product where customers are principals, this flag exposes company	§26.3 step 4

NorthStar pattern	StarLink equivalent	Verdict	Why	Where
			announcements to customers. **The most dangerous pattern found**	
Membership treated as authorization	**Permission → role → scope**; participation grants that conversation only	**REJECT**	The defect class that produces "any authenticated user can read anything". **An early StarLink prototype reproduced it and a test caught a non-participant reading a conversation**	§26.3, §27.9, ADR-07
Chat coupled to the platform's own identity model	Identity behind a **provider interface**	**REJECT**	Identity moves to the HRMS. A coupled design needs re-architecture; an interface needs a new adapter	§17.2, ADR-06
Multiple services behind a gateway	**One deployable, enforced module boundaries**	**REJECT for V1**	The split addressed a problem StarLink does not have, at a cost it cannot yet absorb	§18.5, ADR-04
Shared database across product areas	**Three own databases, and a runtime refusal of any other**	**IMPROVE**	Makes cross-product isolation a property rather than a promise	§35.4
No customer surface	A separate customer surface built from customer use cases	**NOT APPLICABLE**	NorthStar had no customers. **This is the part of StarLink with no precedent to borrow, and therefore the part carrying the most design risk**	§19.1, §19.2
Python / FastAPI implementation	TypeScript / NestJS **PROPOSED**	**NOT APPLICABLE**	A stack choice, not a lesson. §14.3 records that the prototype's **code does not transfer** — only its findings do	§14.3, §39.4

38.3 What is being claimed, and what is not

Claimed: NorthStar was studied at the level of its indexes, its authorization predicate and its realtime lifecycle — not skimmed. Most rows above are KEEP or IMPROVE, and the two most valuable improvements (the signed cursor and the audited privileged read) came from reading its code closely.

Not claimed: that NorthStar was built badly. The REJECT rows are places where a decision that was reasonable for an internal-only tool becomes unsafe once customers are principals in the same system. The `everyone` flag is the clearest example: harmless in a product with no external users, and a disclosure route in a product with them.

And the boundary restated: NorthStar is **REFERENCE ONLY**. It appears in this document in §38 and in the prohibition of §35.5, and nowhere else. StarLink takes no runtime dependency on its code, configuration, databases, services, session, realtime hub, storage or audit — verified by test, from a checkout in which NorthStar is not present (§35.5).

---

39. Technology decision register

Every row is PROPOSED. Nothing in this table has been approved. This is the technical counterpart to the business register in §44, and the two are kept separate on purpose: engineering may recommend technology, but it must not decide business questions.

39.1 The register

#	Decision	Proposed choice	Alternatives considered	Rationale	Status	Approval needed from
T-01	Implementation language	**TypeScript, both ends**	Python backend + TS frontend · Java/C# · untyped JS	The security-critical fields are *shapes* — an internal marker on a message, a role on a participant. One language declares them once (§15.1)	**PROPOSED**	CTO
T-02	Frontend framework	**Next.js**	Vite + React · Remix · two SPAs · server templates	Two isolated surfaces from one codebase (**D-16**); mobile-first cold entry for customers (§15.2)	**PROPOSED**	CTO
T-03	Styling and UI primitives	**Tailwind CSS + headless primitives**	Full component library · CSS modules · styled-components	**NFR-ACC-3** — the internal-note distinction must be layered, not colour-only, and identical across both surfaces (§15.3)	**PROPOSED**	CTO
T-04	Backend framework	**NestJS**	Express/Fastify · FastAPI · Go	Module boundaries enforced by wiring; one guard layer for **FR-AUTHZ-1**; realtime in the same module graph (§15.4)	**PROPOSED**	CTO
T-05	Backend shape	**Modular monolith**	Multiple services	No requirement demands independent scaling; three core operations are transactionally related (§18.5). **Downgraded from "Decided" in v1.0 to PROPOSED**	**PROPOSED**	CTO
T-06	Database	**MongoDB**	PostgreSQL · PostgreSQL + JSONB · two stores	Conversation-shaped reads, per-type message variation, append-only audit, proven paging. **Trade-**	**PROPOSED**	CTO

#	Decision	Proposed choice	Alternatives considered	Rationale	Status	Approval needed from
				offs stated in §15.5		
T-07	Realtime transport	**Socket.IO over WebSocket**	Raw WebSocket · SSE · long polling	Rooms map onto conversations, which is the authorization unit; reconnection matters on the mobile surface (§15.6)	**PROPOSED**	CTO
T-08	Cache / shared state	**None in V1. Redis is FUTURE — re-evaluated for the SLA and after-hours requirements, verdict unchanged (§23.9)**	Redis now	Nothing to hold: stateless sessions, single-process realtime, in-process counters. **Trigger: the multi-instance step, §33.2**	**PROPOSED**	CTO
T-09	Session and authentication	**StarLink-owned signed cookie, version-revocable**	External identity provider · server-side store · bare JWT · off-the-shelf RBAC	Revocation must be effective on the next request (**FR-AUTH-2**); no runtime third-party dependency to log in (§15.7)	**PROPOSED**	CTO
T-10	Authorization model	**In-house permission → role → scope**	RBAC library · policy engine (OPA)	Participation as a conversation-scoped grant is not a standard library shape, and it is the exact property §38 records as the reference platform's worst defect (§26.3)	**PROPOSED**	CTO + HR (roles: **D-11**)
T-11	Attachment storage	**Object-storage abstraction; local driver in dev, S3-compatible in production. Provider unnamed**	Name a provider now · bytes in the database · production filesystem	Bytes must never be publicly reachable (**FR-ATT-1**); hosting is unsettled (§15.8)	**PROPOSED**	CTO + infrastructure owner
T-12	Notification transport	**Adapter interface; in-app in V1, email next**	Commit to a provider · direct email integration · broker pipeline	The channel set depends on **D-01** and **D-12**, which are unanswered (§15.9)	**PROPOSED**	CTO, then business owner for channels
T-13	Message queue / broker	**None. Outbox pattern instead**	Kafka · RabbitMQ · managed queue	One consumer, bounded volume. The outbox provides the same at-least-once guarantee (§29.4)	**PROPOSED**	CTO
T-14	Search	**MongoDB text**	Elasticsearch/	Sufficient for	**PROPOSED**	CTO

#	Decision	Proposed choice	Alternatives considered	Rationale	Status	Approval needed from
		indexes. Dedicated engine FUTURE	OpenSearch now	V1's corpus; two acknowledged gaps that no use case requires (§30.3). **Trigger in §30.4**		
T-15	Reverse proxy / edge	**nginx or the platform's managed equivalent — production only**	TLS in the application · CDN in front	TLS (**NFR-SEC-4**) and two hostnames for two surfaces. **The role is proposed, not the product** (§15.10)	**PROPOSED**	Infrastructure owner
T-16	Packaging	**Docker; Compose for development**	Local runtimes · dev containers	A clean checkout must start, which is what makes the independence claim credible (§15.11)	**PROPOSED**	CTO
T-17	Production deployment topology	**Not proposed — a PRODUCTION DECISION**	Compose · orchestration · managed platform	Deliberately open. Compose is a development tool and this document does not propose it as a production topology (§37.1)	**OPEN**	Infrastructure owner + CTO
T-18	Observability tooling	**Structured logs + metrics + health checks. Provider open**	Managed APM · self-hosted stack	The *signals* are architectural (§32); the tooling is not	**PROPOSED** (signals) / **OPEN** (tooling)	Infrastructure owner
T-19	Container orchestration	**None in V1**	Kubernetes	One deployable, small team. Operational cost with no V1 benefit	**PROPOSED**	CTO
T-20	Malware scanning	**Required if D-07 permits customer uploads. FUTURE for employee uploads**	Scan everything from day one · never scan	Employee uploads are authenticated and attributable; customer uploads are neither to the same degree (§28.2). **A stated, deliberate gap**	**PROPOSED**	CTO + Claims (**D-07**)
T-21	Production identity provider	**StarLink's own store in V1; the production answer is TBD**	HRMS · single sign-on · managed identity provider	The provider interface (§17.2) means the answer changes one component and nothing else. **Deliberately unresolved** — see **D-29**	**OPEN**	CTO + IT

39.2 Where technology waits on a business decision

Nothing in §39.1 forces a business answer, but five rows cannot be finalised without one. This is the dependency direction the brief requires: **business decides, technology follows.**

Technology row	Waits on	What changes with the answer
T-12 notification adapters	**D-01** channel · **D-12** what customers are told	Which adapters exist. A WhatsApp answer also implies the separate webhook receiver in §37.2
T-09 session	**D-02** customer identification · **D-03** identity source	The customer authentication flow. The employee half is unaffected
T-10 authorization	**D-11** which roles exist	The role and permission set, not the model
T-11 storage · T-20 scanning	**D-07** customer attachments	Whether scanning is mandatory, and its recurring cost
T-14 search	**D-08** customer search	Whether customer-scoped search exists at all

39.3 What is genuinely open, not merely proposed

Three rows are marked **OPEN** rather than PROPOSED, because engineering has deliberately not formed a recommendation:

- **T-17 production topology** — depends on hosting, which is not settled.
- **T-18 observability tooling** — the signals matter; the vendor is an infrastructure preference.
- **T-15 proxy product** — the role is proposed; nginx versus a managed ingress is an infrastructure call.

39.4 If the CTO prefers a different stack

The architecture in Part II is **largely stack-independent, deliberately.** Module boundaries, the authorization model, the audit split, the outbox, scope-before-query, the additive-realtime rule and the storage abstraction all survive a change of language or framework. What would change is §18's enforcement mechanism, §19's rendering model, and §24's trade-off discussion.

The rows most worth arguing about, in order: **T-06 database** (the widest consequences), **T-01 language** (it sets T-02 and T-04), and **T-05 backend shape** (it sets the operational burden).

---

40. Architecture decision records

Concise records of the reasoning behind each major technical decision, so a future reader can tell whether a decision is still valid or merely still in place. **Every status is PROPOSED.**

ADR-01 · Frontend technology

Context. Two audiences with genuinely different needs (§19.1): a long-lived employee workload application, and a mobile-first customer surface entered cold, often for the first time. **D-16** proposes they be separate interfaces rather than one surface with role-based hiding.

Decision / proposal. Next.js with TypeScript and Tailwind, as one repository with two route groups and **two independent build and deploy targets** (§19.2). Customer code cannot import employee code, enforced by a build-time test.

Alternatives. Vite + React (the prototype's stack) · two separate repositories · one application with one build and route-level separation · a server-rendered template stack.

Rationale. Two builds mean the customer bundle **cannot leak what it does not contain** — employee components are absent, not hidden. One repository avoids duplicating the API client, generated types and design tokens, which for a small team is real recurring cost. Server rendering serves the customer's cold first paint; client routing serves the employee shell.

Trade-offs. A larger framework surface than strictly needed. The server-rendering model must be understood to be used safely — data fetched for a customer page is serialised into that page, which is why §19.3 forbids customer pages from calling shared operations that return fuller objects.

Status. PROPOSED. Depends on **D-16** for the two-surface premise.

ADR-02 · Backend framework and structure

Context. The architecture requires enforced module boundaries (§18.3) and a single uniform place to apply "authorize before content is read" (**FR-AUTHZ-1**). §38 records that the reference platform's realtime layer reimplemented authorization separately from HTTP, and that this was a defect.

Decision / proposal. NestJS, organised as edge → domain → platform with a one-directional dependency rule, and a build-time test that fails on a violation.

Alternatives. Express or Fastify directly · FastAPI · Go.

Rationale. An explicit module system with dependency injection makes a boundary violation a wiring error rather than a convention someone forgot. Guards give one authorization layer instead of per-endpoint checks. The WebSocket gateway lives in the same module graph, so realtime reuses the session and permission code rather than duplicating it.

Trade-offs. Opinionated and heavier than a bare router. Decorator-driven code can obscure control flow. More concepts before a new team member is productive.

Status. PROPOSED.

ADR-03 · Database

Context. The dominant access pattern is "one conversation, its participants, one page of its messages" (§24). Messages vary by type in a single stream. The audit ledger is append-only and nothing references it. Reporting in V1 is limited to queue depth and load (**FR-REP-1/2**).

Decision / proposal. MongoDB, with three databases — starlink_identity, starlink_chat, starlink_audit (§35.4) — and a replica set even on a single node.

Alternatives. PostgreSQL · PostgreSQL with JSONB for message bodies · a document store plus a relational store for reporting.

Rationale. Participants nested in the conversation means one read serves every authorization decision (§26.3). Per-type message variation needs no nullable column per variant. The paging pattern is proven on this exact shape — §38's measured 301-versus-20,000 result.

Trade-offs — the honest list. PostgreSQL would also work. It would give stronger multi-row atomicity for claim-and-assign and better ad-hoc reporting, at the cost of a join per authorization decision and a migration per message-type variation. MongoDB's costs: multi-document atomicity requires explicit transactions; no joins for the queue-plus-ownership view, so §24.5 accepts controlled denormalisation; text search weaker than a dedicated engine (§30.3); and schema discipline moves into the application, which is why §24.5 fixes the unfixable shapes before first write.

Status. PROPOSED. The row most worth arguing about (§39.4).

ADR-04 · Modular monolith versus microservices

Context. ~200 employees (**NFR-SCL-1**); customer volume unknown (**D-13**). A message and its audit record, an assignment and its notification, and a claim and its ownership record are each transactionally related.

Decision / proposal. A modular monolith — one deployable, boundaries enforced by test. Plus one contingent process: a webhook receiver if **D-01** answers WhatsApp (§37.2).

Alternatives. Six to ten services from the start · a service-per-bounded-context split · a monolith with no enforced boundaries.

Rationale. No Part I requirement demands independent scaling or availability of any module. Choosing a service topology against an unknown load is guessing. Module boundaries are cheap to move while the boundaries are still being learned — and they are.

Trade-offs. A boundary enforced by a test can be relaxed by editing the test; a service boundary cannot. Accepted knowingly: the ability to move a boundary cheaply is currently worth more than the guarantee.

Status. PROPOSED. *Downgraded from "Decided — architecture" in v1.0, at the CTO's instruction, because it is a technical judgement they should be free to overrule.* Revisit when a module develops genuinely different scaling characteristics — the realtime layer is the expected first candidate (§33.2 step 4).

ADR-05 · Realtime transport

Context. Six events justify live delivery and six do not (§20.2). Realtime must be **additive** — **NFR-AVL-2** requires the application to remain fully usable without it. Customers arrive on phones (**NFR-MOB-3**).

Decision / proposal. Socket.IO over WebSocket, rooms scoped to conversations, authorized at subscribe and **invalidated server-side at session end**.

Alternatives. Raw WebSocket · Server-Sent Events · long polling · no realtime in V1.

Rationale. Rooms map onto conversations, which is already the authorization unit, so "who receives this" becomes "who is joined", and the join is where the scope check goes. Automatic reconnection with backoff is provided rather than hand-built, and hand-built reconnection on a mobile customer surface is where subtle bugs live. Because realtime is additive, reconnection is a plain re-read rather than a replay protocol.

Trade-offs. A protocol layer above WebSocket and a required client library, so plain WebSocket clients cannot connect. **Multi-instance operation depends on an adapter — which is the Redis trigger in §33.2.** Stating that now rather than discovering it under load is the point of recording it here.

Status. PROPOSED.

ADR-06 · Session ownership

Context. FR-AUTH-2 requires revocation effective on the next request; **FR-EMP-2** requires deactivation to end access immediately. **A-13** expects the HRMS to become the identity source of record.

Decision / proposal. StarLink issues, verifies and revokes its own sessions — a signed cookie carrying principal and version, with the version checked against the principal on every request. HttpOnly, SameSite-restricted, Secure under TLS.

Alternatives. An external identity provider · a server-side session store · bare JWT with refresh tokens.

Rationale. The version check makes revocation real in one lookup. An external provider would be a runtime third-party dependency **for the ability to log in**, which is a poor trade for an internal tool, and it would compete with the HRMS. A server-side store adds a component (§14.2 shows nothing else needs one). Bare JWTs optimise away the revocation check, and revocation is a requirement.

Trade-offs. Security-critical code we own, so §45 Phase 3 makes a security review a gate. Customer authentication remains genuinely undecided (**D-02**, **D-03**) — this covers the employee half and the shape of the customer half.

Status. PROPOSED. Single sign-on remains the natural future answer (§36.1).

ADR-07 · Authorization model

Context. §38 records the reference platform's most serious defect: membership treated as authorization, so a member effectively held broad read. An early prototype of this product **reproduced it**, and an acceptance test caught a non-participant reading a conversation.

Decision / proposal. Permission → role → scope, evaluated in the order of §26.3, with **participation as a conversation-scoped grant and never a global one**. Unknown permission denied. Expiry read from the clock. Announcement visibility kind-scoped so a customer principal is denied explicitly.

Alternatives. An RBAC library · a policy engine such as OPA · membership-implies-access.

Rationale. The model's distinctive property — participation granting exactly one conversation — is not a standard library's shape, and bending one to fit would obscure the property that matters most. A policy engine is a second system to secure and reason about, for a permission set this size.

Trade-offs. In-house authorization code requires review discipline and thorough negative tests. The mitigation is that the decision lives in **one** function with one call pattern (§18.4 step 3), so it is small enough to review properly.

Status. PROPOSED. The role set depends on **D-11**; the model does not.

ADR-08 · Attachment storage

Context. Claims and grievance work is document work. **FR-ATT-1** requires authorization through the conversation, never through URL knowledge. **FR-ATT-5** makes **download** the audited event.

Decision / proposal. An **object-storage abstraction** — local driver in development, S3-compatible in production, **provider deliberately unnamed**. Opaque server-generated keys. Bytes streamed through the application after the conversation check.

Alternatives. Name a provider now · bytes in the database · a production filesystem · signed direct URLs.

Rationale. Streaming through the application is what allows steps 3, 4 and 5 of §28.4 — the conversation check, the internal-note check, and the download audit — all of which a signed URL handed to a client performs at issuance rather than at fetch. Opaque keys prevent enumeration and leak nothing.

Trade-offs. Bandwidth through the application. **A scoped, short-lived, single-object URL becomes acceptable as a FUTURE optimisation** if issued only after all checks pass and the issuance is what gets audited — a stated cost, not a silent one.

Status. PROPOSED. Customer uploads depend on **D-07**.

ADR-09 · Notification abstraction

Context. The channel set is not knowable: **D-01** decides how customers are reached and **D-12** what they are told. **P-05** requires the record written before delivery is attempted.

Decision / proposal. An **adapter interface** with a **database-backed outbox** and an asynchronous worker. In-app in V1; email next; everything else follows the decisions.

Alternatives. Commit to a provider now · direct integration to one service · a message broker.

Rationale. Adapters let in-app notification ship in V1-A while the channel questions remain open. The outbox provides at-least-once delivery with retry, survives restarts, and keeps external providers off the write path — which is exactly what a broker would provide, without a component to operate and back up.

Trade-offs. Adapters constrain to a common denominator, so a channel-specific capability (WhatsApp templates) needs a channel-specific path. At-least-once means a rare duplicate is possible — accepted, because a lost assignment notification is far worse than a duplicated one.

Status. PROPOSED. Broker trigger recorded in §29.4.

ADR-10 · Search

Context. UC-E09 requires the searcher to find what they were entitled to find. **FR-SRCH-1** requires scope resolved before query. **FR-SRCH-3** requires the term to be audited.

Decision / proposal. MongoDB text indexes for V1, with the query constrained to the caller's readable set rather than filtered afterwards. A dedicated engine is **FUTURE**.

Alternatives. Elasticsearch or OpenSearch now · database queries with post-filtering.

Rationale. Sufficient for V1's corpus with two acknowledged gaps — relevance ranking and typo tolerance — neither of which any use case requires. Post-filtering is rejected outright: a forgotten filter returns everything, whereas an absent scope returns nothing.

Trade-offs. Weaker ranking and no fuzzy matching. And the reason deferral is actively preferable: **a search index that does not reproduce the scope rule is a complete copy of every conversation with the authorization removed.** Building that before it is needed is taking on the risk early.

Status. PROPOSED. Triggers in §30.4.

ADR-11 · Audit

Context. FR-AUD-1 requires an append-only ledger. **NFR-DAT-2/3** and **D-06** require retention per data class, possibly including a deletion obligation. §38 records that the reference platform did not audit privileged reads, which made "who read this customer's history" unanswerable.

Decision / proposal. Two stores — an append-only ledger of actor/action/target/outcome, and a **separately-retained** request-context collection holding network address and user agent with automatic expiry. Linked by correlation id. Privileged reads, attachment downloads and searches-with-terms are audited; **ordinary internal messages are not**. Audit write failure fails a security-critical action.

Alternatives. One record containing everything · audit in the application log · an external audit service.

Rationale. A retention obligation and an immutability obligation held in one record are in direct conflict, and one gets quietly broken — usually immutability, by an undocumented script. Split, both are satisfiable and **D-06** can be answered either way without redesign. Audit in the log is not an audit trail, because logs rotate.

Trade-offs. Two collections and a join for a full forensic picture. The ledger becomes less specific once context expires — accepted, and that is the deliberate design: *who did what* survives, *from which address* does not.

Status. PROPOSED. Retention periods are **D-06**.

ADR-12 · Deployment

Context. Hosting is not settled. **NFR-SEC-4** requires TLS in production. **NFR-RCV-1** requires backup covering attachments. §19.2 needs two hostnames. Independence must be demonstrable from a clean checkout.

Decision / proposal. Docker for development, with Compose bringing up API, both surfaces and a single-node replica set. **A reverse proxy in production only. Production topology is deliberately not proposed** — it is a PRODUCTION DECISION (§37.1).

Alternatives. Compose in production · container orchestration · a managed platform · locally installed runtimes for development.

Rationale. A clean checkout that starts in one command is what makes the independence claim credible rather than asserted. Keeping the proxy out of development means a developer's first run does not require it. Refusing to propose a production topology is deliberate: it depends on hosting decisions this document does not own, and quietly promoting a development Compose file into production architecture is a common and costly mistake.

Trade-offs. Resource overhead on developer machines; container file-watching can be slow, so the frontends may run on the host against a containerised API and database.

Status. PROPOSED for development. **OPEN** for production (**T-17**).

---

41. V1 scope

41.1 The definition

> ***V1 is a communication platform on which every CoverYou employee conducts internal conversation, and on which one department conducts customer conversation under named ownership, with full audit.***

Delivered in two phases so the risky half stays small.

41.2 V1-A — internal communication

Users	All employees
Delivers	Authentication, authorization, internal 1:1 and group conversation, directory, messaging with paging, attachments, unread state, scoped search, in-app notification, realtime, audit ledger, administration console
Blocked by	**Nothing.** No open decision blocks this phase — which is the entire reason the phasing exists
Proves	Identity, authorization, realtime, notification, storage and audit end to end, with real users and real load
Risk	Low. Internal audience, no external exposure, no customer data

41.3 V1-B — customer communication pilot

Users	One department, plus its customers
Delivers	Customer intake, identification, the customer surface, routing and queue, ownership and assignment, cover, multiple staff on a conversation, transfer and escalation, internal notes, lifecycle, queue and load view
Blocked by	**D-01, D-02, D-03, D-04** and — for the document to be valid at all — **D-14**
Pilot department	**Support (18)** or **Claims (4)**
Explicitly not	**Fresh Sales (63).** Piloting the riskiest surface on the largest team is how a pilot becomes an incident
Proves	The customer boundary, the routing model, and the ownership model, at a scale where a mistake is recoverable
Risk	The real risk in the programme, which is why it is second and small

41.4 Company-wide rollout

Follows evidence from V1-B, department by department, largest last. Entry criteria in §45.

41.5 What V1 does not include

Everything in §12.2 (V1.1), §12.3 (needs a business decision), §12.4 (future) and §12.5 (out of scope). The list is not an apology — a bounded V1 that ships is worth more than a complete V1 that does not.

---

42. Future roadmap

High level only. Each item requires its own justification when it is proposed; appearing here is not approval.

Horizon	Candidate	Trigger
Near	Read receipts, typing, reply-to, archive and mute, external notification delivery, announcements	V1-A adoption
Near	Customer attachments and customer search	**D-07**, **D-08** answered
Medium	HRMS identity adapter	HRMS reaches usable maturity
Medium	Customer or policy system integration	**D-03** answered
Medium	Presence and availability	**Only** if **D-05** requires automatic routing around absence
Medium	Multi-brand separation	**D-09** answered yes
Longer	CRM framework integration — conversation history as a consumable asset	CRM framework exists
Longer	Compliance reporting surface	Retention policy settled (**D-06**)
Longer	Analytics beyond queue, load and SLA state — dashboards, trends, cross-team benchmarking	A named business question to answer. **SLA *state* is V1 (§23); SLA *reporting* is not.** Per-agent scorecards stay forbidden (**P-08**)
Longer	Realtime horizontal scaling	Connection count exceeds one process (§37.3)

Deliberately absent: message forwarding (the leak primitive), individual productivity analytics (**P-08**), microservice decomposition (no problem yet exists that it solves), and customer-to-customer contact (no business case, large abuse surface).

---

43. Assumptions

Every assumption is a risk held in trust. Each one below is either confirmable cheaply, or is a decision in §44 wearing different clothes.

ID	Assumption	If wrong	Confirm with
A-01	CoverYou is an insurance distribution business (sales, renewals, retention, claims, grievance)	§7, §11 and §21 need rework. **This is the assumption everything else rests on**	**D-14** — Project Head, two minutes
A-02	Roughly 147 of 195 staff are in customer-contact roles	The business case weakens but does not collapse	Department heads
A-03	Personas in §7 correspond to real functions	The permission model is built on the wrong shapes	**D-11** — CTO + HR
A-04	Customers are willing to converse in text with the company	The customer half has no audience	Business owner
A-05	A customer conversation should have one accountable owner	§11.3, §17.5 and §21.2 change fundamentally	**D-04** — CTO + department heads
A-06	Internal conversation needs no lifecycle	Minor — a state machine can be added	**D-15**
A-07	Staff will use internal notes rather than a side channel	Context fragments and the leak risk moves off-platform	Pilot observation
A-08	Reopen within a bounded window is acceptable to the business	Window length changes; the model does not	**D-08**
A-09	No legal requirement currently dictates retention periods	Retention design changes and may become urgent	**D-06** — Legal
A-10	Customer volume is within employee-scale concurrency	Sizing and possibly topology change	**D-13** — business owner
A-11	A single brand in V1	A tenancy boundary must be retrofitted, expensively	**D-09**
A-12	Staff have devices and connectivity adequate for a web application	Rollout needs a device programme	IT
A-13	The HRMS will eventually be the identity source of record	None — the provider interface holds either way	Already designed for (§17.2)
A-14	NorthStar continues to operate independently and is unaffected	None — StarLink takes no dependency on it (§36.2)	Verified by test

Technical assumptions — new in version 2.0

Every one of these is either cheaply testable or is a technology decision in §39 wearing different clothes.

ID	Assumption	If wrong	Confirm with
A-15	The team can build and maintain TypeScript on both ends	The stack recommendation changes; **§39.4 states which rows shift**	CTO — a team-capability question, not a technical one
A-16	~200 employees and one pilot	§33.2 step 1 arrives earlier,	Load test before V1-B

ID	Assumption	If wrong	Confirm with
	department fit comfortably on one API instance	which is a planned step rather than a redesign	
A-17	Customer concurrency is within employee-scale assumptions	Sizing and possibly topology change	**D-13**
A-18	MongoDB text search is adequate for V1's corpus	§30.4 trigger fires and a search engine is introduced	Measure during V1-A
A-19	A single-node replica set is acceptable in development	Development setup complicates	Verified during Phase 1
A-20	Object storage is available in production	Attachments need a different plan	Infrastructure owner
A-21	An email transport is available for employee notification	V1-A ships with in-app notification only	Infrastructure owner
A-22	The prototype's independence guards can be reproduced in the proposed stack	The independence claim needs a different proof	Phase 1 — the guards are a pattern, not code (§14.3)

---

44. Business decision register

The most important section in this document. Sixteen open items. Nothing here has been decided by engineering on the business's behalf.

Status values: **DECIDED** — settled, recorded for completeness · **PROPOSED** — engineering recommends, awaiting confirmation · **NEEDS DISCUSSION** — genuinely open · **NO PROPOSAL** — deliberately offered without a recommendation, because it is not engineering's to make.

44.1 BLOCKING — the customer half of V1 cannot proceed without these

None of these blocks V1-A. Internal communication can begin while they are resolved.

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
D-14	Is the business context (§3) correct?	Insurance distribution, **inferred**	Confirm · correct · reframe	Answer this first — it validates or invalidates the whole document	If wrong, §7, §11 and §21 are rework	Project Head	**NEEDS CONFIRMATION**
D-01	Which channel do customers reach us on?	**None offered**	(a) web chat on the CoverYou site · (b) WhatsApp Business · (c) both · (d) add email	**No recommendation.** It changes identity, notification, storage, delivery semantics, retention and compliance simultaneously. WhatsApp brings template rules, a service window, a verified	Highest in the document. Rebuilding intake, identity and notification if changed later	CTO + business owner	**NO PROPOSAL**

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
				number and a third party holding conversation data			
D-02	How are customers identified?	Verified session	(a) full accounts with passwords · (b) verified session: policy number or mobile plus a one-time code · (c) magic link · (d) channel-native identity	(b). Policyholders will not maintain a password for a chat, and (b) still yields a verified identity to attach history to	Moderate — contained behind the identity boundary if settled before build	CTO + business owner	**PROPOSED**
D-03	Where does customer identity come from?	—	(a) an existing customer or policy system · (b) StarLink becomes the customer master · (c) minimal record, verified per session	Prefer (a) or (c). **(b) by default would be a large and probably wrong commitment** — leadership has recorded Customer and Policy as entities absent from every system	Determines whether StarLink acquires a data-ownership obligation it was not scoped for	CTO	**NEEDS DISCUSSION**
D-04	Direct, team, queue, or hybrid routing?	Hybrid (§21.2)	(a) direct to a named person · (b) team-assigned · (c) pure queue · (d) hybrid	(d). The departments are different shapes and one model is wrong for most of them. Fallback: (b)	High — it is the routing engine, and ownership, queues and presence all follow from it	CTO + heads of Sales, Support, Claims	**PROPOSED**
D-04a	May a non-owner reply to the customer?	No, by default	(a) owner only · (b) any staff participant · (c) lead override	(a) with (c). Two staff voices in one thread reads as disorganised and makes accountability ambiguous	Moderate — affects the composer and the permission set	CTO + department heads	**PROPOSED**

44.2 IMPORTANT — required before the customer surface ships

Two of these have external lead times. **D-06** needs Legal and **D-11** needs HR, so start them early even though they block no code today.

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
D-05	Who owns a conversation when the advisor is unavailable?	—	(a) auto-return to queue after a threshold · (b) a named backup per	Prefer (b) or (c) — both avoid a presence system entirely. Only	Affects Renewals most, where the relationship is the point	Department heads	**NEEDS DISCUSSION**

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
			advisor · (c) the lead reassigns manually	(a) requires availability tracking (§20.2)			
D-06	Retention and deletion	—	Per data class: conversations, attachments, audit ledger, identifying request context	Engineering has split the ledger from identifying context (§27.6) so any policy is implementable. **We assert no legal requirement — this needs Legal**	A deletion obligation interacting with an append-only ledger is a design constraint, not a setting	Legal + CTO	**NEEDS DISCUSSION — POLICY**
D-11	Which roles exist, and what may a manager read?	No default read	(a) leads read their team by default · (b) no default read; scoped grant, audited	(b). Default read is convenient and is the **widest privacy surface in the product**	Determines the permission model and the privacy posture	CTO + HR	**NEEDS DISCUSSION**
D-10	What customer information should an employee see?	—	(a) name and contact only · (b) plus policies and claims · (c) full history	Depends on **D-03.** More context means better service and a wider breach surface; the trade is the business's	Widens or narrows the breach surface	Business owner + Legal	**NEEDS DISCUSSION**
D-09	Multi-brand / multi-tenant?	Single brand in V1	(a) single brand · (b) tenancy boundary from the start	Cheap to design for now, expensive to retrofit. A second brand appears in the directory, so the question is real	If retrofitted, it is a data-model migration	CTO	**NEEDS DISCUSSION**
D-13	Expected customer volume	—	Conversations per day, concurrent customers, peak	**Not a decision — an input.** Every sizing statement in §13 is estimation without it	Sizing, and possibly topology (§37.3)	Business owner	**NEEDS AN ESTIMATE**
D-15	Do internal conversations need a lifecycle?	No (§21.4)	(a) none · (b) resolve/close on internal threads too	(a). Imposed lifecycle on internal chat produces "closed" threads people keep talking in, and a status nobody maintains. **Buildable either way —**	Low	CTO	**PROPOSED**

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
				this is a product call, not a constraint			
D-16	Is the customer surface a separate interface?	Yes (§6.1)	(a) separate interface · (b) one surface with role-based hiding	(a). The cost is real — two interfaces to build and maintain — and the recommendation still stands: internal notes are what would be hidden, and in a single-surface design every leak is one missed conditional away	Moderate build cost against a materially lower leak risk	CTO	**PROPOSED**

44.3 CAN BE DECIDED LATER — deferrable without blocking design or build

Each maps to a row in §12.3. None changes the data model.

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
D-07	Customer attachments	—	(a) not permitted · (b) permitted with limits and scanning · (c) permitted, claims only	Claims plainly wants them. If permitted, **AV scanning is mandatory and is a recurring cost** — that cost is part of this decision	Enables or blocks document-driven claims work	CTO + Claims	**NEEDS DISCUSSION**
D-08	Customer history, search, reopen window	Reopen 7 days	All past conversations or recent only · search own history · export · window length	7 days is **arbitrary until the business gives a service standard**. Export may be a compliance obligation rather than a feature	Customer experience, and possibly a compliance obligation	Business owner	**NEEDS DISCUSSION**
D-12	Notifications to customers	—	What, on what channel, opt-out or not	Entirely dependent on **D-01**	Customer experience, and cost per message on some channels	Business owner	**NEEDS DISCUSSION**

44.4 The five business decisions to settle in the meeting

Order	ID	Why it is in this position
1	**D-14**	Two minutes, and it validates or invalidates everything else. Ask it before the architecture
2	**D-01**	The largest unknown, and the only decision carrying no recommendation. Needs real time, not a passing answer
3	**D-02**	Shapes the data model and the isolation boundary
4	**D-03**	Decides whether StarLink becomes a customer master — a large commitment if made by default
5	**D-04**	The routing engine, and everything ownership-shaped downstream

44.5 New decisions — the customer journey, after-hours, SLA and the case model

Thirteen additions, **D-17 ... D-29**. Every one arises from the journey and operational requirements in §21, §22 and §23. **No SLA duration, working-hour range or category list is invented below** — the framework is proposed; the values are yours.

BLOCKING for the V1-B customer pilot

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
D-17	**What are the customer categories, and which team does each route to?**	Sales · Renewals · Existing policy/service · Claims · Grievance · Other — **illustrative only** (§21.6)	The list, the team mapping, and **which categories are relationship-shaped** (routing step 2, §21.8)	Start from the illustrative set, correct it against how the business actually divides work. **Keep "Other / not sure"** — without it a customer who cannot classify their problem picks wrongly, and a wrong category means wrong team and wrong clock	Determines routing for every customer conversation	Business owner + heads of Sales, Support, Claims	**PROPOSED**
D-18	**Category structure** — depth, and are sub-categories mandatory?	Two levels; sub-category optional	One level · two · deeper · mandatory vs optional	Two levels, sub-category **optional**. Every mandatory field is a place a customer abandons	Intake friction versus routing precision	Business owner	**PROPOSED**
D-20	**After-hours behaviour** — what is the customer told?	Acknowledge, queue, promise no time (§23.2)	(a) acknowledge only · (b) acknowledge **and publish the team's hours** · (c)	(b) if the business is comfortable publishing hours — it sets an honest expectation.	The first impression for every out-of-hours contact	Business owner	**NEEDS STAKEHOLDER CONFIRMATION**

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
			offer a callback request	**Never a countdown or an estimated response time**			
D-21	**Working hours per team**	None proposed	Days, window, timezone, holidays, per team (§23.1)	**We will not invent these.** Claims and Fresh Sales need not share hours. One timezone per calendar, stated explicitly	Determines after-hours behaviour and every business-hours SLA computation	Business owner + department heads	**NEEDS STAKEHOLDER CONFIRMATION**
D-23	**Business-hours SLA or 24×7, per team?**	Business-hours (§23.2)	(a) business-hours · (b) 24×7 · (c) 24×7 for Claims and Grievance, business-hours elsewhere	(a) for teams staffed only in hours. **A 24×7 target without 24×7 staffing breaches every night** — that is a staffing decision wearing an SLA costume	Sets whether overnight time counts against us	Business owner + CTO	**NEEDS STAKEHOLDER CONFIRMATION**

IMPORTANT — needed before the pilot ships

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
D-19	**Where does the designated-employee mapping come from?**	—	(a) a CRM or policy system · (b) StarLink learns it from the first conversation · (c) manual assignment by a lead · (d) no designated employees in V1	Depends on **D-03**. (b) is cheapest and self-maintaining; (a) is correct if the relationship already exists elsewhere. **(d) is a legitimate answer** — it makes routing purely queue-based and defers §21.7 entirely	Determines whether Renewals keeps its relationships	CTO + business owner	**NEEDS STAKEHOLDER CONFIRMATION**
D-22	**SLA target durations**	None proposed	Per clock (first response, resolution, escalation), per category or per team	**We will not invent these.** Also confirm one behaviour: does waiting on the **customer** stop the resolution clock? (Proposal: yes — otherwise we are	Sets what the business promises	Business owner	**NEEDS STAKEHOLDER CONFIRMATION**

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
				measured on their response time)			
D-24	**Priority definitions**	Category-derived default	(a) derived from category only · (b) a named scale with rules · (c) manual, no default	(b) with (a) as the default. Priority bands are what make "oldest first" safe (§23.4) — without them a routine enquiry queues ahead of a claim	Determines the queue order every morning	Business owner + department heads	**PROPOSED**
D-25	**Escalation thresholds, and is escalation on breach automatic?**	Warning, then breach, then optional automatic escalation (§23.6)	Warning threshold · escalation threshold · **automatic versus lead-decides**	Automatic escalation for Claims and Grievance; lead-decides elsewhere. Automatic everywhere risks escalation becoming noise the leads learn to ignore	Determines whether a late case is caught or missed	Business owner + CTO	**PROPOSED**
D-26	**Customer-visible status vocabulary**	A mapping from internal state	The customer-facing words, and which internal states collapse together	Few words, plainly meaningful — for instance *received · being looked at · waiting for you · resolved*. **Internal state is never shown** (§27.16)	What the customer sees about progress	Business owner	**PROPOSED**
D-29	**Which identity provider is authoritative in production?**	StarLink's own store in V1; HRMS later (**A-13**)	(a) StarLink's own store · (b) HRMS when ready · (c) single sign-on · (d) a managed identity provider	(a) for V1, because the HRMS is not ready and nothing else should block. The provider interface (§17.2) means the answer changes one component	Determines the employee sign-in path and its timing	CTO + IT	**NEEDS STAKEHOLDER CONFIRMATION**

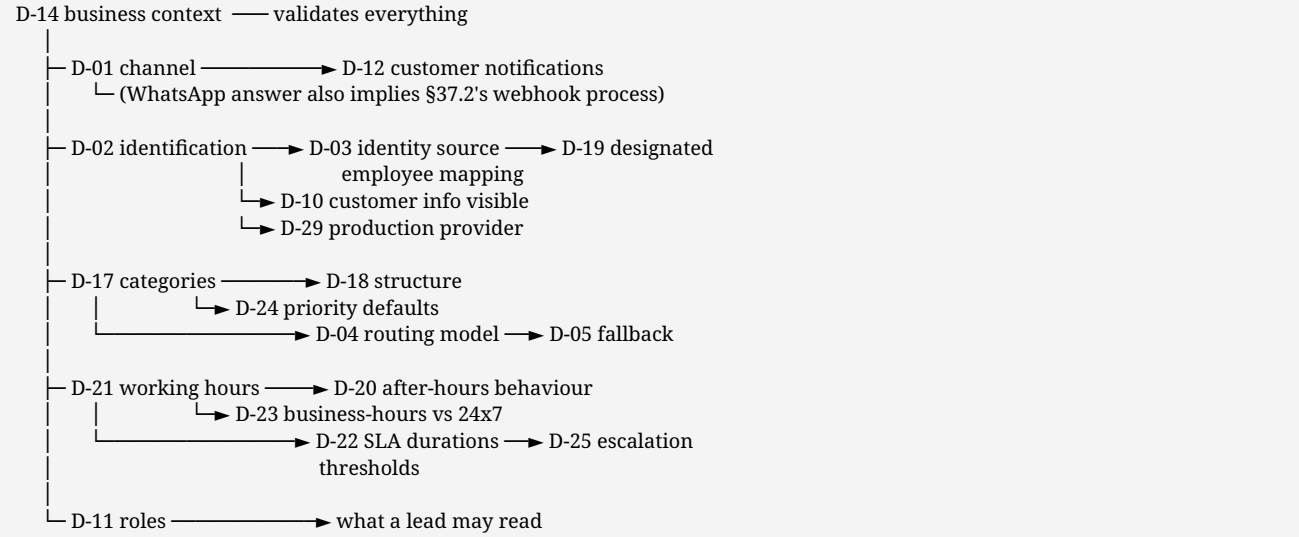
CAN BE DECIDED LATER

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
D-27	**Is a customer-quotable case or ticket number**	Not in V1	(a) none — the conversation *is* the	(a) or (c). The case already has an identifier	Customer experience, and possibly a call-centre	Business owner	**PROPOSED**

ID	Decision	Current proposal	Options	Recommendation	Business impact	Approver	Status
	required?		reference · (b) a reference shown to the customer · (c) internal only	(§22.4), so this is **a display decision, not a structural one** — it can be turned on later with no data change. A visible number also invites the customer to quote it on the phone, which implies a phone process	process		
D-28	**What is the case entity called?**	**Service Case**	Service Case · Customer Service Work Item · Case · Service Request · Ticket	*Service Case*. Deliberately **not** "Ticket" — the whole recommendation in §22.2 is that the customer never experiences a ticket, and internal vocabulary leaks into interfaces	Internal vocabulary, and how staff describe it to customers	Business owner + CTO	**PROPOSED**

44.6 Decision count and dependency

30 business decisions. The 17 from version 2.0 are unchanged in wording and status; 13 are new.



D-17 and D-21 are the two new roots. Nothing in the customer journey can be finalised without them, and both are questions only the business can answer.

45. Implementation approach

High level only. No engineering tasks — those follow architecture sign-off.

Phase 0 — Business validation

Purpose	Establish that the problem and the proposed product are the right ones
Entry	This document reviewed
Activity	Confirm **D-14** · answer **D-01, D-02, D-03, D-04** · nominate the pilot department · confirm personas (**D-11**) · start Legal on **D-06**
Exit	The four blocking decisions answered and recorded; business context confirmed; a named business owner
Produces	An updated decision register. **No code**

Phase 1 — Architecture sign-off

Purpose	Fix the technical approach against confirmed requirements
Entry	Phase 0 exit met
Activity	Revise this document against the answers · **approve or overrule each row of §39** · confirm topology and hosting · agree the non-functional targets that are currently proposals · agree the security review point
Exit	Architecture signed off; **every §39 row moved from PROPOSED to a decision**; NFR targets agreed; infrastructure owner identified
Produces	A signed architecture and a settled technology register. **No code**

Technology validation gate — new in version 2.0. Before Phase 2 begins, three things are proven in a throwaway spike rather than assumed, because each is a load-bearing assumption in §39 and each is cheap to test now and expensive to discover later:

1. **The module boundary test actually fails the build** on a deliberate violation (§18.3). Without this, the modular monolith is a naming convention.
2. **The customer build genuinely cannot contain employee code** (§19.2) — verified by inspecting the emitted bundle, not by reading the import graph.
3. **The message page examines documents proportional to what it returns** (§24.7), reproducing the 301-versus-20,000 measurement on the proposed database.

A spike that fails any of these sends the relevant §39 row back for reconsideration — which is the point of running it before implementation rather than during.

Phase 2 — V1-A, internal communication

Purpose	Prove the platform end to end, with real users and low risk
Entry	Phase 1 exit met

Delivers	§41.2
Exit	Adopted across the directory · authorization verified by test, including the negative cases · audit queryable by the compliance role · realtime degradation exercised · backup **and restore** rehearsed
Gate	Independence verified: the platform operates with NorthStar stopped (§36.2)

Phase 3 — V1-B, customer pilot

Purpose	Exercise the customer boundary at a scale where a mistake is recoverable
Entry	Phase 2 exit met · **D-01–D-04** implemented · staging mirrors production · **security review passed** · customer isolation verified by test
Delivers	§41.3, in **Support (18)** or **Claims (4)** — not Fresh Sales
Exit	Zero internal-note exposure incidents · every conversation has an owner · no conversation orphaned by absence or departure · privileged reads visible in audit · the pilot department prefers it to what they did before
Gate	An explicit go/no-go. A pilot that cannot fail is not a pilot

Phase 4 — Rollout

Purpose	Extend customer communication department by department
Entry	Phase 3 exit met and reviewed
Sequence	Case-shaped departments first (Claims, Grievance) · then relationship-shaped (Retention, Renewals) · **Fresh Sales (63) last**, because it is the largest and the most queue-shaped
Exit	All customer-facing departments migrated; channels outside StarLink retired for in-scope communication
Gate	Each department is its own go/no-go, informed by the previous one

Phase 5 — Consolidation

Retire superseded channels · answer the deferred decisions (**D-07, D-08, D-12**) with pilot evidence rather than speculation · integrate the HRMS identity adapter when the HRMS is ready · expose conversation history to the CRM framework when it exists.

---

46. Rules that must not be traded away

Scope may be cut, dates may move. These are the properties that make the product trustworthy, and trading one away silently is how a communication platform becomes a liability.

1. **A message is durable before it is delivered.** Never the reverse.

2. **Authorization is evaluated before content is read**, on every path, without exception.
3. **Participation grants that conversation. Nothing else.**
4. **An unknown permission is denied**, never treated as unrestricted.
5. **A customer can never see an internal note.** If internal status is uncertain, the send fails.
6. **A customer can never see another customer, or that one exists.**
7. **Exactly one owner per customer conversation, always.**
8. **A conversation owned by a deactivated account is reassigned**, never left orphaned.
9. **Nobody waits invisibly.** An unanswered conversation is visible, never silently held.
10. **The audit ledger is append-only.** No application path updates or deletes it.
11. **Privileged reads are audited. Ordinary internal messages are not.**
12. **Audit failure on a security-critical action fails the action.**
13. **Revocation is effective on the next request**, not at a cache expiry.
14. **A live channel dies with its session**, server-side.
15. **Realtime is additive.** No state exists only in an event.
16. **Message history is immutable.** Correction is a new message.
17. **Identity is replaceable** without touching authorization.
18. No hidden runtime dependency on NorthStar or a peer application. Shared CCS Kernel/IAM/Consent/Work/Event/Audit dependencies are explicit enterprise contracts, not duplication.
19. StarLink does not invent a parallel productivity truth. It emits governed conversation/work facts; CCS PRO may use them in its canonical productivity, SLA and decision-intelligence models.
20. **A business decision is never silently converted into a technical one.**
21. **A customer who contacts us out of hours is acknowledged, stored and queued.** Never lost, never silently held, and never given a response time nobody is rostered to meet.
22. **Internal operational metadata never reaches a customer.** SLA state, escalation level, priority, assignment history and routing information are invisible on every path, fail-closed.
23. **Ownership is never inferred from a network connection.** A dropped socket is not an absence.
24. **SLA exists to get a customer answered, never to score an individual.**

Where this leaves us

What is proposed: one platform, two surfaces, one foundation — for a company where three quarters of the staff talk to customers on channels the company does not own.

What is designed: the architecture in Part II, derived from the requirements in Part I rather than assembled from a feature list. Identity replaceable, authorization separate from authentication, realtime justified event by event, audit split so retention and integrity can both be honoured.

What is proposed, and needs your approval: the technology. TypeScript end to end, Next.js on both surfaces, NestJS, MongoDB, Socket.IO, an object-storage abstraction and adapter-based notification — with Redis, a broker, a search engine and orchestration all deliberately left out until something makes them necessary. Twenty rows in §39, twelve records in §40. **Including the modular monolith, which version 1.0 called decided and this version returns to PROPOSED** so you can overrule it.

What is not decided, and is not ours to decide: sixteen items in §44, five of which the customer half of V1 waits on. The channel decision carries no recommendation on purpose.

What we recommend: confirm the business context, answer the four blocking decisions, and start V1-A — which no open decision blocks.

> ***Nothing in this document is approved, and no production code is justified by it.*** *A prototype exists from earlier evaluation and is deliberately set aside. It was built in a* ***different stack*** *from the one proposed here (§14.3), so its* ***code does not carry forward*** *— only two findings do: the independence architecture holds, and the authorization model catches the defect it was designed to catch, because an early version reproduced that defect and a test caught it. It did not prove the product, and it does not pre-commit the stack.* ***The product starts with your answers; the build starts with §39.***

Part IV — CCS/CY Brain Alignment & Production-Scale Conversation OS

> *v2.2 precedence rule: Sections 47–68 are additive alignment requirements. Where a v2.1 statement conflicts with CCS PRO canonical ownership, CY Brain constitutional boundaries, privacy controls, or the production-scale requirements below, this Part IV governs StarLink. It does not create a second Customer, Consent, Work, Event, Audit, Policy, Claim or Opportunity truth.*

47. Architecture authority, gap analysis and reconciliation

StarLink v2.1 is strong as a self-contained chat/service design, especially around durability-before-delivery, customer/staff boundary, audit of privileged reads, queue visibility, cursor pagination, ownership history and explicit failure behaviour. The principal gap is not missing chat functionality; it is enterprise alignment. CCS PRO already defines the company-wide Conversation OS, Unified Inbox, Work Orchestrator, canonical customer/business objects, contact governance, AI handoff and omnichannel continuity. CY Brain defines command/intelligence/assurance over those shared engines. StarLink must therefore become the implementation architecture for the Conversation OS bounded context, not a parallel platform foundation.

Area	v2.1 risk/gap	v2.2 treatment
Identity / customer	Local StarLink identity/customer decisions could drift from CCS.	Central IAM + canonical Person/Customer resolution. StarLink keeps only channel/session/participant projections and external-channel identities.
Routing / ownership	Conversation-specific assignment may become a second allocator.	All executable customer work resolves through CCS Work Orchestrator; StarLink supplies channel/session context and consumes Assignment/Reservation decisions.
Consent / DND / purpose	Channel availability can be mistaken for permission.	Every outbound/re-engagement action invokes canonical Consent & Contact Governance at execution time.
Audit	Local append-only audit can become a second enterprise ledger.	Security-critical and business ownership events publish to CCS Event/Audit contract. Local technical delivery log is not enterprise audit truth.
Realtime scale	Single process/in-process hub remains structurally fragile at 500+ simultaneous users or multiple instances.	Shared realtime backplane from customer pilot: stateless API, horizontally scalable realtime nodes, Redis-compatible adapter/counters, bounded queues and connection draining.
Async work	Notification outbox exists, but AI, indexing, media, webhooks and channel delivery can still compete with synchronous chat.	Durable async job fabric with explicit queues, retry/DLQ, idempotency and backpressure.
Omnichannel	Website-centric journey; CCS expects one conversation across approved channels.	Channel adapter layer + ConversationIdentity bridge; same business conversation may continue across Web/App/WhatsApp/Email/Call without duplicating work.
AI	Mostly future/implicit.	AI triage, summary, reply assistance,

		knowledge retrieval and quality support are first-class but evidence-bound and human-governed.
Privacy	Good customer/staff isolation but not full DPDP lifecycle integration.	Purpose-bound visibility, minimisation, PII vault/token use, DSR/retention/legal-hold hooks and processor/channel controls.

48. Canonical ownership map — one truth, clear bounded context

Object / capability	Authoritative owner	StarLink responsibility
Person / Customer / household / organisation	CCS Customer Graph / Identity	Reference canonical IDs; maintain channel identity links and verified-session binding only.
Employee identity, role, scope, delegation, revocation	Central IAM	Consume authorization decisions; cache only bounded, versioned projections for latency.
Consent / DND / communication purpose / restrictions	CCS Consent & Contact Governance	Request eligibility before outbound send, bot re-engagement, channel handoff and proactive notifications.
Opportunity / Renewal / Claim / Complaint / Service Case / Booking / Policy	CCS domain engine	Link conversation to the business object; never recreate its state machine.
Work assignment, reservation, capacity, SLA execution	CCS Work Orchestrator / SLA runtime	Expose inbox/queue; submit routing context; consume authoritative assignment/work events.
Conversation / Message / participant-local state	StarLink Conversation Service	Authoritative message/thread content, channel delivery mapping, unread/read state, internal-note boundary.
Enterprise Event / Audit truth	CCS Event & Audit Ledger	Transactional outbox publishes material events; local operational logs remain diagnostic only.
Search index for chat content	StarLink serving index under CCS authz	Derived projection; rebuildable; never authority for permission or retention.
Brain dashboards / decision intelligence	CY Brain	Consume conversation/work/security metrics and exceptions; Brain never becomes chat message store.

> *Constitutional rule: Conversation is an interaction container. Work/Case/Opportunity is the business obligation. A chat may create, attach to, or update work through governed commands, but chat state must never become the only record that a customer needs service, a claim exists, a payment is pending or a complaint is unresolved.*

49. Unified Conversation model — one thread can cross channels without losing context

The canonical StarLink conversation model is channel-neutral. A Conversation has a business subject and participants; ChannelSessions attach transport-specific identity, delivery and session facts. This allows a customer to start on the website, continue on WhatsApp/app/email, call the RM, and return to the website without producing four independent cases.

Entity	Key fields / behaviour
Conversation	conversation_id, customer/person ref, conversation_type, business_object_links[], purpose, sensitivity, lifecycle, current work/case refs, created_at, last_activity_at.
ChannelSession	channel, external_thread_id, verified identity level, start/end, transport rules/window, adapter version, delivery capability, processor/vendor ref.
Participant	principal/customer/team/system/AI, role, scope, added_by, effective_from/to, reply authority, internal-only flag.
Message	immutable id, conversation_id, channel_session_id, sender, message_class, internal/customer-visible boundary, body/ref, client idempotency key, created_at, reply_to, edit/redaction lineage.
DeliveryReceipt	message x destination/channel, queued/sent/delivered/read/failed/unknown, provider IDs, timestamps, retry state.
ConversationLink	Opportunity/Policy/Claim/Complaint/Booking/Payment/Task/ Callback/WorkItem links with relation type and effective dates.
OwnershipEpisode	authoritative work/assignment ref plus local display projection; never overwrite historical ownership.

- Website chat, app chat and approved external channels reuse the same Conversation where business semantics and identity permit; transport threads remain separate ChannelSessions.
- Channel switching is not copy/paste. A HANDOFF event records origin channel, destination channel, reason, customer confirmation/eligibility, and continuity token.
- If identity confidence changes, access narrows; historical customer messages are never exposed merely because a new channel claims the same phone/email.
- Internal employee chat may use the same message infrastructure but a different ConversationType and security policy; internal notes in customer conversations are permanently non-customer-deliverable by schema, not only by UI styling.

50. Conversation vs Service Case vs Work Item — the hybrid model

StarLink should keep the excellent v2.1 conclusion that the customer experiences a conversation, not a ticket form. Internally, however, CCS needs case/work discipline. The right model is Conversation-first UX + canonical Case/Work behind it.

Customer situation	Conversation behaviour	Canonical business object
New sales question	Chat/AI discovers need and may warm-transfer.	Create/resolve Opportunity only when eligibility and intent threshold are satisfied.
Existing policy service	Continue thread; agent sees policy context.	Create/attach Service Case / Endorsement work where action is required.
Claim	Conversation is communication surface.	Claim remains authoritative object; claim deadlines and documents never live only in chat.
Complaint / grievance	Immediate specialist routing and restricted visibility.	Complaint/Grievance Case with regulatory SLA and audit.
Payment/issuance blocker	Chat can explain/request document.	Existing Booking/Payment/PICV/Issuance

		work item is updated/linked.
Simple FAQ resolved by AI	Conversation may close without case.	No synthetic case created.

51. Website → AI → agent journey, including surge and fallback

1. Widget loads in anonymous mode with only public-approved knowledge. No customer history is fetched before identity/authorization.
2. Customer chooses category or types naturally. AI may classify intent, language, urgency and likely product, but category/routing remains explainable and correctable.
3. Identity is established only when needed. Existing customer resolution uses CCS Identity/Customer Graph; unknown visitor receives a temporary interaction identity until minimum data is legitimately captured.
4. Conversation Orchestrator checks active Conversation/Case/Opportunity to avoid duplicate work, then asks Work Orchestrator for next eligible handling route.
5. Routing considers intent, product, relationship owner/preference, skill/certification, language, current presence, declared schedule, active workload, chat-concurrency capacity, SLA/urgency, customer value/risk and queue fairness.
6. If AI can safely answer, it responds using approved knowledge and permitted context. If confidence is low, a hard rule applies, or customer asks for a person, AI hands off with summary + collected facts + evidence links.
7. If an eligible agent is immediately available, reserve a chat capacity slot atomically before assignment. The reservation has TTL; on expiry or disconnect it returns to queue.
8. If all agents are busy, persist the message first, queue visibly, send an honest acknowledgement, and show position/status only if policy permits. Do not promise a fabricated wait time.
9. During surge, admission control protects the system: new chats may remain in asynchronous queue/AI self-service while realtime agent reservations are bounded. No request is silently dropped.
10. When assigned, customer context appears in one agent workspace: Conversation, Customer 360 summary, active Opportunity/Policy/Claim/Case, last commitments, consent/contact constraints, AI summary and next-best actions. Customer never repeats what the system already knows.

52. Realtime production topology — baseline for 500+ simultaneous users

500 simultaneous users is not a large internet-scale number, but it is large enough that a single in-process realtime hub should not be an architectural assumption. More importantly, website campaigns and external-channel bursts can create connection and message spikes that are unrelated to headcount. v2.2 therefore makes multi-instance capability a V1-B production requirement while keeping the service topology simple.

Layer	v2.2 production baseline	Scale/failure property
Edge / load balancer	TLS termination, WebSocket upgrade, HTTP/2/3 where supported, rate limits, bot/WAF rules. Sticky sessions only if Socket.IO transport requires them; do not use affinity as correctness.	Absorbs connection spikes and protects origin.
API service	Stateless NestJS instances; REST/command path separate from long-lived socket lifecycle.	Horizontal scale; zero-downtime deploy.
Realtime gateway	Socket.IO/WebSocket nodes with	Connections can move/reconnect

	subscribe-time authz, session-version checks, presence leases and graceful drain.	without losing authoritative state.
Shared realtime backplane	Redis-compatible managed service for Socket.IO adapter/pub-sub, presence leases, ephemeral counters and distributed rate limits.	Cross-instance fan-out and coordination.
Durable async job fabric	Managed queue/broker (e.g., SQS-class/RabbitMQ-class; choice by ADR) for notifications, webhooks, AI jobs, indexing, media scan, transcript/summarisation and integration retries.	Buffers spikes; independently scalable workers; DLQ.
Conversation store	MongoDB remains acceptable for message/thread documents if benchmarks pass; replica set, majority durability for acknowledged sends, explicit indexes and backup/PITR.	Durable history and paging.
Canonical business services	CCS APIs/Event Fabric for IAM, Customer, Work, Consent, Audit and business objects.	No duplicated business truth.
Object storage	Direct/pre-signed upload/download with malware/DLP pipeline and quarantine.	Application servers do not proxy large files under load.
Search	Derived index only when corpus/latency trigger is met; initial Mongo search can remain if measured.	Search can scale independently; rebuildable.

> *Latest-architecture principle applied: keep live delivery realtime, but move anything that does not need an immediate answer off the request path. Queue-driven async work absorbs bursts and prevents notification, AI, indexing or third-party latency from slowing message persistence.*

53. Message correctness, ordering and delivery semantics

- Persist-before-publish remains non-negotiable. HTTP/message command returns success only after durable storage under the configured write concern.
- Every send uses client_message_id/idempotency_key unique within sender/session. Retry after timeout returns the original message.
- Assign a server-side monotonically ordered conversation sequence or deterministic ordering key; timestamps alone are insufficient under concurrency.
- Realtime event is a hint containing IDs/version/sequence, not the authoritative body. Client detects sequence gaps and re-fetches.
- Exactly-once transport is not assumed. Durable commands are idempotent; queues are at-least-once; consumers deduplicate on event/job id.
- Transactional outbox couples message/business state commit to event publication. A DB commit without downstream event eventually heals through outbox replay.
- Delivery to external channels distinguishes ACCEPTED_BY_PROVIDER, DELIVERED, READ where available, FAILED, UNKNOWN and SUPERSEDED. UNKNOWN triggers reconciliation rather than optimistic success.
- Message edits/deletes are governed append-only corrections/tombstones according to policy; original compliance evidence is preserved where retention/legal hold requires it.

54. Scale envelope, SLOs and load assumptions

Do not size only to 195 employees. The architecture is sized to simultaneous employees + customers + channel sessions. Initial acceptance envelope should be explicit and synthetic-testable, then revised with production telemetry.

Dimension	Initial engineering envelope	Acceptance expectation
Authenticated employees online	500 concurrent	No functional degradation; presence/read/typing bounded.
Concurrent customer sessions	1,500 concurrent website/app sessions during burst	Queue and AI self-service remain available; realtime nodes autoscale or retain headroom.
Active agent chats	Configurable 3–6 per chat-qualified agent by role/complexity	Work Orchestrator prevents unsafe over-assignment.
Message ingress burst	100 msg/s for 5 min synthetic spike	p95 persist acknowledgement within target; no loss/duplication.
Realtime fan-out	2,000 connected sockets; reconnect storm test at 50% within 60s	Backplane stable; jitter/backoff prevents thundering herd.
Message history	10M+ messages planning horizon	Cursor paging remains bounded; index regression alerts.
Attachments	Direct object-store transfer; application metadata only	Large uploads do not exhaust API memory/connections.
Queue burst	10x normal notifications/webhooks/AI jobs	Backlog drains within defined recovery SLO without impacting chat sends.

Suggested SLO categories (exact numbers approved after benchmark): message durability availability; p95/p99 send acknowledgement; p95 thread open/page; realtime event latency when connected; routing/assignment latency; queue age by priority; notification delivery lag; recovery time after node loss; zero unauthorized cross-customer reads; zero orphaned customer work.

55. Capacity-aware routing, agent concurrency and fairness

Signal	Use
Agent state	AVAILABLE / BUSY / BREAK / OFFLINE / AFTER_CALL / TRAINING; presence is a hint, schedule and work state are authoritative.
Channel capacity	Voice consumes ~1 slot; chat consumes weighted slots; email/asynchronous work consumes different capacity. Weights configurable by role/team.
Conversation complexity	Claims/grievance/high-risk cases may consume more than one chat capacity unit; simple sales FAQ less.
Relationship continuity	Preferred relationship owner gets bounded tolerance, then eligible backup before SLA/customer suffers.
Priority	Customer-requested, complaint/claim, payment, renewal urgency, hot sales intent and SLA risk can outrank ordinary queue work.
Fairness	Age/SLA anti-starvation; no queue can remain permanently low priority.
Reservation	Atomic reserve → assign → accept/start; TTL and release on failure. Prevents double-claim.

Surge control	Per-team max in-flight reservations, tenant/global concurrency ceiling, rate limiting and queue backpressure.

Agent concurrency should never be a hard-coded “5 chats each”. Start with a policy envelope, then adapt from measured response latency, conversation complexity, abandonment, SLA and customer outcomes. AI may recommend capacity changes; rules/Work Orchestrator enforce them.

56. Omnichannel adapter architecture and channel continuity

Channel	Adapter responsibilities	Special rules
Website / App	WebSocket/session, authenticated or temporary identity, browser lifecycle, push fallback.	Reconnect/resume token; no reliance on tab staying open.
WhatsApp	Webhook verification, inbound/outbound IDs, templates/session-window rules, media, delivery/read receipts.	Consent/purpose/contact-governance at send time; processor/vendor recorded.
Email	Inbound threading, aliases/shared mailbox identity, attachments, outbound provider IDs.	Provider remains transport; CCS owns business context and DLP/audit.
SMS	Short transactional/status messages and approved fallback.	Not a rich conversation substitute; channel policy applies.
Voice / telephony	Link call session, IVR intent, transcript/summary refs, callback promise.	Call can attach to same Conversation/Case; raw recording governed separately.
AI bot	AI participant with explicit identity and handoff boundary.	No pretending to be human; governed tools/data and evidence.

- Channel adapters normalize inbound envelopes into Conversation commands/events; they do not own customer/business semantics.
- Duplicate provider webhooks are expected and idempotently absorbed.
- Inbound customer messages are customer-intent signals and may temporarily override promotional contact suppression where law/policy allows; this decision remains in canonical Contact Governance.
- If a channel provider is down, other channels may be offered only when purpose, consent, urgency and customer preference allow. No silent channel hopping.

57. AI-native Conversation OS — useful, governed and non-duplicative

Capability	Behaviour / guardrail
Intent + entity extraction	Extract category, product/policy reference, urgency, language and requested action with confidence; deterministic routing rules own final activation.
AI self-service	Answer approved public/authorized knowledge; use RAG over versioned knowledge and policy content; cite source internally and escalate when uncertain.
Human handoff summary	Concise summary, facts collected, unanswered question, sentiment/urgency, relevant objects and evidence. Never ask

	customer to repeat.
Agent Copilot	Suggested replies, policy/FAQ retrieval, object lookup, next action, translation, tone simplification, form/document checklist.
Conversation-to-work	AI may propose Create Case/Task/Callback/Opportunity; user/rule confirms where required. No hidden second workflow.
After-call / long-thread summary	Structured summary attached to conversation and object; source messages remain authoritative.
Quality / risk assist	Detect possible mis-selling, prohibited disclosure, unresolved commitment, complaint signal, missing disclosure or sensitive-data exposure.
Supervisor assist	Cluster queue themes, identify widespread issue, recommend staffing/re-routing; no unsupported individual performance judgement.
Customer sentiment	Use as soft signal, never sole basis for adverse/customer-impacting decision.
AI memory	Customer facts enter Golden Profile only via governed extraction/confirmation/provenance; model context itself is not a database.

> *AI output is recommendation or derived interpretation, not business truth. Store model/version, evidence, confidence and acceptance/rejection for high-impact suggestions. Hard compliance, entitlement, payment, policy and case state remain deterministic authority.*

58. DPDP, privacy, security and sensitive-conversation controls

- Every customer conversation is tagged with purpose, source/channel, sensitivity and business-object context. Possession of data never equals permission for every use.
- Normal agents use customer/contact tokens and click-to-contact; raw mobile/email/PII exposure is separate capability, purpose-bound, time-limited where appropriate and audited.
- Claims, medical, grievance/legal and financial/payment information use stricter visibility than ordinary sales chat. Customer 360 may reveal the existence/status of a restricted case without exposing its sensitive detail.
- Internal notes are schema-enforced staff-only content. External adapters reject any message whose visibility class is not explicitly customer-deliverable.
- Outbound WhatsApp/SMS/email/voice rechecks Consent/DND/purpose/contact pressure immediately before send, not only when the conversation was created.
- Attachments inherit conversation/object sensitivity but also get their own classification, malware/DLP result, retention and download policy.
- Data Principal requests, correction/erasure, legal hold and retention events propagate to conversation indexes, exports, AI/vector stores and processors according to the central privacy control plane.
- Operational logs never contain message bodies, raw contact data, auth tokens or document payloads. Privileged reads/searches/downloads are auditable.
- Session revocation, employee exit and role change invalidate access on the next request/subscription authorization check; realtime sockets are closed/re-authorized.

59. Attachment, media and DLP pipeline

1. Client requests upload intent with conversation/object/purpose metadata.
2. Authorization and allowed MIME/size policy evaluated before signed upload URL is issued.
3. Client uploads directly to object storage quarantine prefix; application nodes never proxy the full file by default.
4. Async scanner validates MIME by content, malware, archive bombs, macro risk, DLP/PII classification and optional OCR metadata.
5. Clean asset moves/logically promotes to approved state; infected/suspicious remains quarantined and cannot be delivered.
6. Message references immutable attachment_id, not a permanent public URL.
7. Download uses short-lived signed URL after fresh authorization; high-risk download is audited/watermarked where policy requires.
8. Retention/legal hold/deletion operate on the attachment object and its derived OCR/search/vector artifacts.

60. Search, knowledge and long-thread usability

- Search is authorization-first. Build the permitted conversation/object scope before applying text search; never query a global text index and filter after disclosure.
- Cursor pagination remains standard. Deep offset paging is prohibited.
- When corpus/latency trigger is crossed, derived OpenSearch-class index may be introduced with customer/internal visibility partitions, retention propagation and rebuildability.
- Agent thread view supports AI summary, pinned/structured facts, object timeline, unresolved commitments and “jump to evidence” rather than forcing users to scroll thousands of messages.
- Knowledge retrieval uses a separate governed knowledge corpus (product wording, SOP, FAQs, approved templates). Customer conversations are not automatically training/knowledge documents.

61. Observability, SRE and management control

Signal family	Minimum telemetry
Realtime	connected sockets, auth failures, reconnect rate, disconnect reasons, event latency, backplane publish errors, per-node connection distribution.
Messaging	send ack latency, persist failures, duplicate/idempotent retries, sequence-gap recovery, message rate, oversized payload rejects.
Queues	depth, oldest age, enqueue/dequeue rate, retry count, DLQ, priority backlog, worker saturation.
Routing	unassigned count, reservation expiry, assignment latency, orphan/inactive-owner count, queue age by team, capacity mismatch.
Database	p95/p99 query latency, examined/returned ratio, connection pool saturation, replication lag, storage/index growth, backup/PITR health.
Channels	webhook lag, provider errors, delivery UNKNOWN, template

	rejection, rate limits, circuit-breaker state.
AI	latency, token/cost, confidence, escalation rate, tool errors, accepted/rejected suggestions, hallucination/quality samples.
Security/privacy	403 bursts, unmask/download/search anomalies, cross-customer denial attempts, DLP/malware events, privileged-read volume.
Customer	first response, waiting age, abandonment, reopen, unresolved commitments, channel handoff success, CSAT where captured.

Use OpenTelemetry-compatible traces/metrics/log correlation as the preferred standard for distributed request/job visibility once more than one runtime component exists. This does not mean storing message content in traces.

62. Failure, resilience and graceful degradation

Failure	Required behaviour
One API/realtime node dies	Load balancer removes node; sockets reconnect with jitter; clients re-fetch gap; no message loss.
Redis/backplane unavailable	Existing node-local connections may degrade; new cross-node fan-out/presence marked degraded. Durable send still writes DB; clients recover by fetch. Hard distributed rate/authorization decisions fail safe per policy.
Async queue unavailable	Message persistence remains available; outbox accumulates safely within bounds; nonessential AI/index/notification jobs pause; alert before storage pressure.
Conversation DB unavailable	Fail visibly; no fake empty inbox/history. Readiness removes unhealthy instances.
CCS IAM unavailable	Existing narrowly cached session may continue only within approved short policy window if allowed; privileged/new authorization fails closed.
CCS Consent/Contact Governance unavailable	No new proactive/outbound customer contact that requires an eligibility decision. Inbound servicing can be handled according to explicit fail-safe policy without inventing permission.
CY Brain unavailable	Chat execution continues; command dashboards/analytics/AI recommendations may degrade. Brain is not transaction authority.
External channel outage	Persist inbound/outbound intent state, reconcile provider status, expose degradation; do not mark delivered.
AI unavailable	Human workflow remains fully usable. No case or message becomes blocked solely because AI is down.
Object storage/scanner unavailable	Text conversation continues; file path explicitly unavailable/queued; never bypass scanning to “keep moving”.

63. Data lifecycle, retention, archival and legal hold

- Hot conversation store contains active/recent threads; archival tier is introduced before sharding when working set/index growth justifies it.
- Archive remains readable through same API/authz, with higher latency communicated. Moving to archive never changes conversation identity.

- Retention is policy-driven by conversation type, linked business object, regulatory/legal requirement, source/purpose and legal hold — not one global delete period.
- Internal social-like chatter can have shorter retention than customer complaint/claim evidence. The policy engine supplies the effective rule.
- Deleted/erased eligible PII is removed or de-identified from message/search/AI derivatives while audit may retain minimal non-PII evidence where required.
- Backups/PITR have separate retention and documented erasure handling; restore procedures re-apply post-backup deletion/consent events before restored data becomes active.

64. Configuration, feature flags and safe rollout

Configuration domain	Examples
Channel policy	enabled channels, provider, templates, media limits, service windows, fallback.
Routing policy	team/category mapping, skills, relationship tolerance, capacity weights, reservation TTL, fallback queues.
SLA/business calendar	team calendar, timezone, holidays, first-response/resolution/wake policies.
AI policy	models, allowed tools/data, confidence thresholds, auto-answer topics, human-approval classes, cost limits.
Privacy/security	visibility classes, PII reveal, retention, DLP, download, legal hold, audit rules.
Realtime/scale	connection limits, per-user/device limits, rate limits, queue bounds, reconnect policy, circuit-breakers.

Configuration is versioned, effective-dated and previewable. High-risk changes run blast-radius preview/simulation before publish. Feature flags support canary by team/channel/tenant, but no flag may bypass a constitutional security/compliance invariant.

65. Load, chaos, security and golden-journey test lab

Test family	Must prove
Load	500 employees + 1,500 customer sockets; target message rate; routing burst; attachment metadata; thread paging; unread counts.
Reconnect storm	Node restart/deploy causes mass reconnect with jitter; no duplicate messages; backlog recovers.
Multi-instance correctness	Message sent through node A reaches recipient on B; distributed claim/reservation/rate-limit remains correct.
Queue overload	10x async job burst; bounded backlog, no synchronous chat regression, DLQ/retry visible.
DB failover	Replica primary loss; explicit degradation then recovery; acknowledged sends meet durability policy.
Channel webhook duplication/out-of-order	Idempotency/reconciliation prevents duplicate customer messages/state corruption.
Authorization race	Role revoked/employee exits while socket open; next

	read/send/subscription denied and socket removed.
Internal-note leak	Property test: no customer API/adapter can serialize/deliver INTERNAL visibility message under any route.
Cross-customer isolation	Fuzz IDs/cursors/search/attachments; zero existence leakage.
AI red-team	Prompt injection, malicious attachment text, sensitive-data request, tool overreach, unsupported policy answer and human-handoff correctness.
Privacy lifecycle	Consent withdrawal/erasure/legal hold propagates to outbound eligibility, search, AI derivatives and processors as designed.
Employee exit	All executable customer work is reassigned/covered; historical attribution remains; zero inactive-owner conversations.

66. Recommended delivery phases — simple first, scale-safe from the start

Phase	Scope	Exit gate
0 — Architecture & spikes	CCS contracts, message store benchmark, multi-instance Socket.IO+Redis spike, queue/outbox spike, security boundary tests.	Measured viability; no unresolved ownership conflict.
1 — Internal Conversation OS	1:1/groups/object-linked chat, attachments, search, unread, notifications, IAM integration, audit events.	Adoption + security + reliability gates.
2 — Website customer pilot	Website widget, category/AI intake, identity, Work Orchestrator routing, queue, SLA, human handoff, DPDP controls.	Load/chaos/security pilot gate; 500+ concurrent envelope proven in staging.
3 — Omnichannel	WhatsApp/app/email/voice linking, unified agent inbox, channel reconciliation.	One-conversation continuity and provider-failure tests.
4 — AI assist & automation	RAG, summaries, smart replies, conversation-to-work, quality/risk signals.	Evidence/quality/permission benchmarks; deterministic fallback.
5 — Scale optimisation	Separate realtime nodes/workers/search/archive/read replicas only when telemetry triggers.	SLO-driven, not fashion-driven.

67. v2.1 decisions/statements explicitly superseded or clarified

v2.1 statement	v2.2 decision
Redis deferred until after multi-instance trigger.	For V1-A developer/local single-node it may be absent; for V1-B production and 500+ concurrent-user acceptance, shared Redis-compatible realtime backplane/counters are baseline.
No message broker in V1.	No heavyweight Kafka requirement. A durable managed async queue/job fabric is baseline for notifications, integrations, AI/index/media tasks and spike absorption.

StarLink identity store is V1 source of record.	Superseded for enterprise rollout. Central IAM is authority; StarLink may hold only bootstrap/dev projection behind a provider contract until IAM cutover.
StarLink audit store as complete audit.	Local audit may exist for deployment independence, but enterprise security/business events must publish into the canonical CCS Event/Audit contract; no competing compliance truth.
Exactly one owner per conversation.	Keep as UX projection where applicable, but executable responsibility belongs to canonical Work/Case ownership. Participants, temporary cover and historical ownership episodes remain distinct.
No runtime dependency on any internal platform.	Clarified: no NorthStar/peer-app dependency. CCS kernel services are intentional platform dependencies; CY Brain is not required for transaction correctness.
StarLink not a productivity instrument.	Keep local principle: no vanity/leaderboard scoring inside chat. Emit facts to CCS, where governed KRA/KPI/productivity models may consume them.
Website chat is primary customer channel.	Website remains important intake, but architecture is channel-neutral and designed for approved WhatsApp/App/Email/Voice continuity.

68. Production architecture acceptance gate

> *StarLink v2.2 is not "production ready" because the document is detailed. Production acceptance is evidence, not prose.*

1. Authority gate: no duplicate Person/Customer/Consent/Work/Case/Event/Audit authority; CCS/CY Brain contract map approved.
2. Correctness gate: persist-before-publish, idempotency, ordering/gap recovery, atomic assignment and transactional outbox tests pass.
3. Scale gate: agreed 500+ concurrent employee / customer envelope, burst and reconnect tests pass with headroom.
4. Resilience gate: node, Redis/backplane, queue, DB, object store, CCS service and provider failure tests behave exactly as documented.
5. Security gate: cross-customer isolation, internal-note leak property test, session revocation, attachment access and privileged-read audit pass.
6. Privacy gate: purpose/consent/minimisation/retention/DSR hooks and downstream processor/channel controls are evidenced.
7. Operational gate: dashboards/alerts exist for sockets, queue age, routing orphan count, DLQ, DB/query health, channel errors and security anomalies.
8. Business gate: routing categories, team calendars, SLA rules, fallback/cover rules, agent concurrency and customer-visible statuses are configured and signed off.
9. AI gate: human fallback works with AI entirely disabled; RAG/tool permissions and quality samples pass before autonomous self-service expands.
10. Rollback gate: canary/rollback and schema compatibility tested; no deployment requires a non-reversible breaking migration.

v2.2 Final architecture position

StarLink should be built as CoverYou's specialised Conversation OS implementation: one durable conversation fabric for internal and customer communication, plugged into CCS PRO's canonical Customer, Work, Consent, IAM, Event/Audit, Notification and domain engines, and observed/optimised by CY Brain. The employee experience remains WhatsApp-simple; the backend is event-driven, privacy-aware, capacity-aware, multi-instance-ready and failure-visible. 500 users chatting at once is an ordinary tested operating point, not an emergency architecture change.

> *One conversation fabric. One company context. One canonical business truth. Realtime for experience; durable state for correctness; queues for spikes; CCS for authority; CY Brain for command and intelligence.*